Exploratory Data Analysis - Project

Student Performance (Mathematics) Dataset

Course: Exploratory Data Analysis | **Submitted by:** Kotni Subramanyam - 23BDS0181

Dataset: `student-mat.csv` (395 students x 33 attributes: demographics, family background, study habits, alcohol use, absences and three grade periods G1, G2, G3).

Objective: use every technique practised in the lab assessments to understand what is associated with students' final grade (G3), and to check how reliable the data and the conclusions are.

Youtube link https://youtu.be/ByXBCLcaTnM

In [1]:
```
# ====================================================================
# 1. SETUP - imports and global settings
# ====================================================================

# Import system utilities for module referencing
import sys
# Import warnings module to suppress unnecessary warning messages
import warnings

# Suppress all warnings so output stays clean during analysis
warnings.filterwarnings("ignore")

# Print confirmation that setup is complete
print("=" * 78)
print("CELL 1: SETUP COMPLETE")
print("Environment configured: Warnings suppressed, system libraries ready.")
print("=" * 78)
```

```
==============================================================================
CELL 1: SETUP COMPLETE
Environment configured: Warnings suppressed, system libraries ready.
==============================================================================
```

In [2]:
```
# ====================================================================
# HELPER FUNCTIONS (paths, loading, cleaning, save figure/table)
# All helper functions are defined here so every subsequent cell can use them.
# No external R datasets (AirPassengers, mtcars) are used - everything
# relies on student-mat.csv.
# ====================================================================

# Path handling for locating CSV files and saving results
from pathlib import Path

# Matplotlib for plotting
import matplotlib
```

```
# Set non-interactive Agg backend if running in a terminal/headless script environm
if "ipykernel" not in sys.modules:
    matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
# NumPy for numerical operations (arrays, math)
import numpy as np  # noqa: E402
# Pandas for tabular data manipulation
import pandas as pd  # noqa: E402
# Seaborn for statistical visualisation built on top of matplotlib
import seaborn as sns  # noqa: E402

# ---------------------------------------------------------------
# Directory setup
# ---------------------------------------------------------------

# ROOT is the current working directory (where the script is run from)
ROOT = Path.cwd()

# Look for student-mat.csv in either the root folder or a data/ subfolder
DATA_DIR = next((p for p in (ROOT, ROOT / "data") if (p / "student-mat.csv").exists

# Create output directories for saving figures, tables, and logs
RESULTS = ROOT / "results"          # all results go under this folder
FIG_DIR = RESULTS / "figures"       # saved PNG figures
TAB_DIR = RESULTS / "tables"        # saved CSV tables
LOG_DIR = RESULTS / "logs"          # log files

# Create all output directories if they don't already exist
for _d in (FIG_DIR, TAB_DIR, LOG_DIR):
    _d.mkdir(parents=True, exist_ok=True)

# Path to the raw student CSV file
STUDENT_CSV = DATA_DIR / "student-mat.csv"
# Path where the cleaned version will be saved
STUDENT_CLEAN_CSV = DATA_DIR / "student-mat-clean.csv"

# ---------------------------------------------------------------
# Column groups (same split used in the R labs)
# ---------------------------------------------------------------

# All numeric columns in the student dataset
NUMERIC_COLS = ["age", "Medu", "Fedu", "traveltime", "studytime", "failures",
                "famrel", "freetime", "goout", "Dalc", "Walc", "health",
                "absences", "G1", "G2", "G3"]

# All categorical (text) columns in the student dataset
CATEGORICAL_COLS = ["school", "sex", "address", "famsize", "Pstatus", "Mjob",
                    "Fjob", "reason", "guardian", "schoolsup", "famsup", "paid",
                    "activities", "nursery", "higher", "internet", "romantic"]

# A smaller subset of numeric columns used for quick summaries
CORE_NUM = ["age", "studytime", "failures", "absences", "G1", "G2", "G3"]

# Set the default seaborn theme for all plots (white grid background, notebook sizi
sns.set_theme(style="whitegrid", context="notebook")
```

```python
# ---------------------------------------------------------------
# Registry: tracks which module is active and what figures/tables it produces
# ---------------------------------------------------------------
_REG: dict = {"module": "unassigned", "figures": {}, "tables": {}}


def set_module(name: str) -> None:
    """Set the current module name for labelling saved outputs."""
    _REG["module"] = name
    # Ensure a list exists for this module's figures and tables
    _REG["figures"].setdefault(name, [])
    _REG["tables"].setdefault(name, [])


def registry() -> dict:
    """Return the global registry dictionary."""
    return _REG


def section(title: str) -> None:
    """Print a visually prominent section header."""
    print("\n" + "=" * 78 + f"\n{title}\n" + "=" * 78)


def save_fig(name: str, fig=None, dpi: int = 130) -> Path:
    """Save current (or given) figure to results/figures/<module>_<name>.png.

    Also displays the figure inline (for Jupyter) and then closes it
    to free memory.
    """
    # Use the current figure if none was passed
    fig = fig or plt.gcf()
    # Build the output path using the current module prefix
    path = FIG_DIR / f"{_REG['module']}_{name}.png"
    # Apply tight layout to avoid clipping labels
    fig.tight_layout()
    # Save the figure at the specified DPI
    fig.savefig(path, dpi=dpi, bbox_inches="tight")
    # Display inline in Jupyter
    plt.show()
    # Close the figure to free memory
    plt.close(fig)
    # Register the figure name for tracking
    _REG["figures"][_REG["module"]].append(path.name)
    print(f"[figure] {path.name}")
    return path


def save_table(df: pd.DataFrame | pd.Series, name: str, index: bool = True) -> Path
    """Save a DataFrame or Series as a CSV file under results/tables/."""
    path = TAB_DIR / f"{_REG['module']}_{name}.csv"
    # Convert Series to DataFrame if needed, then save to CSV
    (df.to_frame() if isinstance(df, pd.Series) else df).to_csv(path, index=index)
    # Register the table name for tracking
    _REG["tables"][_REG["module"]].append(path.name)
```

```
    print(f"[table]  {path.name}")
    return path


# ---------------------------------------------------------------------
# Data loading and cleaning functions
# ---------------------------------------------------------------------

def load_student_raw() -> pd.DataFrame:
    """Load raw student-mat.csv exactly as downloaded (395 x 33, 6 missing cells)."
    return pd.read_csv(STUDENT_CSV)


def get_mode(s: pd.Series):
    """Return the most frequent non-null value (mirrors the get_mode() helper from
    return s.dropna().mode().iloc[0]


def clean_student(df: pd.DataFrame) -> pd.DataFrame:
    """Cleaning pipeline derived from labs 2/3:
    - '?' placeholder -> NaN
    - numeric NaN -> median imputation
    - categorical NaN -> mode imputation
    - drop duplicates
    - reset index

    (All missing cells are categorical: 6 true NaN in sex/address/famsize/schoolsup
    plus 8 '?' placeholders in Pstatus/address that R's is.na() did not detect.)
    """
    # Replace '?' placeholders with proper NaN values
    out = df.replace("?", np.nan)
    # Impute each column: numeric with median, categorical with mode
    for col in out.columns:
        if pd.api.types.is_numeric_dtype(out[col]):
            if out[col].isna().any():
                out[col] = out[col].fillna(out[col].median())
        elif out[col].isna().any():
            out[col] = out[col].fillna(get_mode(out[col]))
    # Remove duplicate rows and reset the index
    out = out.drop_duplicates().reset_index(drop=True)
    return out


def load_student(clean: bool = True) -> pd.DataFrame:
    """Load the student data; ``clean=True`` applies :func:`clean_student`.

    Also saves the cleaned version to disk for reuse.
    """
    raw = load_student_raw()
    if not clean:
        return raw
    out = clean_student(raw)
    # Save the cleaned data for later reference
    out.to_csv(STUDENT_CLEAN_CSV, index=False)
    return out
```

```python
# ---------------------------------------------------------------
# Statistics helper functions
# ---------------------------------------------------------------

def iqr_bounds(s: pd.Series, k: float = 1.5):
    """Return (lower_fence, upper_fence) for IQR-based outlier detection.

    k = 1.5 for mild outliers, k = 3.0 for extreme outliers.
    """
    q1, q3 = s.quantile(0.25), s.quantile(0.75)
    i = q3 - q1  # interquartile range
    return q1 - k * i, q3 + k * i


def minmax(x, lo=0.0, hi=1.0):
    """Min-max scaling: rescale array x to the range [lo, hi]."""
    x = np.asarray(x, dtype=float)
    return lo + (x - x.min()) * (hi - lo) / (x.max() - x.min())


def skewness_e1071(x) -> float:
    """Skewness using the e1071::skewness default (type 3): m3 / s^3 with populatio

    This matches R's e1071 package type-3 formula.
    """
    x = np.asarray(x, dtype=float)
    n = len(x)
    # Second central moment (population variance)
    m2 = np.mean((x - x.mean()) ** 2)
    # Third central moment
    m3 = np.mean((x - x.mean()) ** 3)
    # Apply the correction factor to match e1071 type 3
    return float(m3 / m2 ** 1.5 * ((n - 1) / n) ** 1.5)


def kurtosis_e1071(x) -> float:
    """Excess kurtosis using the e1071::kurtosis default (type 3).

    Returns kurtosis - 3 (excess), matching R's e1071 package.
    """
    x = np.asarray(x, dtype=float)
    n = len(x)
    # Second and fourth central moments (population)
    m2 = np.mean((x - x.mean()) ** 2)
    m4 = np.mean((x - x.mean()) ** 4)
    # Apply the correction factor to match e1071 type 3
    return float((m4 / m2 ** 2) * ((n - 1) / n) ** 2 - 3)


def zscore_df(df: pd.DataFrame) -> pd.DataFrame:
    """Column z-scores with the sample sd (ddof=1), identical to R's scale().

    Each column is centred to mean 0 and scaled to standard deviation 1.
    """
    return (df - df.mean()) / df.std(ddof=1)
```

```
# Make the helpers reachable as U.<name> exactly like in the project modules
# This lets every cell call U.load_student(), U.save_fig(), etc.
U = sys.modules[__name__]

# Print confirmation of helper functions and data availability
print("=" * 78)
print("CELL 2: HELPER FUNCTIONS INITIALIZED")
print(f"Dataset location: {STUDENT_CSV.resolve()} (found: {STUDENT_CSV.exists()})")
print(f"Figures output directory: {FIG_DIR.resolve()}")
print(f"Tables output directory:  {TAB_DIR.resolve()}")
print(f"Total numeric features: {len(NUMERIC_COLS)}")
print(f"Total categorical features: {len(CATEGORICAL_COLS)}")
print("=" * 78)
```

```
==============================================================================
CELL 2: HELPER FUNCTIONS INITIALIZED
Dataset location: D:\DOWNLOAD\files\student-mat.csv (found: True)
Figures output directory: D:\DOWNLOAD\files\results\figures
Tables output directory:  D:\DOWNLOAD\files\results\tables
Total numeric features: 16
Total categorical features: 17
==============================================================================
```

2. Data Loading, First Look & Data Dictionary

Shape, dtypes, head/tail, missing values (including hidden '?' placeholders), summary statistics.

In [3]:
```
# ------------------------------------------------------------------
# 2. Data Loading, First Look & Data Dictionary
# ------------------------------------------------------------------

# Re-import for standalone cell execution
import numpy as np
import pandas as pd

# Title for this section
TITLE = "AST02 - Exploration, Missing Values & Basic Statistics"

def run():
    # 1. Load the raw (uncleaned) student data
    data = U.load_student_raw()

    # 2-3. Display first and last 10 records to inspect the data
    U.section("2-3. First / last 10 records")
    print(data.head(10).to_string())
    print(data.tail(10).to_string())

    # 4-7. Basic dataset properties: shape, column names, data types, summary stati
    U.section("4-7. Shape, columns, dtypes, summary")
    print("rows, cols:", data.shape)
    print("columns:", list(data.columns))
    print(data.dtypes.to_string())
```

```python
# describe(include="all") gives stats for both numeric and categorical columns
desc = data.describe(include="all").T
U.save_table(desc, "summary_all_columns")

# 8-11. Missing values analysis
U.section("8-11. Missing values")
# Count NaN values per column
na_col = data.isna().sum()
# Show only columns that have missing values
print(na_col[na_col > 0])
print("Total missing:", int(na_col.sum()))
U.save_table(na_col.rename("missing"), "missing_per_column")

# Show the actual rows that contain missing values
rows_na = data[data.isna().any(axis=1)]
print("Rows with >=1 missing:\n", rows_na.to_string())
U.save_table(rows_na, "rows_with_missing")

# Detect hidden missing values coded as '?' (not caught by isna)
hidden = (data == "?").sum()
hidden = hidden[hidden > 0]
print("\nHIDDEN missing values coded as '?' (not caught by isna):", hidden.to_d
      f"-> true total missing = {int(na_col.sum() + hidden.sum())}")
U.save_table(hidden.rename("question_mark_placeholders"), "hidden_missing_place

# 12. Remove rows with missing values (listwise deletion)
U.section("12. Remove rows with missing values (listwise deletion)")
dropped = data.dropna()
print(f"{len(data)} -> {len(dropped)} rows")

# 13-15. Different imputation strategies: mean, median, mode
U.section("13-15. Mean / median / mode imputation")
work = data.copy()
# Mean imputation for the 'age' column (numeric)
work["age"] = work["age"].fillna(work["age"].mean())
# Median imputation for the 'G1' column (numeric)
work["G1"] = work["G1"].fillna(work["G1"].median())
# Mode imputation for the 'sex' column (categorical)
mode_sex = U.get_mode(work["sex"])
work["sex"] = work["sex"].fillna(mode_sex)
print(f"mean(age)={data['age'].mean():.4f}  median(G1)={data['G1'].median()}  m

# 16-17. Forward fill and backward fill (R equivalent: zoo::na.locf)
U.section("16-17. Forward / backward fill (R: zoo::na.locf)")
# Forward fill: propagate last valid value forward
ffill = data.ffill()
# Backward fill: propagate next valid value backward
bfill = data.bfill()
print("NaNs after ffill:", int(ffill.isna().sum().sum()), "| after bfill:", int

# 18. Constant fill: replace remaining NaN with 0 (numeric) or "Unknown" (categ
U.section("18. Constant fill for remaining NaN")
const = work.copy()
for c in const.columns:
    if pd.api.types.is_numeric_dtype(const[c]):
        const[c] = const[c].fillna(0)
```

```
        else:
            const[c] = const[c].fillna("Unknown")
    print("NaNs after constant fill:", int(const.isna().sum().sum()))

    # 19. Check for and remove duplicate rows
    U.section("19. Duplicates")
    print("duplicate rows:", int(const.duplicated().sum()))
    const = const.drop_duplicates()

    # 20-22. Convert columns to factors, rename, and create derived columns
    U.section("20-22. Factors, rename, derived column")
    fac = const.copy()
    # Convert all non-numeric columns to pandas Categorical type (R's factor())
    for c in fac.select_dtypes(exclude="number").columns:
        fac[c] = fac[c].astype("category")
    # Also convert age to categorical (ordered factor)
    fac["age"] = fac["age"].astype("category")
    print(fac[["age", "sex", "school"]].dtypes.to_string())
    # Rename 'sex' to 'gender' for clarity
    fac = fac.rename(columns={"sex": "gender"})
    # Create a derived column: sum of first and second period grades
    fac["G1_plus_G2"] = fac["G1"] + fac["G2"]
    print(fac[["gender", "G1", "G2", "G1_plus_G2"]].head())

    # 23-28. Compute descriptive statistics for all numeric columns
    U.section("23-28. Descriptive statistics for every numeric column")
    num = const[U.NUMERIC_COLS]
    stats = pd.DataFrame({
        "min": num.min(), "max": num.max(), "range": num.max() - num.min(),
        "mean": num.mean(), "median": num.median(),
        "variance": num.var(), "std": num.std(),
        "IQR": num.quantile(0.75) - num.quantile(0.25),
    })
    print(stats.round(3).to_string())
    U.save_table(stats.round(4), "numeric_descriptive_stats")
    # Spot-check specific values against the R lab output
    print(f"\nLab spot-checks -> min/max G1: {num['G1'].min()}/{num['G1'].max()}, "
          f"var(G2)={num['G2'].var():.5f}, sd(G2)={num['G2'].std():.5f}, IQR(age)={

    # Visualise missing values as a bar chart
    U.section("Missing-value overview figure")
    fig, ax = U.plt.subplots(figsize=(10, 3.5))
    na_col[na_col > 0].plot.bar(ax=ax, color="indianred")
    ax.set_title("Missing values per column (raw data)")
    ax.set_ylabel("count")
    U.save_fig("missing_values", fig)

    return {"total_missing_isna": int(na_col.sum()), "hidden_question_marks": int(h

# Set the module label and run
U.set_module("m02")
run()
```

```
================================================================================
2-3. First / last 10 records
================================================================================
  school sex  age address famsize Pstatus  Medu  Fedu      Mjob      Fjob      reaso
n guardian  traveltime  studytime  failures schoolsup famsup paid activities nursery
higher internet romantic  famrel  freetime  goout  Dalc  Walc  health  absences  G1
G2  G3
0     GP   F   18       U     GT3       A     4     4   at_home   teacher      cours
e   mother           2          2         0       yes     no   no         no     yes
yes       no       no       4         3      4     1     1       3         6   5   6
6
1     GP   F   17       U     GT3       T     1     1   at_home     other      cours
e   father           1          2         0       no    yes   no         no      no
yes      yes       no       5         3      3     1     1       3         4   5   5
6
2     GP   F   15       U     LE3       T     1     1   at_home     other       othe
r   mother           1          2         3       yes     no  yes         no     yes
yes      yes       no       4         3      2     2     3       3        10   7   8
10
3     GP   F   15       U     GT3       T     4     2    health  services        hom
e   mother           1          3         0       no    yes  yes        yes     yes
yes      yes      yes       3         2      2     1     1       5         2  15  14
15
4     GP   F   16       U     GT3       T     3     3     other     other        hom
e   father           1          2         0       no    yes  yes         no     yes
yes       no       no       4         3      2     1     2       5         4   6  10
10
5     GP   M   16       U     LE3       T     4     3  services     other  reputatio
n   mother           1          2         0       no    yes  yes        yes     yes
yes      yes       no       5         4      2     1     2       5        10  15  15
15
6     GP   M   16       U     LE3       T     2     2     other     other        hom
e   mother           1          2         0       no     no   no         no     yes
yes      yes       no       4         4      4     1     1       3         0  12  12
11
7     GP   F   17       U     GT3       A     4     4     other   teacher        hom
e   mother           2          2         0       yes    yes   no         no     yes
yes       no       no       4         1      4     1     1       1         6   6   5
6
8     GP   M   15       U     LE3       A     3     2  services     other        hom
e   mother           1          2         0       no    yes  yes         no     yes
yes      yes       no       4         2      2     1     1       1         0  16  18
19
9     GP   M   15       U     GT3       T     3     4     other     other        hom
e   mother           1          2         0       no    yes  yes        yes     yes
yes      yes       no       5         5      1     1     1       5         0  14  15
15
    school sex  age address famsize Pstatus  Medu  Fedu      Mjob      Fjob      rea
son guardian  traveltime  studytime  failures schoolsup famsup paid activities nurse
ry higher internet romantic  famrel  freetime  goout  Dalc  Walc  health  absences
G1  G2  G3
385     MS   F   18       R     GT3       T     2     2   at_home     other       ot
her   mother          2          3         0        no     no  yes         no     y
es    yes       no       no       5         3      3     1     3       4         2
10   9  10
386     MS   F   18       R     GT3       T     4     4   teacher   at_home  reputat
```

```
ion   mother          3          1          0        no    yes  yes        yes     y
es    yes      yes      yes       4          4      3     2     2       5         7
6   5   6
387     MS   F   19       R     GT3       T     2     3  services     other      cou
rse   mother          1          3          1        no     no   no        yes
no    yes      yes       no       5          4      2     1     2       5         0
7   5   0
388     MS   F   18       U     LE3       T     3     1   teacher  services      cou
rse   mother          1          2          0        no    yes  yes         no     y
es    yes      yes       no       4          3      4     1     1       1         0
7   9   8
389     MS   F   18       U     GT3       T     1     1     other     other      cou
rse   mother          2          2          1        no     no   no        yes     y
es    yes       no       no       1          1      1     1     1       5         0
6   5   0
390     MS   M   20       U     LE3       A     2     2  services  services      cou
rse    other          1          2          2        no    yes  yes         no     y
es    yes       no       no       5          5      4     4     5       4        11
9   9   9
391     MS   M   17       U     LE3       T     3     1  services  services      cou
rse   mother          2          1          0        no     no   no         no
no    yes      yes       no       2          4      5     3     4       2         3
14  16  16
392     MS   M   21       R     GT3       T     1     1     other     other      cou
rse    other          1          1          3        no     no   no         no
no    yes       no       no       5          5      3     3     3       3         3
10   8   7
393     MS   M   18       R     LE3       T     3     2  services     other      cou
rse   mother          3          1          0        no     no   no         no
no    yes      yes       no       4          4      1     3     4       5         0
11  12  10
394     MS   M   19       U     LE3       T     1     1     other   at_home      cou
rse   father          1          1          0        no     no   no         no     y
es    yes      yes       no       3          2      3     3     3       5         5
8   9   9

================================================================================
4-7. Shape, columns, dtypes, summary
================================================================================
rows, cols: (395, 33)
columns: ['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu',
'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime', 'failures', 'school
sup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic',
'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2', 'G
3']
school          str
sex             str
age           int64
address         str
famsize         str
Pstatus         str
Medu          int64
Fedu          int64
Mjob            str
Fjob            str
reason          str
```

```
guardian        str
traveltime    int64
studytime     int64
failures      int64
schoolsup       str
famsup          str
paid            str
activities      str
nursery         str
higher          str
internet        str
romantic        str
famrel        int64
freetime      int64
goout         int64
Dalc          int64
Walc          int64
health        int64
absences      int64
G1            int64
G2            int64
G3            int64
[table]  m02_summary_all_columns.csv

================================================================================
8-11. Missing values
================================================================================
sex          2
address      1
famsize      1
schoolsup    2
dtype: int64
Total missing: 6
[table]  m02_missing_per_column.csv
```

```
Rows with >=1 missing:
     school  sex  age address famsize Pstatus  Medu  Fedu      Mjob      Fjob      r
eason guardian  traveltime  studytime  failures schoolsup famsup paid activities nur
sery higher internet romantic  famrel  freetime  goout  Dalc  Walc  health  absences
G1  G2  G3
14      GP    M   15       U     NaN       A     2     2     other     other
home    other           1          3         0        no    yes   no         no
yes    yes      yes      yes       4         5      2     1     1       3         0
14  16  16
45      GP    F   15     NaN     LE3       A     4     3     other     other      co
urse   mother           1          2         0       yes    yes  yes        yes
yes    yes      yes      yes       5         2      2     1     1       5         8
8   8   6
62      GP    F   16       U     LE3       T     1     2     other  services  reputa
tion   father           1          2         0       NaN     no   no         yes
yes    yes      yes       no       4         4      3     1     1       1         4
8  10   9
130     GP    F   15       R     GT3       T     3     4  services   teacher      co
urse   father           2          3         2       NaN    yes   no         no
yes    yes      yes      yes       4         2      2     2     2       5         0
12   0   0
248     GP  NaN   18       R     LE3       T     3     3     other  services      co
urse   mother           1          2         1        no    yes   no         no
yes    yes      yes      yes       4         3      3     1     3       5         8
3   5   5
249     GP  NaN   16       U     GT3       T     0     2     other     other       o
ther   mother           1          1         0        no     no  yes         no
no    yes      yes       no       4         3      2     2     4       5         0
13  15  15
[table]  m02_rows_with_missing.csv

HIDDEN missing values coded as '?' (not caught by isna): {'address': 2, 'Pstatus':
6} -> true total missing = 14
[table]  m02_hidden_missing_placeholders.csv

================================================================================
12. Remove rows with missing values (listwise deletion)
================================================================================
395 -> 389 rows

================================================================================
13-15. Mean / median / mode imputation
================================================================================
mean(age)=16.6962  median(G1)=11.0  mode(sex)=F

================================================================================
16-17. Forward / backward fill (R: zoo::na.locf)
================================================================================
NaNs after ffill: 0 | after bfill: 0

================================================================================
18. Constant fill for remaining NaN
================================================================================
NaNs after constant fill: 0

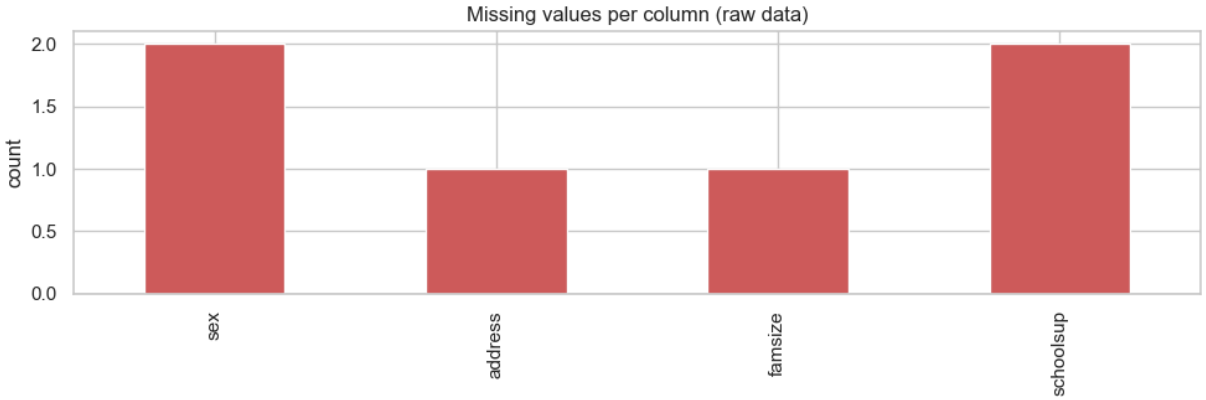
================================================================================
```

```
19. Duplicates
================================================================================
duplicate rows: 0
================================================================================
20-22. Factors, rename, derived column
================================================================================
age       category
sex       category
school    category
  gender  G1  G2  G1_plus_G2
0      F   5   6          11
1      F   5   5          10
2      F   7   8          15
3      F  15  14          29
4      F   6  10          16

================================================================================
23-28. Descriptive statistics for every numeric column
================================================================================
             min  max  range    mean  median  variance    std  IQR
age           15   22      7  16.696    17.0     1.628  1.276  2.0
Medu           0    4      4   2.749     3.0     1.198  1.095  2.0
Fedu           0    4      4   2.522     2.0     1.184  1.088  1.0
traveltime     1    4      3   1.448     1.0     0.487  0.698  1.0
studytime      1    4      3   2.035     2.0     0.704  0.839  1.0
failures       0    3      3   0.334     0.0     0.553  0.744  0.0
famrel         1    5      4   3.944     4.0     0.804  0.897  1.0
freetime       1    5      4   3.235     3.0     0.998  0.999  1.0
goout          1    5      4   3.109     3.0     1.239  1.113  2.0
Dalc           1    5      4   1.481     1.0     0.793  0.891  1.0
Walc           1    5      4   2.291     2.0     1.659  1.288  2.0
health         1    5      4   3.554     4.0     1.933  1.390  2.0
absences       0   75     75   5.709     4.0    64.050  8.003  8.0
G1             3   19     16  10.909    11.0    11.017  3.319  5.0
G2             0   19     19  10.714    11.0    14.149  3.762  4.0
G3             0   20     20  10.415    11.0    20.990  4.581  6.0
[table]  m02_numeric_descriptive_stats.csv

Lab spot-checks -> min/max G1: 3/19, var(G2)=14.14892, sd(G2)=3.76150, IQR(age)=2.0

================================================================================
Missing-value overview figure
================================================================================
```




```
[figure] m02_missing_values.png
```

Out[3]: {'total_missing_isna': 6, 'hidden_question_marks': 8}

3. Data Cleaning, Outliers & Univariate Analysis

Imputation, duplicates, type fixes, IQR outliers, then distributions of numeric and categorical features.

In [4]:
```
# ---------------------------------------------------------------------
# 3. Data Cleaning, Outliers & Univariate Analysis
# ---------------------------------------------------------------------

# Re-import for standalone cell execution
import numpy as np
import pandas as pd
import seaborn as sns

# Create a shortcut to matplotlib.pyplot through the U module
plt = U.plt

# Title for this section
TITLE = "AST03 - Cleaning, Outliers & Univariate Visualisation"

def cleaning(raw: pd.DataFrame) -> pd.DataFrame:
    """Tasks 3-10: clean the raw data by handling missing values, dropping columns,
    imputing, and removing duplicates. Returns the cleaned DataFrame."""

    U.section("Tasks 3-10: missing values, column drop, imputation, duplicates")

    # Count total missing values detected by isna()
    print("Total missing (isna):", int(raw.isna().sum().sum()))
    # Show the rows that have at least one missing value
    print(raw[raw.isna().any(axis=1)].to_string())

    # Task 5: drop columns where more than 70% of values are missing
    keep = raw.columns[raw.isna().mean() <= 0.70]
    # Also replace '?' placeholders with NaN (they act as hidden missing values)
    data = raw[keep].replace("?", np.nan)
    print("'?' placeholders found:", int((raw == "?").sum().sum()))
    print("Columns dropped (>70% missing):", sorted(set(raw.columns) - set(keep)) o

    # Tasks 6-8: impute missing values - median for numeric, mode for categorical
    for c in data.columns:
        if pd.api.types.is_numeric_dtype(data[c]):
            data[c] = data[c].fillna(data[c].median())
        else:
            data[c] = data[c].fillna(U.get_mode(data[c]))

    # Tasks 9-10: detect and remove duplicate rows
    print("Duplicates:", int(data.duplicated().sum()))
    data = data.drop_duplicates()
    print("NaNs left:", int(data.isna().sum().sum()), "| shape:", data.shape)

    # Task 11: identify columns with questionable data types
```

```
    U.section("Task 11: columns with questionable data types")
    # yes/no string columns should be boolean
    yn = [c for c in data.columns if set(data[c].unique()) <= {"yes", "no"}]
    # Integer-coded ordinal scales should be ordered categoricals
    ordinal = ["Medu", "Fedu", "traveltime", "studytime", "failures", "famrel",
               "freetime", "goout", "Dalc", "Walc", "health"]
    print("yes/no strings (better as bool):", yn)
    print("integer-coded ordinal scales (better as ordered categorical):", ordinal)

    # Tasks 12-13: convert types and demonstrate upper/lower case transformations
    U.section("Task 12-13: factors and upper/lower-case text")
    typed = data.copy()
    # Convert yes/no columns to True/False
    for c in yn:
        typed[c] = typed[c].map({"yes": True, "no": False})
    # Convert all remaining text columns to pandas Categorical
    for c in typed.select_dtypes(exclude=["number", "bool"]).columns:
        typed[c] = typed[c].astype("category")

    # Demonstrate upper-case and lower-case conversion
    upper = data.copy()
    for c in upper.select_dtypes(exclude="number").columns:
        upper[c] = upper[c].str.upper()
    print("upper-case sample:", upper.iloc[0, [0, 1, 3, 8]].tolist())
    print("lower-case again :", [str(v).lower() for v in upper.iloc[0, [0, 1, 3, 8]

    # Return the cleaned data (keeping original case for later tasks)
    return data

def outliers(data: pd.DataFrame) -> pd.DataFrame:
    """Detect and remove outliers using the IQR method. Returns data with extreme o

    # Task 14: box plots to visually identify outliers
    U.section("Task 14: boxplots for outlier detection")
    fig, axes = plt.subplots(1, 4, figsize=(14, 4))
    for ax, c in zip(axes, ["age", "G3", "absences", "failures"]):
        sns.boxplot(y=data[c], ax=ax, color="lightsteelblue")
        ax.set_title(c)
    U.save_fig("boxplots_outliers", fig)

    # Task 15: calculate IQR fences and count outliers for key columns
    U.section("Task 15: IQR-method outliers")
    rows = []
    for c in ["age", "G1", "G2", "G3", "absences", "failures"]:
        lo, hi = U.iqr_bounds(data[c])
        # Count values below lower fence or above upper fence
        n = int(((data[c] < lo) | (data[c] > hi)).sum())
        rows.append({"column": c, "lower_fence": lo, "upper_fence": hi, "n_outliers
    tab = pd.DataFrame(rows)
    print(tab.to_string(index=False))
    U.save_table(tab, "iqr_outliers", index=False)

    # Task 16: remove extreme outliers (3 x IQR) on absences and G3
    U.section("Task 16: remove extreme outliers (3 x IQR) on absences & G3")
    clean = data.copy()
    for c in ["G3", "absences"]:
```

```
        lo, hi = U.iqr_bounds(clean[c], k=3.0)  # k=3 for extreme outliers only
        before = len(clean)
        clean = clean[(clean[c] >= lo) & (clean[c] <= hi)]
        print(f"{c}: fences=({lo:.1f}, {hi:.1f}) removed {before - len(clean)} rows
    print("rows:", len(data), "->", len(clean))
    return clean

def numeric_plots(data: pd.DataFrame):
    """Tasks 17-20, 29-30: create various univariate numeric plots."""

    U.section("Tasks 17-20, 29-30: histogram, density, violin, dot plot, facets")

    # Task 17: basic histogram of G3 (final grades)
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.hist(data["G3"], bins=range(0, 22), color="skyblue", edgecolor="black")
    ax.set(title="Distribution of Final Grades (G3)", xlabel="G3", ylabel="Frequenc
    U.save_fig("17_histogram_G3", fig)
    # Note the bimodal distribution: main peak + spike at 0
    print("G3 is bimodal-ish: a main peak near 10-11 plus a spike of 0s "
          f"({int((data['G3'] == 0).sum())} students).")

    # Task 18: histogram with kernel density estimate (KDE) overlay
    fig, ax = plt.subplots(figsize=(7, 4))
    sns.histplot(data["G3"], bins=range(0, 22), stat="density", kde=True, color="go
    ax.set_title("Histogram with density curve")
    U.save_fig("18_hist_density_G3", fig)

    # Task 19: violin plot of age distribution
    fig, ax = plt.subplots(figsize=(4, 5))
    sns.violinplot(y=data["age"], color="gold", cut=0, ax=ax)
    ax.set_title("Violin plot: age")
    U.save_fig("19_violin_age", fig)

    # Task 20: dot plot (strip of dots stacked at each value) for a small sample
    small = data["G3"].sample(40, random_state=1)
    fig, ax = plt.subplots(figsize=(8, 3.5))
    for val, cnt in small.value_counts().sort_index().items():
        # Stack dots vertically for each unique grade value
        ax.scatter([val] * cnt, range(1, cnt + 1), color="goldenrod", edgecolor="bl
    ax.set(title="Dot plot of G3 (random sample, n=40)", xlabel="G3", yticks=[])
    U.save_fig("20_dotplot_G3", fig)

    # Task 29: smooth density plot of G3
    fig, ax = plt.subplots(figsize=(7, 4))
    sns.kdeplot(data["G3"], fill=True, cut=0, ax=ax)
    ax.set_title("Density plot of G3")
    U.save_fig("29_density_G3", fig)

    # Task 30: faceted density plots showing G3 distribution for each mother's job
    g = sns.FacetGrid(data, col="Mjob", col_wrap=3, height=3, sharey=False)
    g.map_dataframe(sns.kdeplot, x="G3", fill=True, cut=0)
    g.set_titles("Mjob = {col_name}")
    g.figure.suptitle("G3 distribution by mother's job", y=1.03)
    U.save_fig("30_facet_density_G3_by_Mjob", g.figure)

def categorical_plots(data: pd.DataFrame):
```

```
    """Tasks 21-28: create categorical summaries and charts."""

    U.section("Tasks 21-28: categorical summaries and charts")

    # Tasks 21-22: frequency and relative frequency table for 'sex'
    freq = data["sex"].value_counts()
    summary = pd.DataFrame({"count": freq, "proportion": freq / freq.sum(),
                            "percent": 100 * freq / freq.sum()}).round(4)
    print(summary)
    U.save_table(summary, "sex_frequency")

    # Task 23: simple bar chart of sex distribution
    fig, ax = plt.subplots(figsize=(5, 4))
    sns.countplot(data=data, x="sex", color="gold", edgecolor="black", ax=ax)
    ax.set(title="Frequency bar chart", ylabel="Frequency")
    U.save_fig("23_bar_sex", fig)

    # Task 24: horizontal bar chart of mother's job, ordered by frequency
    order = data["Mjob"].value_counts().index
    fig, ax = plt.subplots(figsize=(6, 4))
    sns.countplot(data=data, y="Mjob", order=order, color="steelblue", ax=ax)
    ax.set(title="Students' mother's job", xlabel="Frequency", ylabel="Mother's job
    U.save_fig("24_hbar_Mjob", fig)

    # Task 25: percentage bar chart of sex
    fig, ax = plt.subplots(figsize=(5, 4))
    (100 * data["sex"].value_counts(normalize=True)).plot.bar(color="teal", ax=ax,
    ax.set(title="Percentage of students by gender", ylabel="%")
    U.save_fig("25_percent_bar_sex", fig)

    # Task 26: pie chart of gender distribution
    fig, ax = plt.subplots(figsize=(5, 5))
    freq.plot.pie(autopct="%1.1f%%", startangle=90, ax=ax, colors=["#f8766d", "#00b
    ax.set(title="Gender distribution", ylabel="")
    U.save_fig("26_pie_sex", fig)

    # Task 27: lollipop chart for mother's job
    f = data["Mjob"].value_counts().sort_values()
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.hlines(y=f.index, xmin=0, xmax=f.values, color="gray")
    ax.plot(f.values, f.index, "o", markersize=9)
    ax.set(title="Lollipop chart: mother's job", xlabel="Frequency")
    U.save_fig("27_lollipop_Mjob", fig)

    # Task 28: dot chart for mother's job (similar to lollipop but without stems)
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.plot(f.values, f.index, "o", markersize=9)
    ax.grid(axis="y", linestyle=":")
    ax.set(title="Dot chart: mother's job", xlabel="Frequency")
    U.save_fig("28_dotchart_Mjob", fig)

def run():
    """Execute all parts of section 3: cleaning, outlier removal, and plotting."""
    raw = U.load_student_raw()
    data = cleaning(raw)
    clean = outliers(data)
```

```
    numeric_plots(data)
    categorical_plots(data)
    return {"rows_after_extreme_outlier_removal": len(clean)}

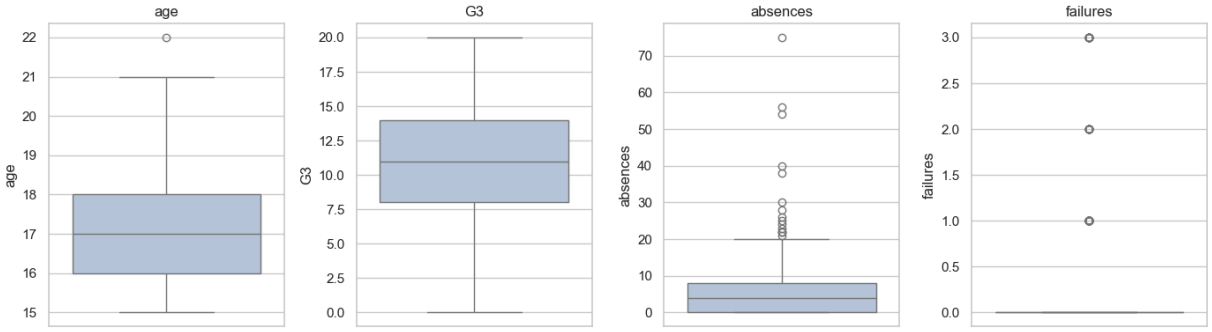
# Set the module label and run
U.set_module("m03")
run()
```

```
================================================================================
Tasks 3-10: missing values, column drop, imputation, duplicates
================================================================================
Total missing (isna): 6
    school  sex  age address famsize Pstatus  Medu  Fedu      Mjob      Fjob      re
ason guardian  traveltime  studytime  failures schoolsup famsup paid activities nurs
ery higher internet romantic  famrel  freetime  goout  Dalc  Walc  health  absences
G1  G2  G3
14      GP    M   15       U     NaN       A     2     2     other     other
home    other           1          3         0        no    yes   no         no
yes    yes      yes      yes       4         5      2     1     1       3         0
14  16  16
45      GP    F   15     NaN     LE3       A     4     3     other     other      co
urse   mother           1          2         0       yes    yes  yes        yes
yes    yes      yes      yes       5         2      2     1     1       5         8
8   8   6
62      GP    F   16       U     LE3       T     1     2     other  services  reputa
tion   father           1          2         0       NaN     no   no        yes
yes    yes      yes       no       4         4      3     1     1       1         4
8  10   9
130     GP    F   15       R     GT3       T     3     4  services   teacher      co
urse   father           2          3         2       NaN    yes   no         no
yes    yes      yes      yes       4         2      2     2     2       5         0
12   0   0
248     GP  NaN   18       R     LE3       T     3     3     other  services      co
urse   mother           1          2         1        no    yes   no         no
yes    yes      yes      yes       4         3      3     1     3       5         8
3   5   5
249     GP  NaN   16       U     GT3       T     0     2     other     other       o
ther   mother           1          1         0        no     no  yes        no
no    yes      yes       no       4         3      2     2     4       5         0
13  15  15
'?' placeholders found: 8
Columns dropped (>70% missing): none
Duplicates: 0
NaNs left: 0 | shape: (395, 33)

================================================================================
Task 11: columns with questionable data types
================================================================================
yes/no strings (better as bool): ['schoolsup', 'famsup', 'paid', 'activities', 'nurs
ery', 'higher', 'internet', 'romantic']
integer-coded ordinal scales (better as ordered categorical): ['Medu', 'Fedu', 'trav
eltime', 'studytime', 'failures', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'he
alth']

================================================================================
Task 12-13: factors and upper/lower-case text
================================================================================
upper-case sample: ['GP', 'F', 'U', 'AT_HOME']
lower-case again : ['gp', 'f', 'u', 'at_home']

================================================================================
Task 14: boxplots for outlier detection
================================================================================
```

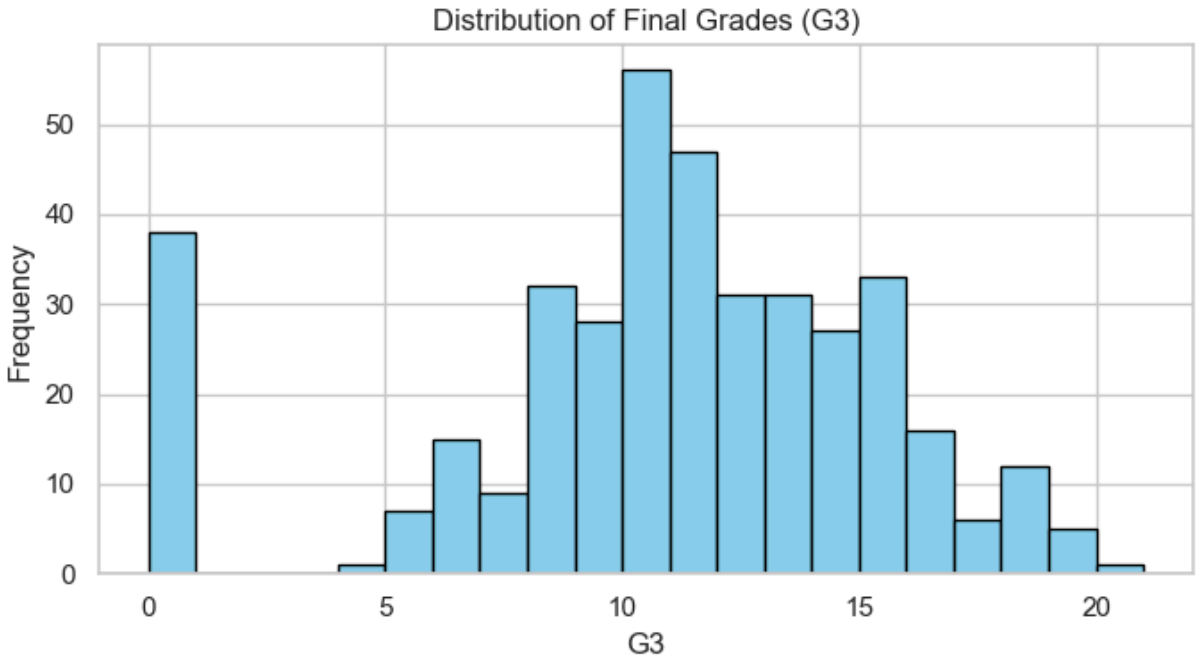


[figure] m03_boxplots_outliers.png

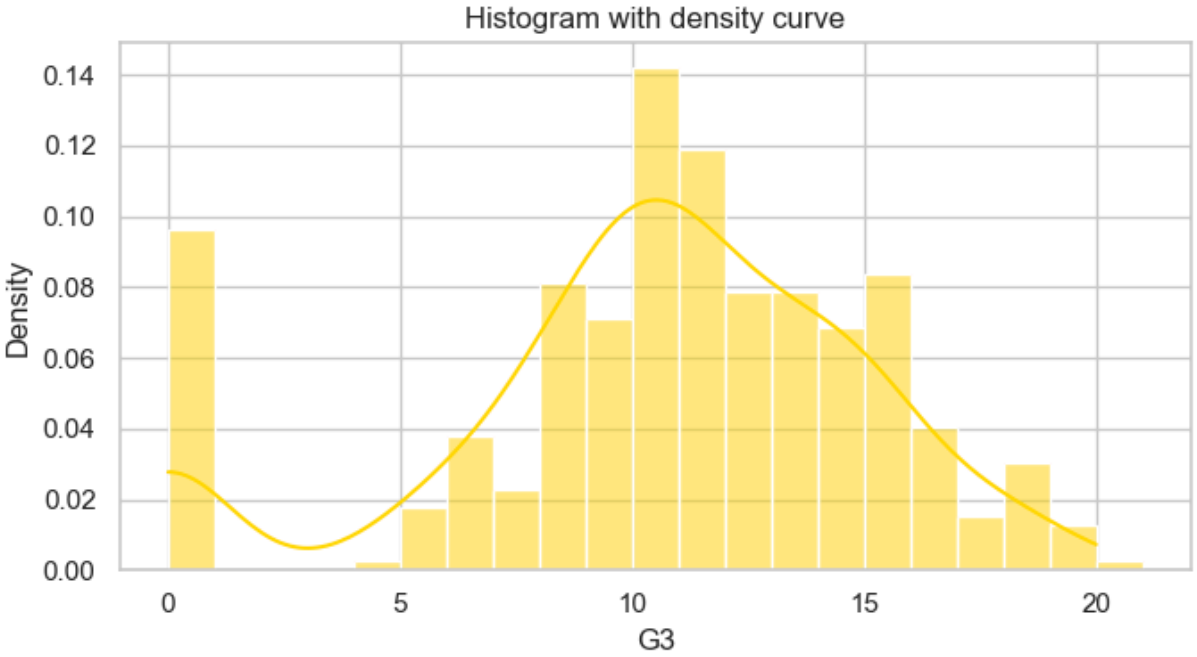
```
================================================================================
Task 15: IQR-method outliers
================================================================================
  column  lower_fence  upper_fence  n_outliers
     age         13.0         21.0           1
      G1          0.5         20.5           0
      G2          3.0         19.0          13
      G3         -1.0         23.0           0
absences        -12.0         20.0          15
failures          0.0          0.0          83
[table]  m03_iqr_outliers.csv

================================================================================
Task 16: remove extreme outliers (3 x IQR) on absences & G3
================================================================================
G3: fences=(-10.0, 32.0) removed 0 rows
absences: fences=(-24.0, 32.0) removed 5 rows
rows: 395 -> 390

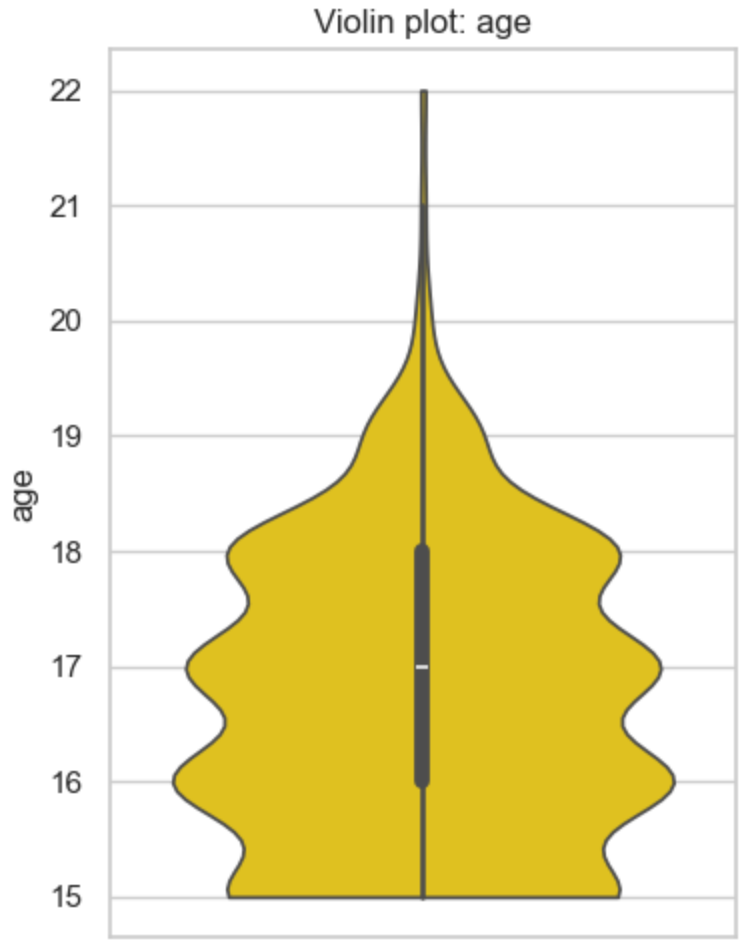
================================================================================
Tasks 17-20, 29-30: histogram, density, violin, dot plot, facets
================================================================================
```



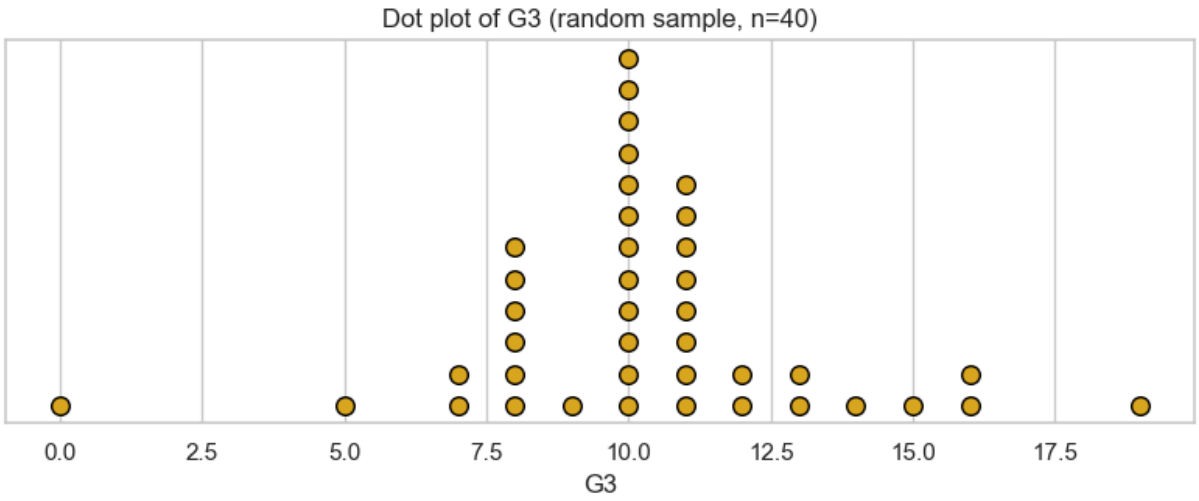
[figure] m03_17_histogram_G3.png
G3 is bimodal-ish: a main peak near 10-11 plus a spike of 0s (38 students).



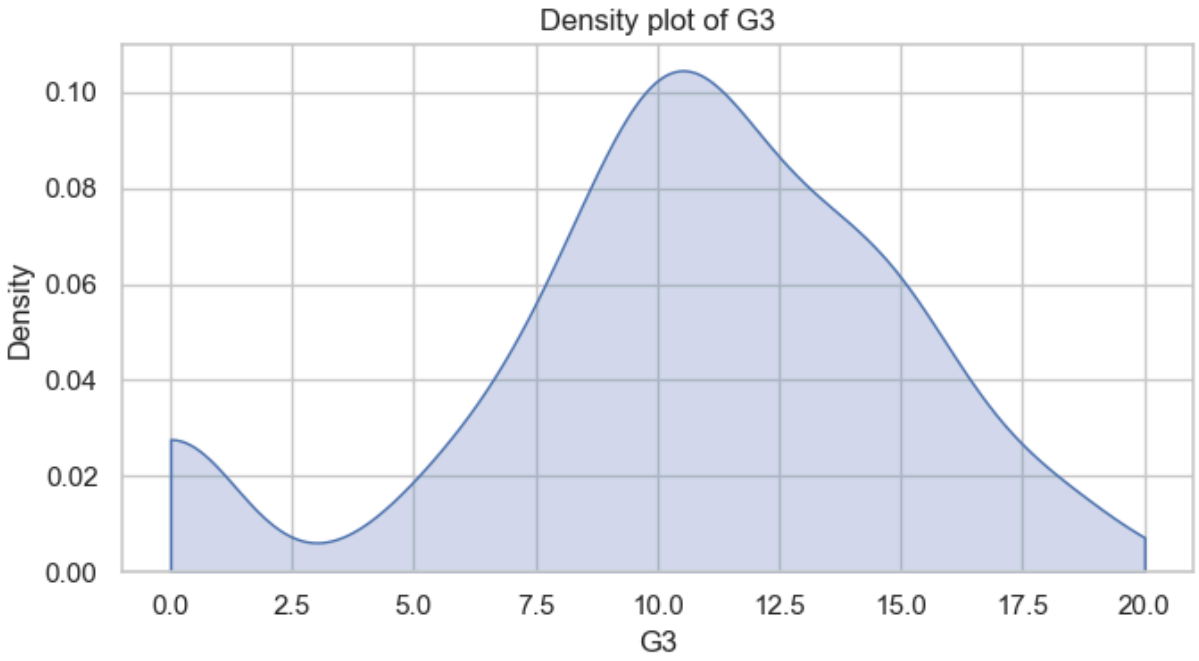
[figure] m03_18_hist_density_G3.png



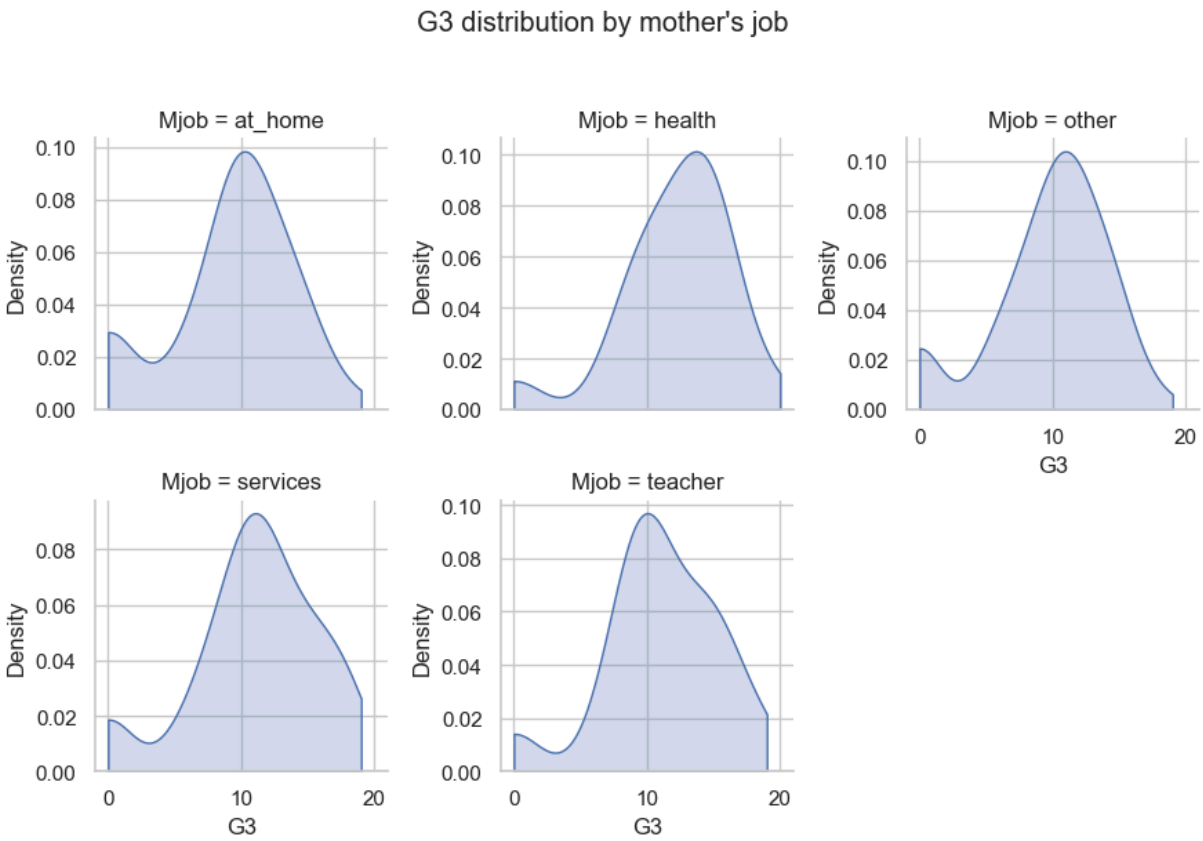
[figure] m03_19_violin_age.png



[figure] m03_20_dotplot_G3.png

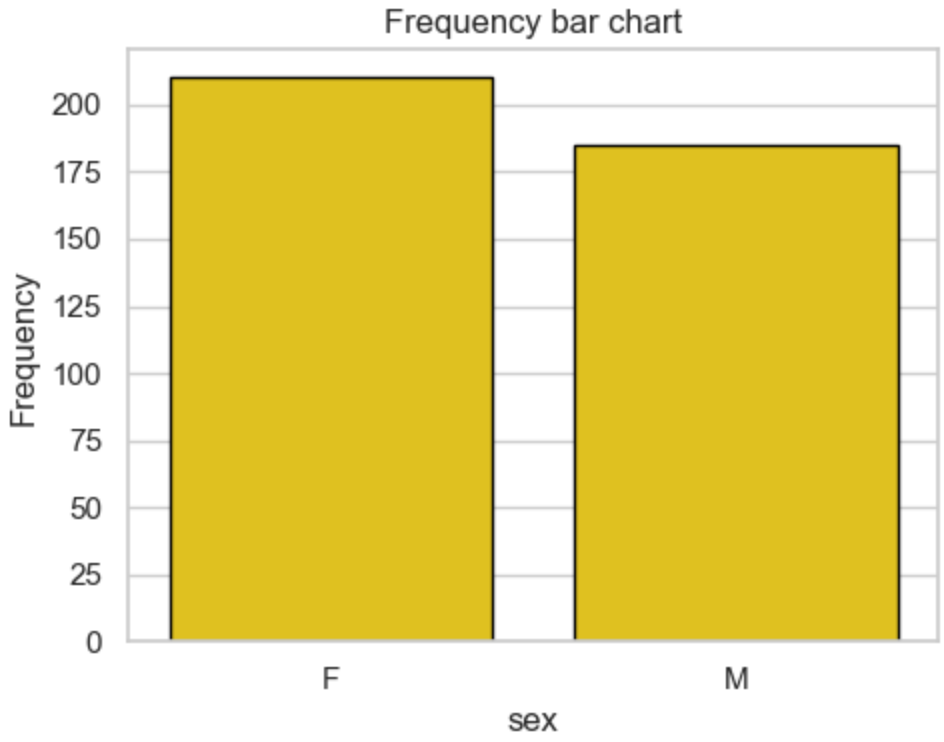


[figure] m03_29_density_G3.png

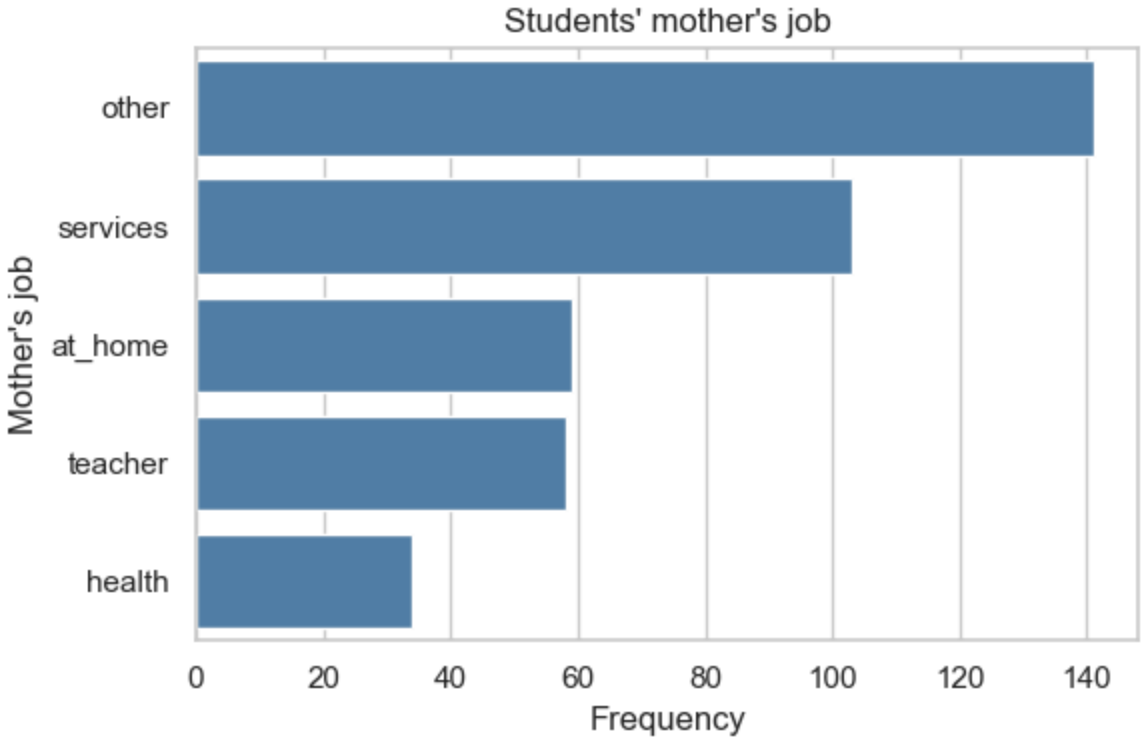


[figure] m03_30_facet_density_G3_by_Mjob.png

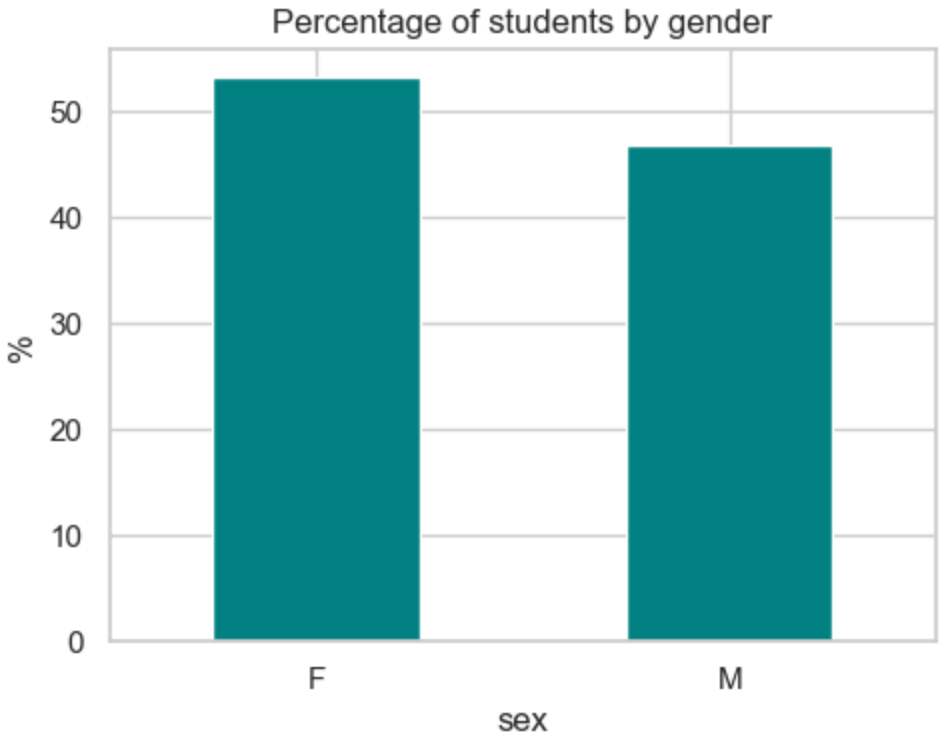
```
================================================================================
Tasks 21-28: categorical summaries and charts
================================================================================
     count  proportion  percent
sex
F      210      0.5316  53.1646
M      185      0.4684  46.8354
[table]  m03_sex_frequency.csv
```



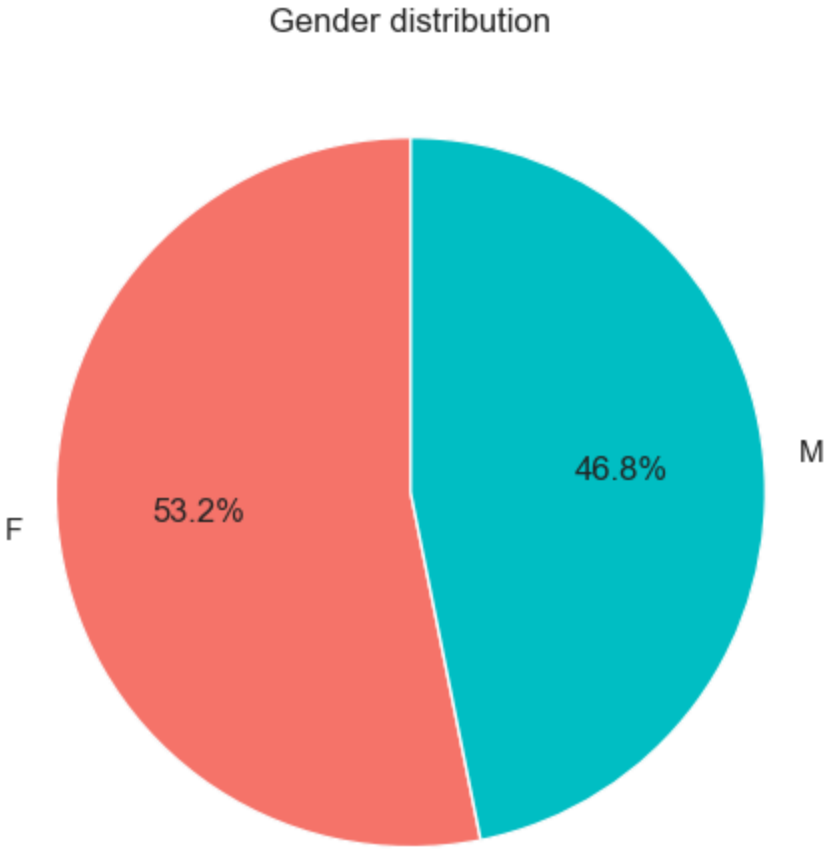
[figure] m03_23_bar_sex.png



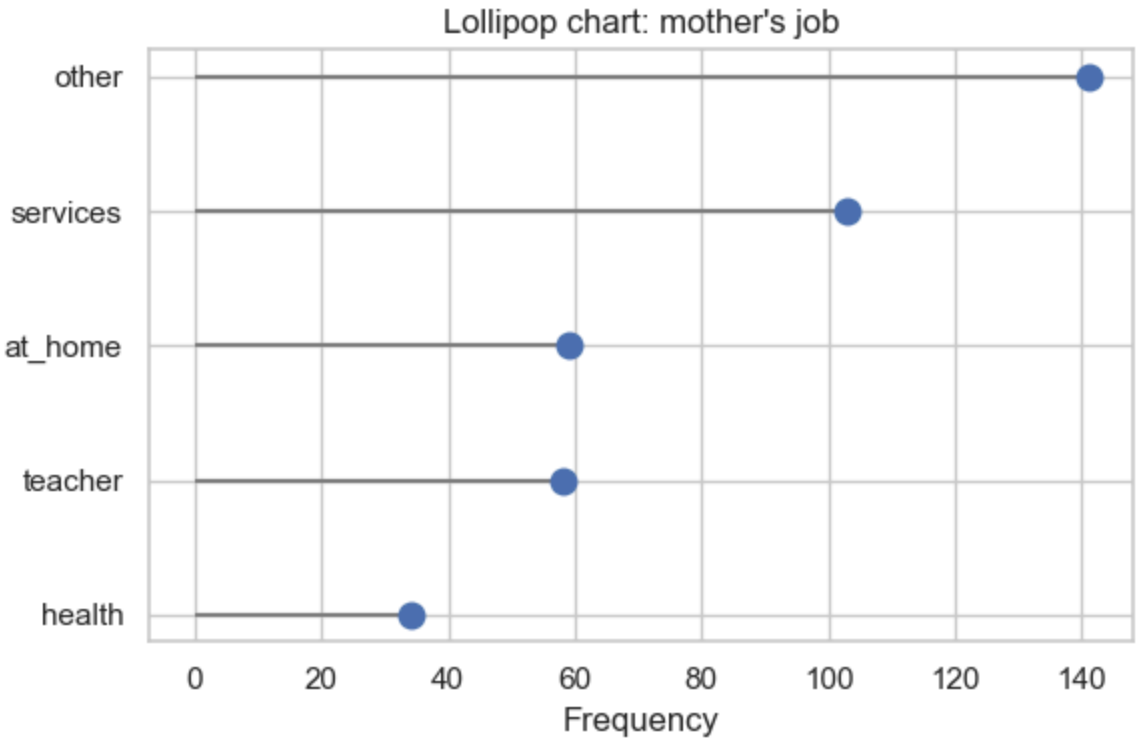
[figure] m03_24_hbar_Mjob.png



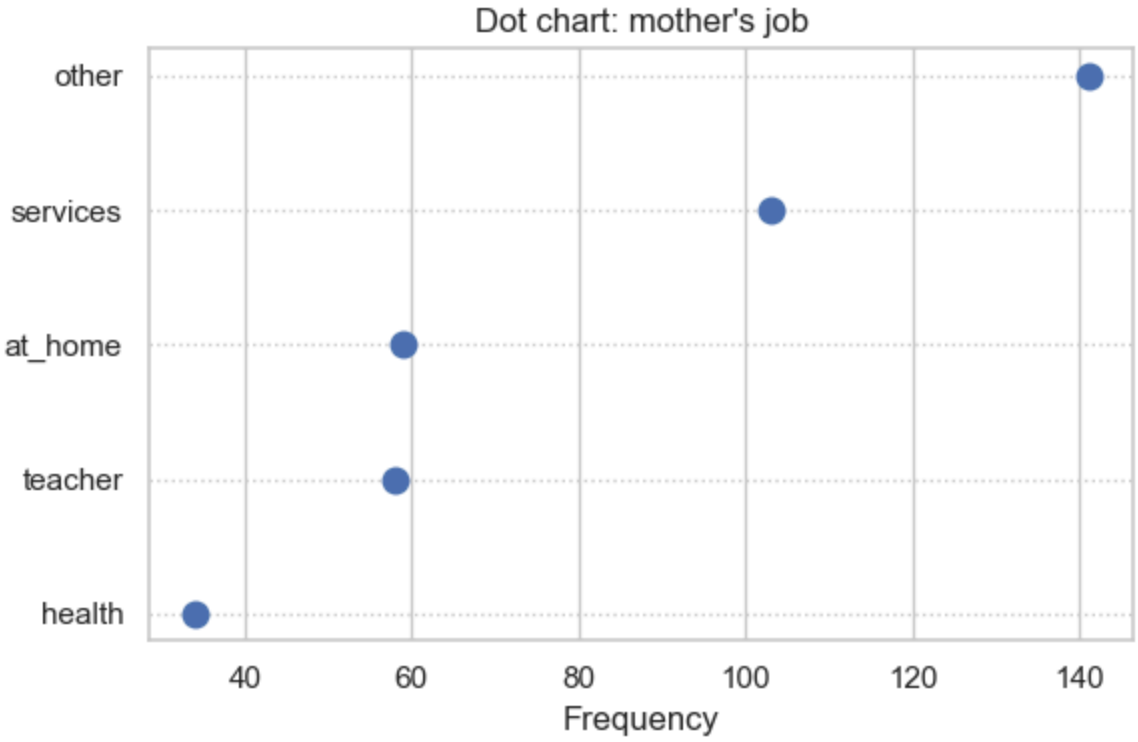
[figure] m03_25_percent_bar_sex.png



[figure] m03_26_pie_sex.png



[figure] m03_27_lollipop_Mjob.png



[figure] m03_28_dotchart_Mjob.png

Out[4]: {'rows_after_extreme_outlier_removal': 390}

4. Data Transformation & Merging

Joins/concatenation on toy tables, min-max / z-score / decimal scaling (also applied to real columns).

In [5]:
```
# ---------------------------------------------------------------------
# 4. Data Transformation & Merging
# ---------------------------------------------------------------------

# Re-import for standalone cell execution
import numpy as np
import pandas as pd

# Title for this section
TITLE = "AST01 - Merging & Data Transformation"

def _toy_frames():
    """Create three small DataFrames to demonstrate merging operations.

    These toy datasets simulate product/location tables used in the R lab.
    Typos from the lab sheet fixed (Mumabai->Mumbai, Tpoothpaste->Toothpaste).
    """
    # Product details: 10 products with IDs, names, and prices
    df1 = pd.DataFrame({
        "ProductID": range(1, 11),
        "NameofProd": ["Jeans", "Tshirt", "Shirt", "Trousers", "Sandals", "Shoes",
                       "Watch", "Sun Glasses", "Socks", "Blazer"],
        "Price": [4000, 2500, 4500, 3000, 3500, 5000, 7500, 9000, 150, 6000]})

    # Company locations: overlapping + non-overlapping ProductIDs (11,12 not in df1
    df2 = pd.DataFrame({
        "ProductID": [1, 2, 3, 4, 5, 6, 7, 8, 11, 12],
        "Company_Location": ["Delhi", "Bangalore", "Mumbai", "Chennai", "Bhubaneswa
                             "Pune", "Kolkata", "Hyderabad", "Vizag", "Berhampur"]}

    # Second product table: IDs 101-110 (no overlap with df1 for append demo)
    df3 = pd.DataFrame({
        "ProductID": range(101, 111),
        "NameofProd": ["Facewash", "Shampoo", "Sunscreen", "Body Lotion", "Conditio
                       "Toothpaste", "Toothbrush", "Belt", "Napkin", "Towel"],
        "Price": [1000, 250, 1200, 450, 500, 180, 50, 2300, 100, 700]})

    return df1, df2, df3

def part_a_dataframes():
    """Part A: demonstrate DataFrame operations - append, concat, merge, and all jo
    U.section("Part A - DataFrame operations (append / concat / merge / joins)")
    df1, df2, df3 = _toy_frames()

    # rbind equivalent: stack df1 and df3 vertically (same columns, different rows)
    appended = pd.concat([df1, df3], ignore_index=True)
    print("rbind(df1, df3):", appended.shape)
    U.save_table(appended, "appended", index=False)
```

```
    # cbind equivalent: paste df1 and df2 side by side (duplicate ProductID kept, a
    concatenated = pd.concat([df1, df2], axis=1)
    print("cbind(df1, df2):\n", concatenated.head())
    U.save_table(concatenated, "concatenated", index=False)

    # merge on a shared key column (inner join by default)
    merged = df1.merge(df2, on="ProductID")
    print("merge on ProductID:\n", merged)

    # Demonstrate all four join types: inner, left, right, full outer
    for how, label in [("inner", "inner_join"), ("left", "left_join"),
                       ("right", "right_join"), ("outer", "full_outer_join")]:
        j = df1.merge(df2, on="ProductID", how=how).sort_values("ProductID")
        print(f"\n{label} -> {j.shape[0]} rows, NaNs={int(j.isna().sum().sum())}")
        U.save_table(j, label, index=False)

def part_b_vector():
    """Part B: apply data transformations (min-max, z-score, decimal scaling) on a
    U.section("Part B - Data transformations on the 10-value vector")

    # Vector derived from registration number 23BDS0181 -> "0181" -> 181, then nine
    kotni = np.array([181, 12, 45, 78, 23, 56, 89, 34, 67, 90], dtype=float)

    # Build a table comparing different scaling methods
    res = pd.DataFrame({
        "original": kotni,
        # Min-max to range [-3, 3]
        "minmax_-3_3": U.minmax(kotni, -3, 3),
        # Min-max to range [-1, 1]
        "minmax_-1_1": U.minmax(kotni, -1, 1),
        # Min-max to range [0, 1]
        "minmax_0_1": U.minmax(kotni, 0, 1),
        # Z-score: (x - mean) / sample_std
        "zscore": (kotni - kotni.mean()) / kotni.std(ddof=1),
    })

    # Decimal scaling: divide by 10^j where j = ceil(log10(max(|x|)))
    j = int(np.ceil(np.log10(np.abs(kotni).max())))
    res["decimal_scaling"] = kotni / 10 ** j
    print(f"Decimal scaling j = {j}")
    print(res.round(4))
    U.save_table(res.round(6), "vector_transformations", index=False)

    # Plot: left = rescaled versions overlaid; right = original values as bar chart
    fig, axes = U.plt.subplots(1, 2, figsize=(12, 4))
    res.drop(columns=["original", "decimal_scaling"]).plot(ax=axes[0], marker="o")
    axes[0].set_title("Rescaled versions of the vector")
    axes[0].set_xlabel("Index")
    axes[1].bar(range(10), res["original"], color="steelblue")
    axes[1].set_title("Original vector")
    axes[1].set_xlabel("Index")
    U.save_fig("vector_transformations", fig)

def part_c_student_scaling():
    """Part C (extension): apply the same transforms to real columns of the student
```

```
    U.section("Part C (extension) - scaling real columns of student-mat.csv")

    # Import scikit-learn scalers for min-max and standard (z-score) scaling
    from sklearn.preprocessing import MinMaxScaler, StandardScaler

    # Load the cleaned student data
    df = U.load_student(clean=True)

    # Select a few numeric columns to demonstrate scaling
    cols = ["age", "absences", "G1", "G2", "G3"]

    # Min-max scaling: transforms each column to [0, 1]
    mm = pd.DataFrame(MinMaxScaler().fit_transform(df[cols]), columns=cols)

    # Z-score scaling: transforms each column to mean=0, std=1 (uses population std
    zs = pd.DataFrame(StandardScaler().fit_transform(df[cols]), columns=cols)

    # Decimal scaling: divide by 10^(ceil(log10(max(|x|)))) for each column
    ds = df[cols] / 10 ** np.ceil(np.log10(df[cols].abs().max()))

    print("Min-max head:\n", mm.head().round(3))

    # Save summary statistics for each scaling method
    U.save_table(mm.describe().round(3), "student_minmax_describe")
    U.save_table(zs.describe().round(3), "student_zscore_describe")
    U.save_table(ds.describe().round(3), "student_decimal_describe")

    # Box plot comparison of the three scaling methods
    fig, axes = U.plt.subplots(1, 3, figsize=(14, 3.6))
    for ax, (name, d) in zip(axes, [("Min-Max [0,1]", mm), ("Z-score", zs), ("Decim
        d.boxplot(ax=ax)
        ax.set_title(name)
    U.save_fig("student_scaling_boxplots", fig)

def run():
    """Execute all three parts of section 4."""
    part_a_dataframes()
    part_b_vector()
    part_c_student_scaling()

# Set the module label and run
U.set_module("m01")
run()
```

```
================================================================================
Part A - DataFrame operations (append / concat / merge / joins)
================================================================================
rbind(df1, df3): (20, 3)
[table]  m01_appended.csv
cbind(df1, df2):
    ProductID NameofProd  Price  ProductID Company_Location
0           1      Jeans   4000          1            Delhi
1           2     Tshirt   2500          2        Bangalore
2           3      Shirt   4500          3           Mumbai
3           4   Trousers   3000          4          Chennai
4           5    Sandals   3500          5      Bhubaneswar
[table]  m01_concatenated.csv
merge on ProductID:
    ProductID   NameofProd  Price Company_Location
0           1        Jeans   4000            Delhi
1           2       Tshirt   2500        Bangalore
2           3        Shirt   4500           Mumbai
3           4     Trousers   3000          Chennai
4           5      Sandals   3500      Bhubaneswar
5           6        Shoes   5000             Pune
6           7        Watch   7500          Kolkata
7           8  Sun Glasses   9000        Hyderabad

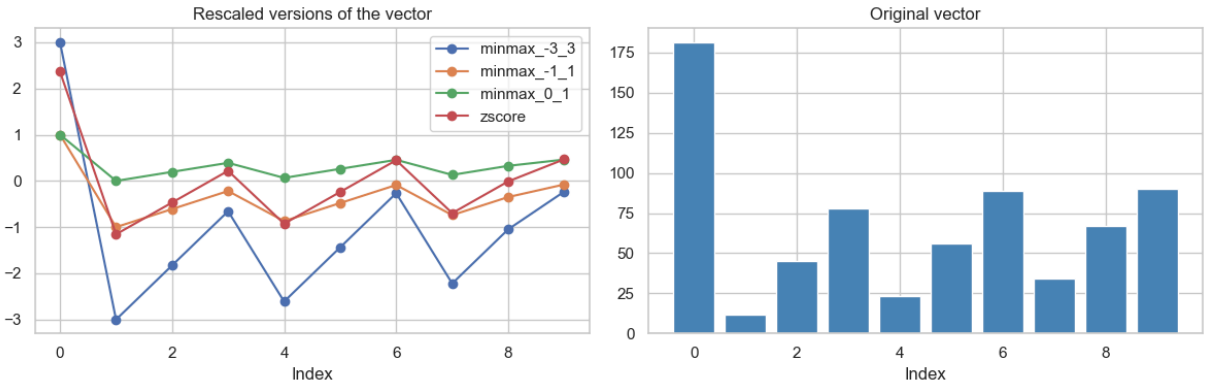
inner_join -> 8 rows, NaNs=0
[table]  m01_inner_join.csv

left_join -> 10 rows, NaNs=2
[table]  m01_left_join.csv

right_join -> 10 rows, NaNs=4
[table]  m01_right_join.csv

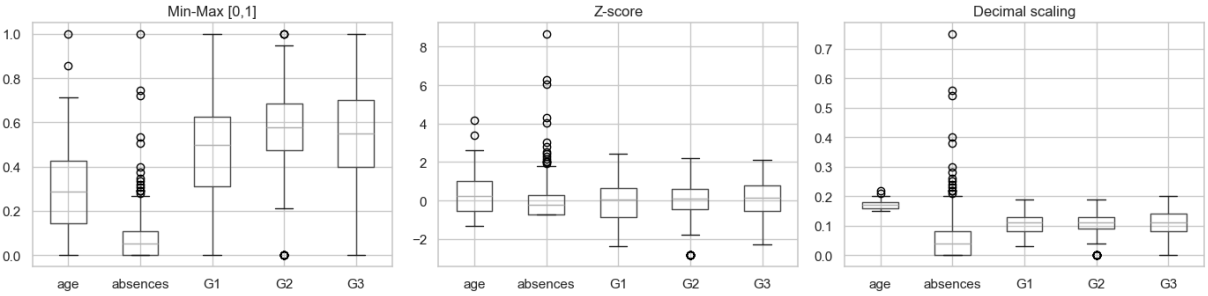
full_outer_join -> 12 rows, NaNs=6
[table]  m01_full_outer_join.csv

================================================================================
Part B - Data transformations on the 10-value vector
================================================================================
Decimal scaling j = 3
   original  minmax_-3_3  minmax_-1_1  minmax_0_1  zscore  decimal_scaling
0     181.0       3.0000       1.0000      1.0000  2.3619           0.181
1      12.0      -3.0000      -1.0000      0.0000 -1.1550           0.012
2      45.0      -1.8284      -0.6095      0.1953 -0.4682           0.045
3      78.0      -0.6568      -0.2189      0.3905  0.2185           0.078
4      23.0      -2.6095      -0.8698      0.0651 -0.9260           0.023
5      56.0      -1.4379      -0.4793      0.2604 -0.2393           0.056
6      89.0      -0.2663      -0.0888      0.4556  0.4474           0.089
7      34.0      -2.2189      -0.7396      0.1302 -0.6971           0.034
8      67.0      -1.0473      -0.3491      0.3254 -0.0104           0.067
9      90.0      -0.2308      -0.0769      0.4615  0.4682           0.090
[table]  m01_vector_transformations.csv
```



[figure] m01_vector_transformations.png

```
================================================================================
Part C (extension) - scaling real columns of student-mat.csv
================================================================================
Min-max head:
      age  absences     G1     G2    G3
0   0.429     0.080  0.125  0.316  0.30
1   0.286     0.053  0.125  0.263  0.30
2   0.000     0.133  0.250  0.421  0.50
3   0.000     0.027  0.750  0.737  0.75
4   0.143     0.053  0.188  0.526  0.50
[table]  m01_student_minmax_describe.csv
[table]  m01_student_zscore_describe.csv
[table]  m01_student_decimal_describe.csv
```



[figure] m01_student_scaling_boxplots.png

5. Bivariate Analysis

Numeric vs numeric, categorical vs numeric, categorical vs categorical, correlation matrix.

In [6]:
```
# ----------------------------------------------------------------------
# 5. Bivariate Analysis
# ----------------------------------------------------------------------

# Re-import for standalone cell execution
import numpy as np
import pandas as pd
import seaborn as sns

# Create a shortcut to matplotlib.pyplot
plt = U.plt
```

```python
# Title for this section
TITLE = "AST04 - Bivariate Analysis"

def run():
    # Load cleaned student data (no NA group in legends, improvement over raw R plo
    d = U.load_student(clean=True)

    # 2. Contingency table + stacked bar chart (sex by school)
    U.section("2. Contingency table + stacked bars (sex x school)")
    # Cross-tabulation counts how many students fall into each sex-school combinati
    ct = pd.crosstab(d["sex"], d["school"])
    print(ct)
    U.save_table(ct, "contingency_sex_school")
    # Stacked bar using pandas/matplotlib
    ct.T.plot(kind="bar", stacked=True, color=["lightpink", "lightblue"], figsize=(
    plt.title("Sex distribution by school (matplotlib/pandas)")
    plt.ylabel("Number of students")
    U.save_fig("02a_stacked_bar_base")
    # Same stacked bar using seaborn's histplot
    fig, ax = plt.subplots(figsize=(6, 4))
    sns.histplot(data=d, x="school", hue="sex", multiple="stack", shrink=0.8, ax=ax
    ax.set(title="Sex distribution by school (seaborn)", ylabel="Number of students
    U.save_fig("02b_stacked_bar_seaborn")

    # 3. Grouped bar chart: higher education preference split by sex
    U.section("3. Grouped bars (higher education preference by sex)")
    fig, ax = plt.subplots(figsize=(6, 4))
    sns.countplot(data=d, x="sex", hue="higher", ax=ax)
    ax.set(title="Higher-education preference by sex", ylabel="Number of students")
    U.save_fig("03_grouped_bar_higher_sex")

    # 4. Scatter plot with regression trend line (G1 vs G3)
    U.section("4. Scatter + trend line (G1 vs G3)")
    # Calculate Pearson correlation between first period grade and final grade
    r = d["G1"].corr(d["G3"])
    fig, ax = plt.subplots(figsize=(6, 5))
    sns.regplot(data=d, x="G1", y="G3", line_kws={"color": "red"}, scatter_kws={"al
    ax.set_title(f"First vs final grade (r = {r:.3f})")
    U.save_fig("04_scatter_G1_G3")
    print(f"Pearson r(G1,G3) = {r:.4f}")

    # 5 & 7. Line plot and bar chart of mean grades across periods
    U.section("5 & 7. Line plot and bar chart of mean grades")
    # Calculate average grades for G1, G2, G3
    means = d[["G1", "G2", "G3"]].mean()
    print(means.round(5))
    # Line plot showing grade progression
    fig, ax = plt.subplots(figsize=(5.5, 4))
    ax.plot(means.index, means.values, "o-")
    ax.set(title="Average student grades", xlabel="Grade period", ylabel="Average g
    U.save_fig("05_line_mean_grades")
    # Bar chart of the same means
    fig, ax = plt.subplots(figsize=(5.5, 4))
    ax.bar(means.index, means.values, color="skyblue", edgecolor="black")
    ax.set(title="Average grades comparison", ylabel="Average marks")
    U.save_fig("07_bar_mean_grades")
```

```
    # 6 & 12. Correlation heatmap and correlogram
    U.section("6 & 12. Correlation heatmap / correlogram")
    # Heatmap for selected columns
    cols6 = ["studytime", "absences", "G1", "G2", "G3"]
    fig, ax = plt.subplots(figsize=(6, 5))
    sns.heatmap(d[cols6].corr(), annot=True, fmt=".2f", cmap="YlOrRd", ax=ax)
    ax.set_title("Correlation heatmap")
    ax.tick_params(axis="y", labelrotation=0)
    U.save_fig("06_heatmap")

    # Full correlation matrix for core numeric columns
    corr = d[U.CORE_NUM].corr()
    print(corr.round(3))
    U.save_table(corr.round(4), "correlation_matrix")

    # Lower-triangle correlogram (masks upper triangle)
    mask = np.triu(np.ones_like(corr, dtype=bool))
    fig, ax = plt.subplots(figsize=(7, 6))
    sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="coolwarm", center=0,
                square=True, ax=ax)
    ax.set_title("Correlogram (lower triangle)")
    U.save_fig("12_correlogram")

    # 8-11. Distribution of G3 by sex: density, box, violin, and violin+box combo
    U.section("8-11. Density, box, violin, violin+box of G3 by sex")

    # Density plot comparing male and female G3 distributions
    fig, ax = plt.subplots(figsize=(7, 4))
    sns.kdeplot(data=d, x="G3", hue="sex", fill=True, alpha=0.5, common_norm=False,
    ax.set_title("Distribution of final grade by sex")
    U.save_fig("08_density_G3_sex")

    # Box plot of G3 by sex
    fig, ax = plt.subplots(figsize=(5, 4.5))
    sns.boxplot(data=d, x="sex", y="G3", hue="sex", legend=False, ax=ax)
    ax.set_title("Final grade by sex")
    U.save_fig("09_box_G3_sex")

    # Violin plot of G3 by sex (shows full distribution shape)
    fig, ax = plt.subplots(figsize=(5, 4.5))
    sns.violinplot(data=d, x="sex", y="G3", hue="sex", legend=False, cut=0, ax=ax)
    ax.set_title("Final grade by sex (violin)")
    U.save_fig("10_violin_G3_sex")

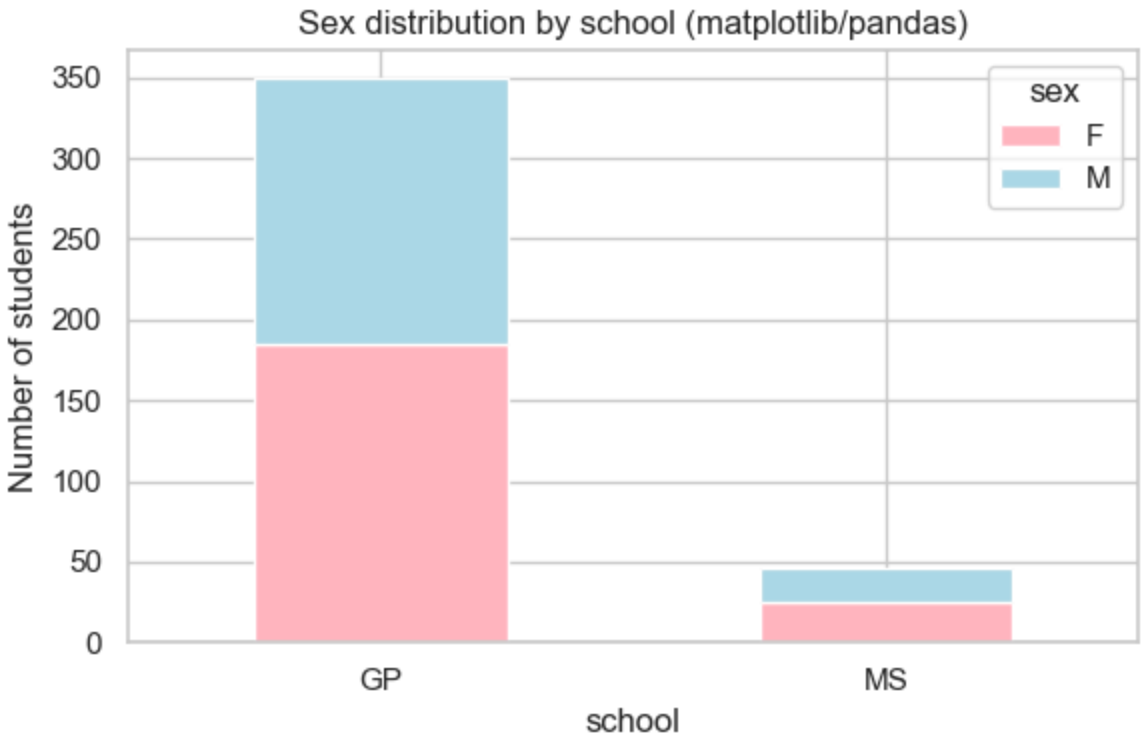
    # Combined violin + box plot for detailed view
    fig, ax = plt.subplots(figsize=(5, 4.5))
    sns.violinplot(data=d, x="sex", y="G3", hue="sex", legend=False, alpha=0.6, inn
    sns.boxplot(data=d, x="sex", y="G3", width=0.15, color="white", fliersize=3, ax
    ax.set_title("Violin + box: final grade by sex")
    U.save_fig("11_violin_box_G3_sex")

    # Print descriptive statistics by sex
    print(d.groupby("sex")["G3"].describe().round(2))

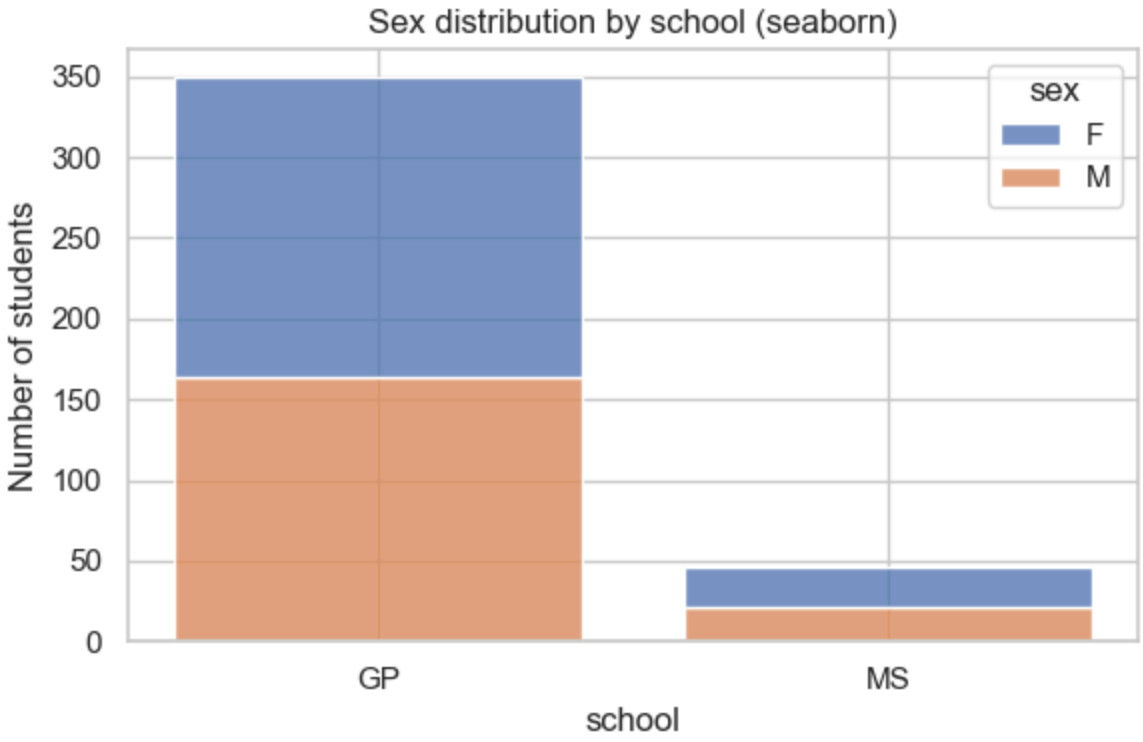
# Set the module label and run
```

```
U.set_module("m04")
run()
```

```
================================================================================
2. Contingency table + stacked bars (sex x school)
================================================================================
school   GP  MS
sex
F       185  25
M       164  21
[table]  m04_contingency_sex_school.csv
```

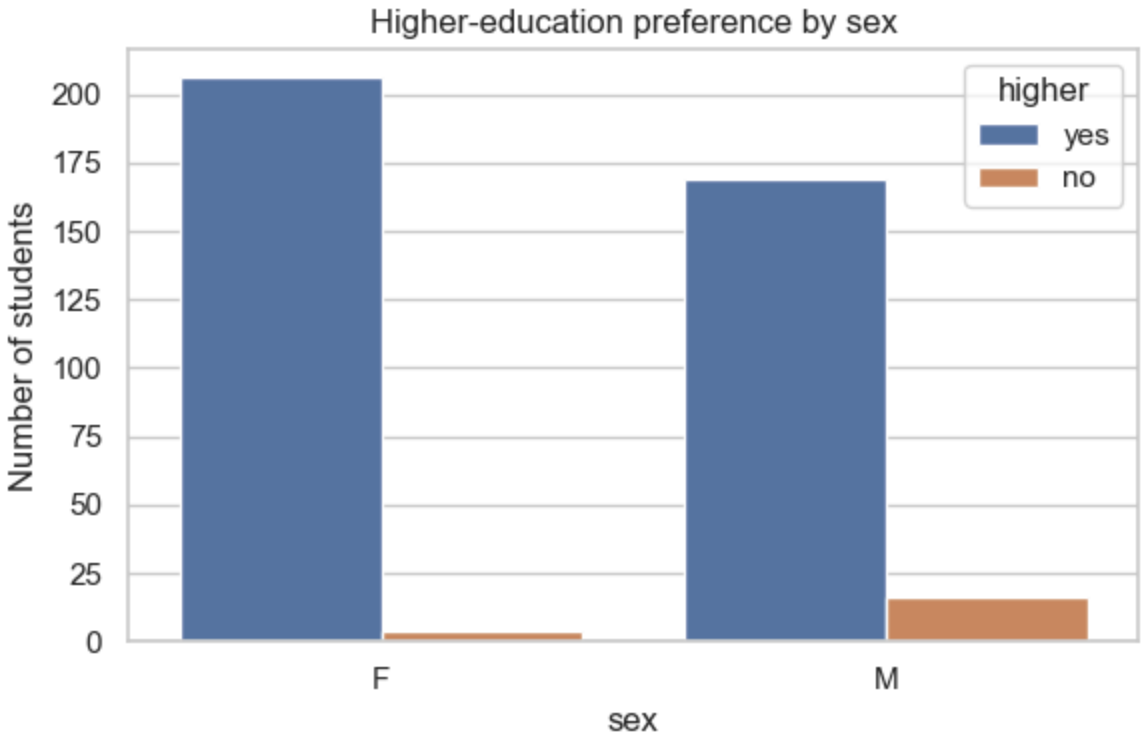


[figure] m04_02a_stacked_bar_base.png



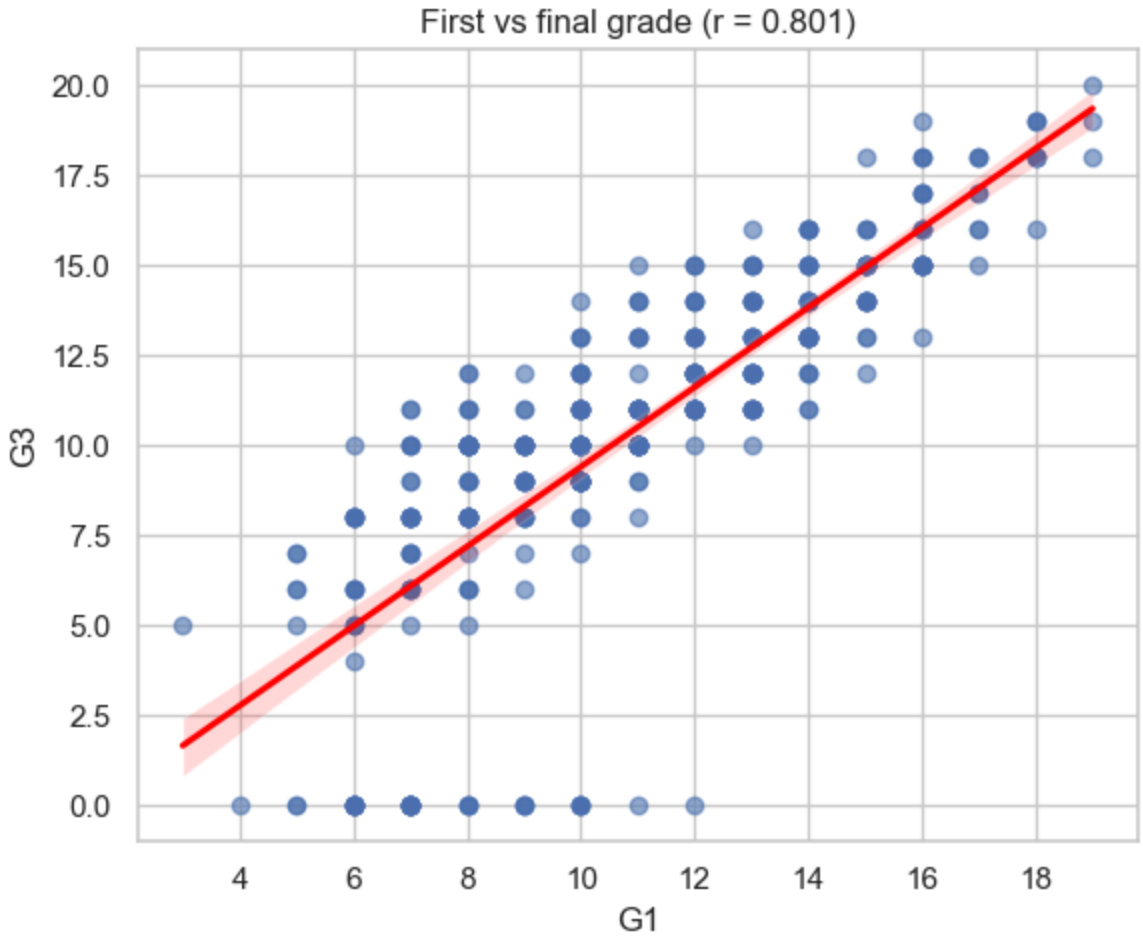
[figure] m04_02b_stacked_bar_seaborn.png

==========
3. Grouped bars (higher education preference by sex)
==========



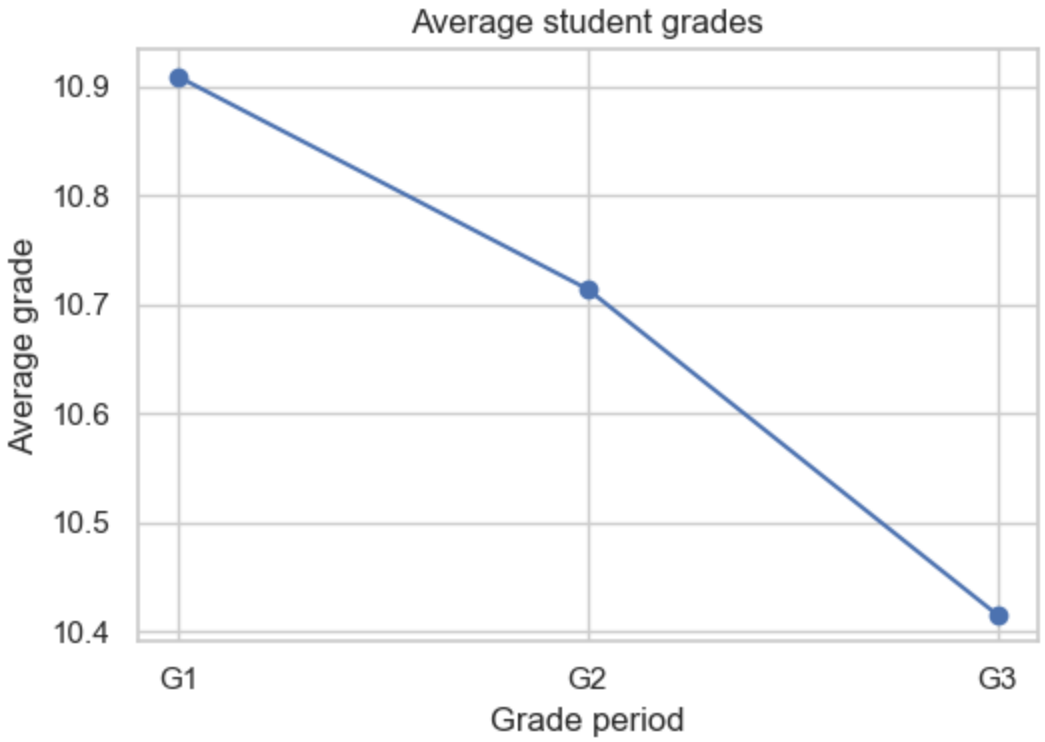
[figure] m04_03_grouped_bar_higher_sex.png

==========
4. Scatter + trend line (G1 vs G3)
==========

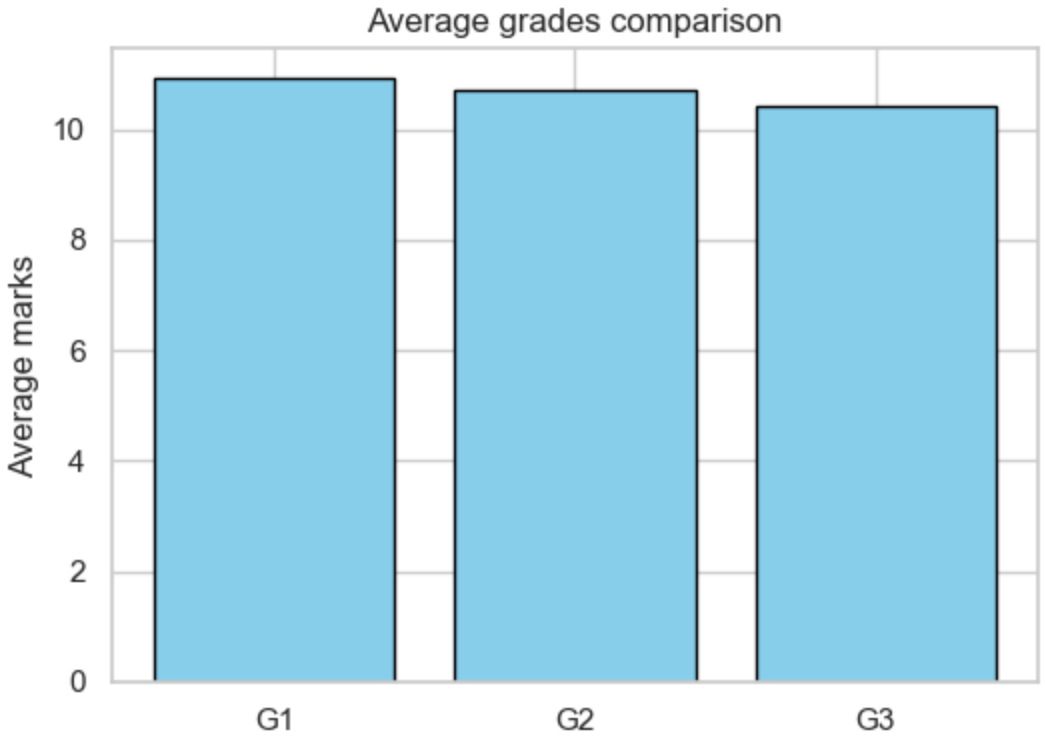


[figure] m04_04_scatter_G1_G3.png
Pearson r(G1,G3) = 0.8015

==========
5 & 7. Line plot and bar chart of mean grades
==========
G1   10.90886
G2   10.71392
G3   10.41519
dtype: float64

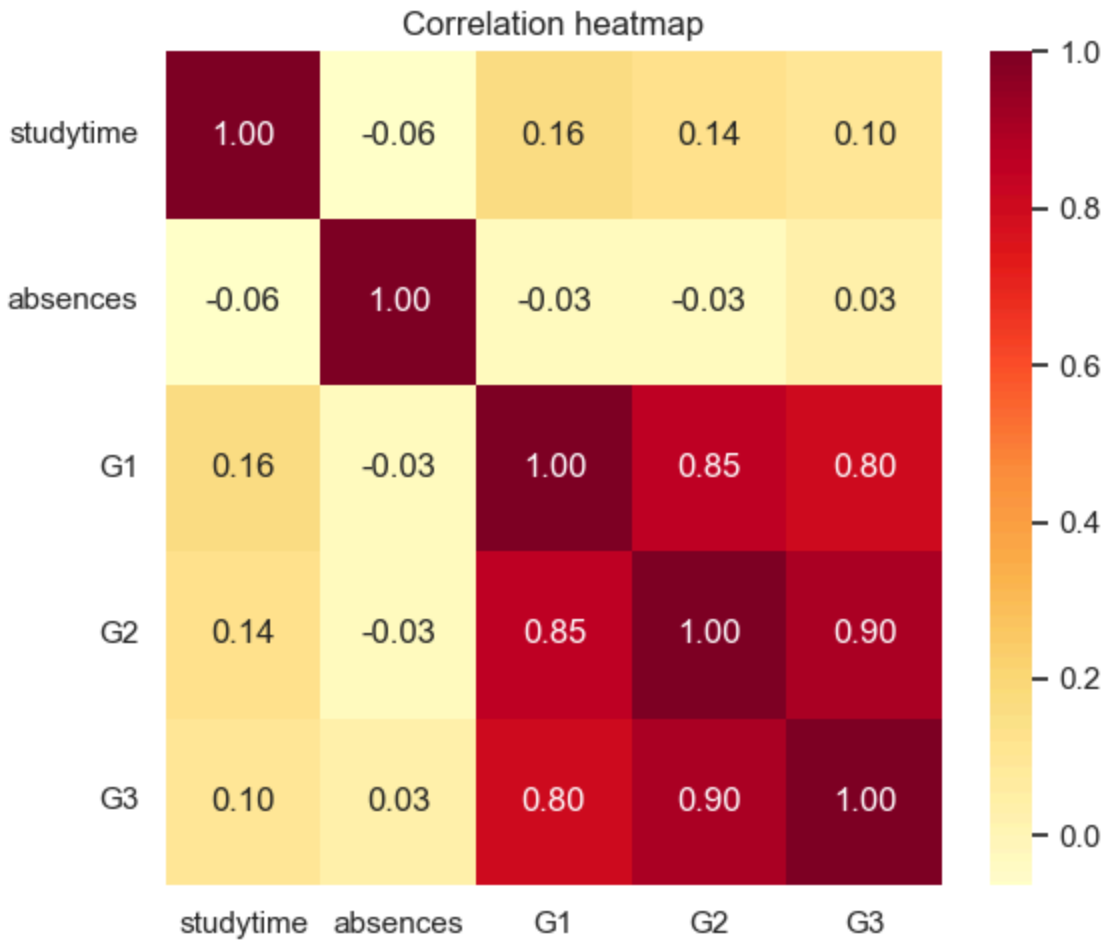


[figure] m04_05_line_mean_grades.png



[figure] m04_07_bar_mean_grades.png

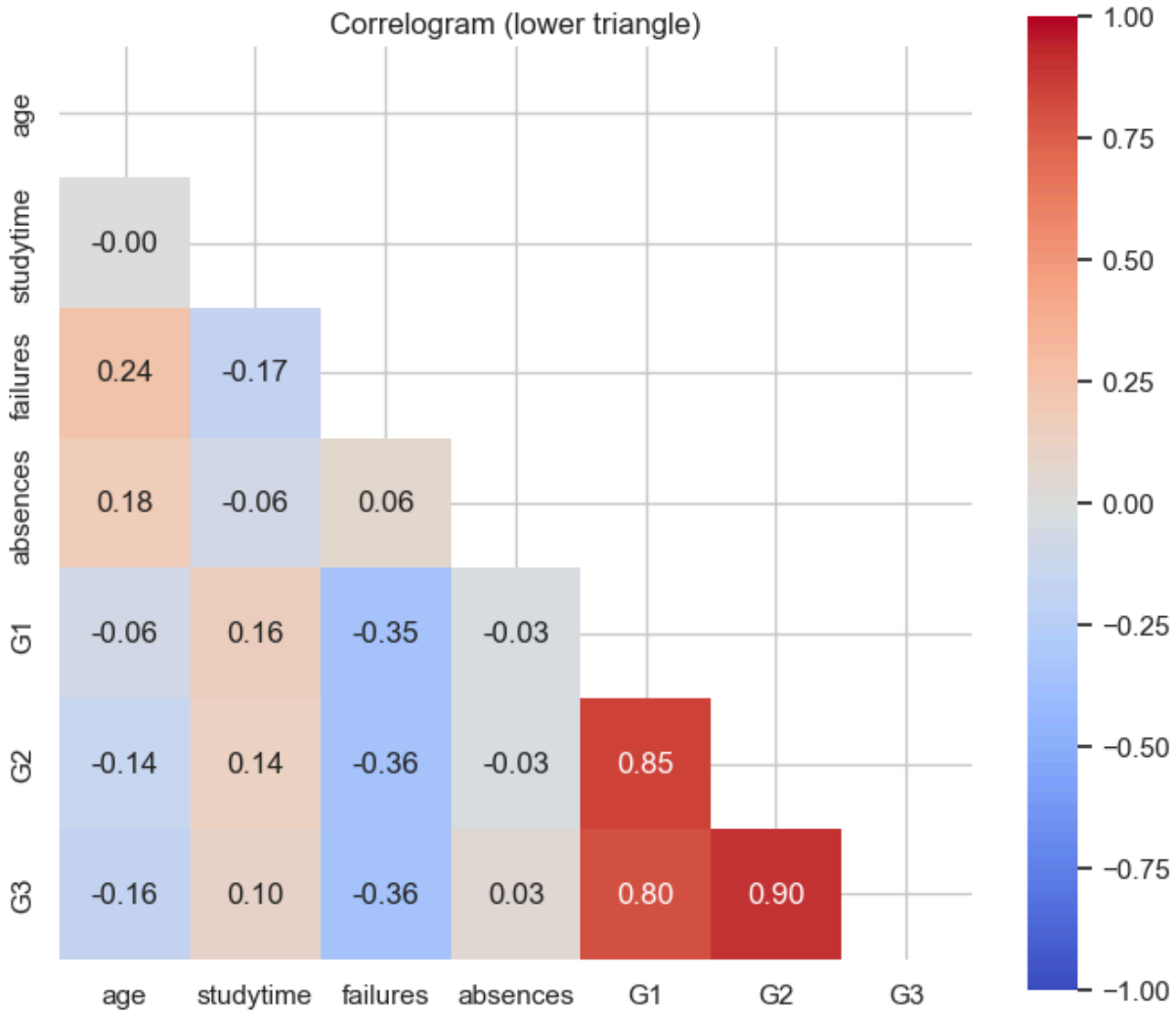
```
================================================================================
6 & 12. Correlation heatmap / correlogram
================================================================================
```



[figure] m04_06_heatmap.png

	age	studytime	failures	absences	G1	G2	G3
age	1.000	-0.004	0.244	0.175	-0.064	-0.143	-0.162
studytime	-0.004	1.000	-0.174	-0.063	0.161	0.136	0.098
failures	0.244	-0.174	1.000	0.064	-0.355	-0.356	-0.360
absences	0.175	-0.063	0.064	1.000	-0.031	-0.032	0.034
G1	-0.064	0.161	-0.355	-0.031	1.000	0.852	0.801
G2	-0.143	0.136	-0.356	-0.032	0.852	1.000	0.905
G3	-0.162	0.098	-0.360	0.034	0.801	0.905	1.000

[table] m04_correlation_matrix.csv

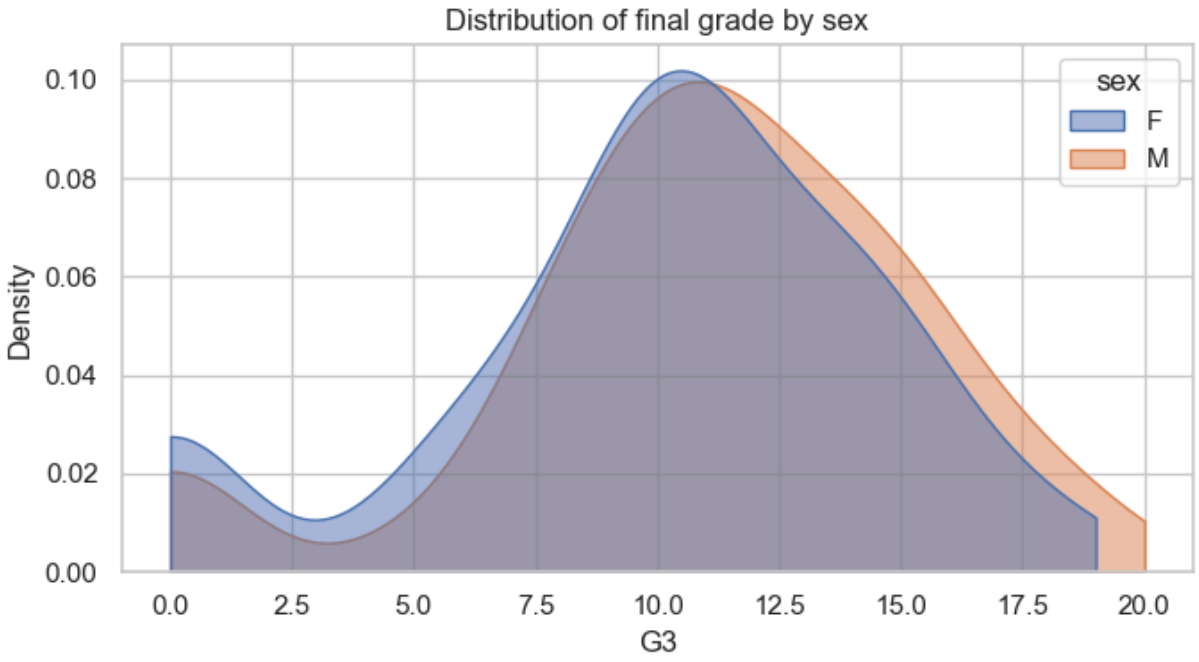


[figure] m04_12_correlogram.png

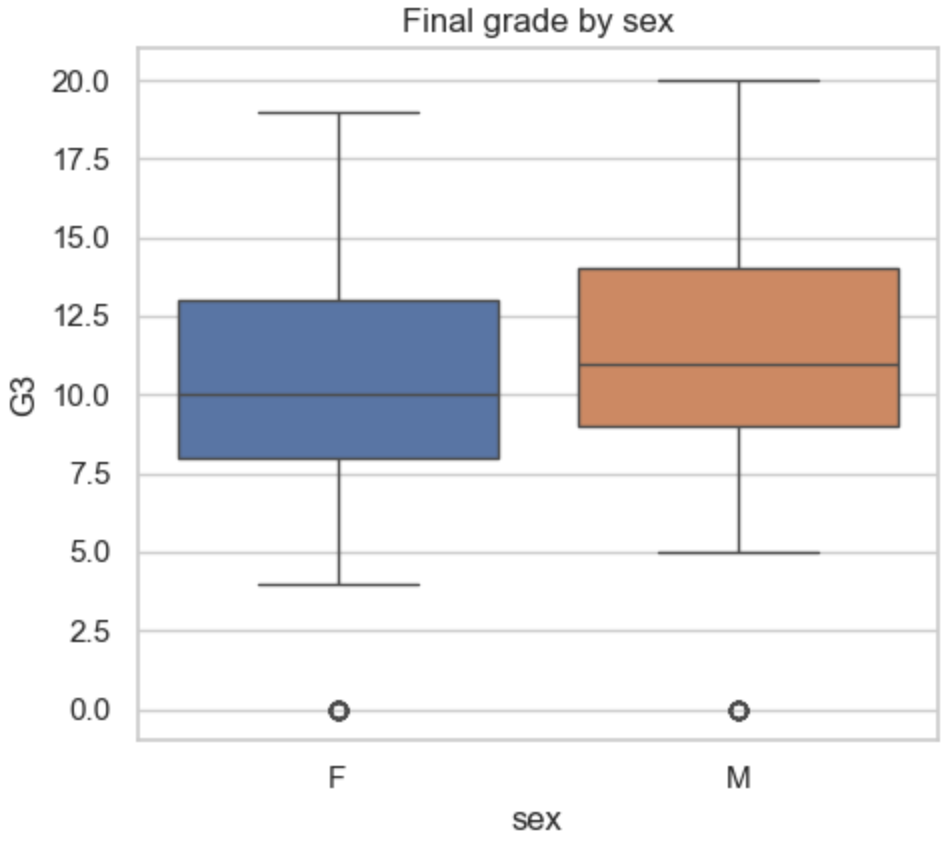
==========

8-11. Density, box, violin, violin+box of G3 by sex

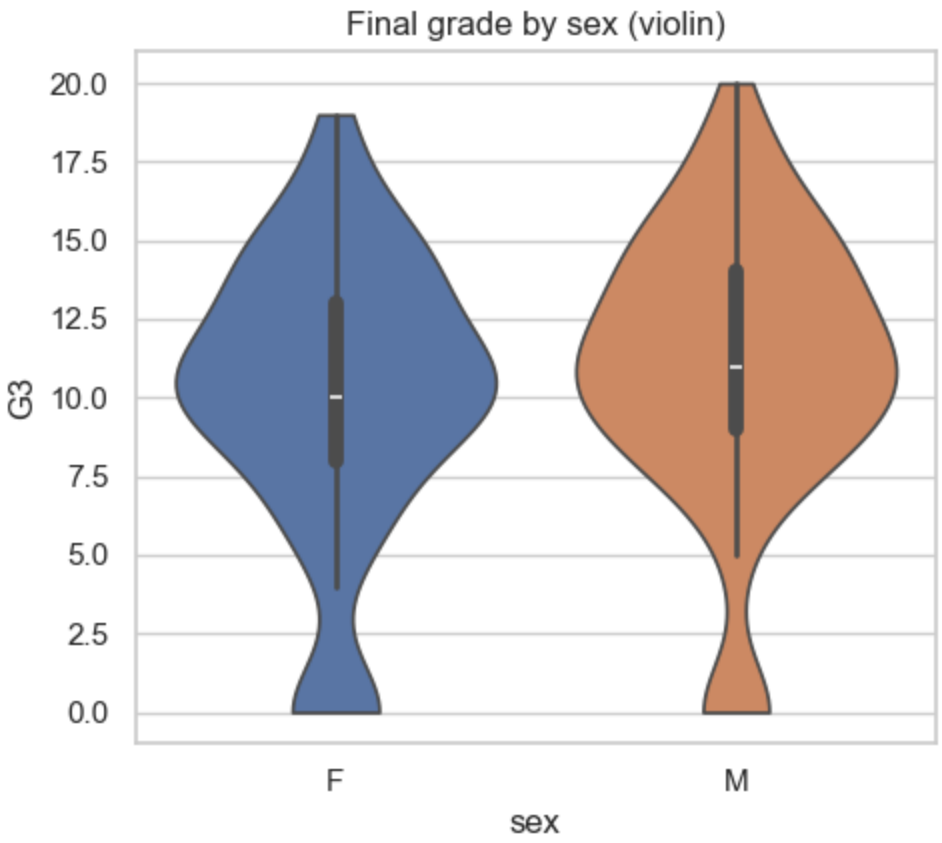
==========



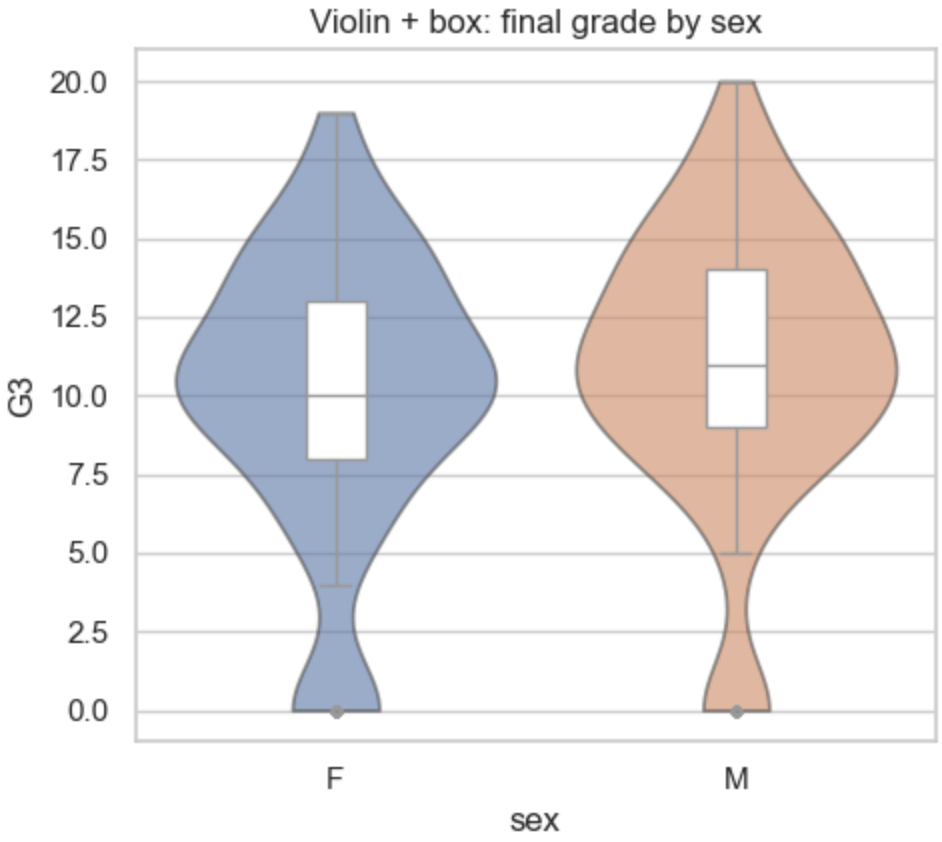
[figure] m04_08_density_G3_sex.png



[figure] m04_09_box_G3_sex.png



[figure] m04_10_violin_G3_sex.png



```
[figure] m04_11_violin_box_G3_sex.png
     count   mean   std  min  25%   50%   75%   max
sex
F    210.0   9.97  4.63  0.0  8.0  10.0  13.0  19.0
M    185.0  10.92  4.49  0.0  9.0  11.0  14.0  20.0
```

6. Multivariate Visualisation

Colour/shape/size encodings, facets, pairs plot, parallel coordinates.

In [7]:

```
# ---------------------------------------------------------------------
# 6. Multivariate Visualisation
# ---------------------------------------------------------------------

# Re-import for standalone cell execution
import pandas as pd
import seaborn as sns
# parallel_coordinates for multi-dimensional visualisation
from pandas.plotting import parallel_coordinates

# Create a shortcut to matplotlib.pyplot
plt = U.plt

# Title for this section
TITLE = "AST05 - Multivariate Visualisation"

def run():
    # Load cleaned student data
    d = U.load_student(clean=True)

    # 1. Identify numeric vs categorical variables
    U.section("1. Numerical vs categorical variables")
    num = d.select_dtypes(include="number").columns.tolist()
    cat = d.select_dtypes(exclude="number").columns.tolist()
    print(f"{len(num)} numeric: {num}\n{len(cat)} categorical: {cat}")

    # 2-5. Scatter plots with different visual encodings
    U.section("2-5. Scatter encodings: colour, shape, size, bubble")

    # Colour encoding: sex mapped to point colour
    fig, ax = plt.subplots(figsize=(7, 5))
    sns.scatterplot(data=d, x="G1", y="G3", hue="sex", s=45, ax=ax)
    ax.set_title("G1 vs G3 by sex (colour)")
    U.save_fig("02_scatter_color")

    # Colour + shape encoding: sex=colour, school=shape
    fig, ax = plt.subplots(figsize=(7, 5))
    sns.scatterplot(data=d, x="G1", y="G3", hue="sex", style="school", s=55, ax=ax)
    ax.set_title("G1 vs G3 by sex (colour) and school (shape)")
    U.save_fig("03_scatter_color_shape")

    # Colour + size encoding: sex=colour, absences=point size
    fig, ax = plt.subplots(figsize=(7.5, 5))
    sns.scatterplot(data=d, x="G1", y="G3", hue="sex", size="absences", sizes=(15,
```

```
    ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
    ax.set_title("G1 vs G3 - colour: sex, size: absences")
    U.save_fig("04_scatter_color_size")

    # Bubble plot: size represents absences, no colour grouping
    fig, ax = plt.subplots(figsize=(7.5, 5))
    sns.scatterplot(data=d, x="G1", y="G3", size="absences", sizes=(15, 300), alpha
    ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
    ax.set_title("Bubble plot: grades and absences")
    U.save_fig("05_bubble")

    # 6-7. Faceted scatter plots
    U.section("6-7. Facets (wrap and grid)")

    # Facet wrap: separate panels for each school
    g = sns.relplot(data=d, x="G1", y="G3", col="school", kind="scatter", height=4,
    g.figure.suptitle("G1 vs G3 for different schools", y=1.03)
    U.save_fig("06_facet_wrap_school", g.figure)

    # Facet grid: rows=sex, columns=school (2x2 panel layout)
    g = sns.relplot(data=d, x="G1", y="G3", row="sex", col="school", kind="scatter"
    g.figure.suptitle("G1 vs G3 by sex (rows) and school (columns)", y=1.02)
    U.save_fig("07_facet_grid_sex_school", g.figure)
    print(d.groupby(["sex", "school"])[["G1", "G3"]].mean().round(2))

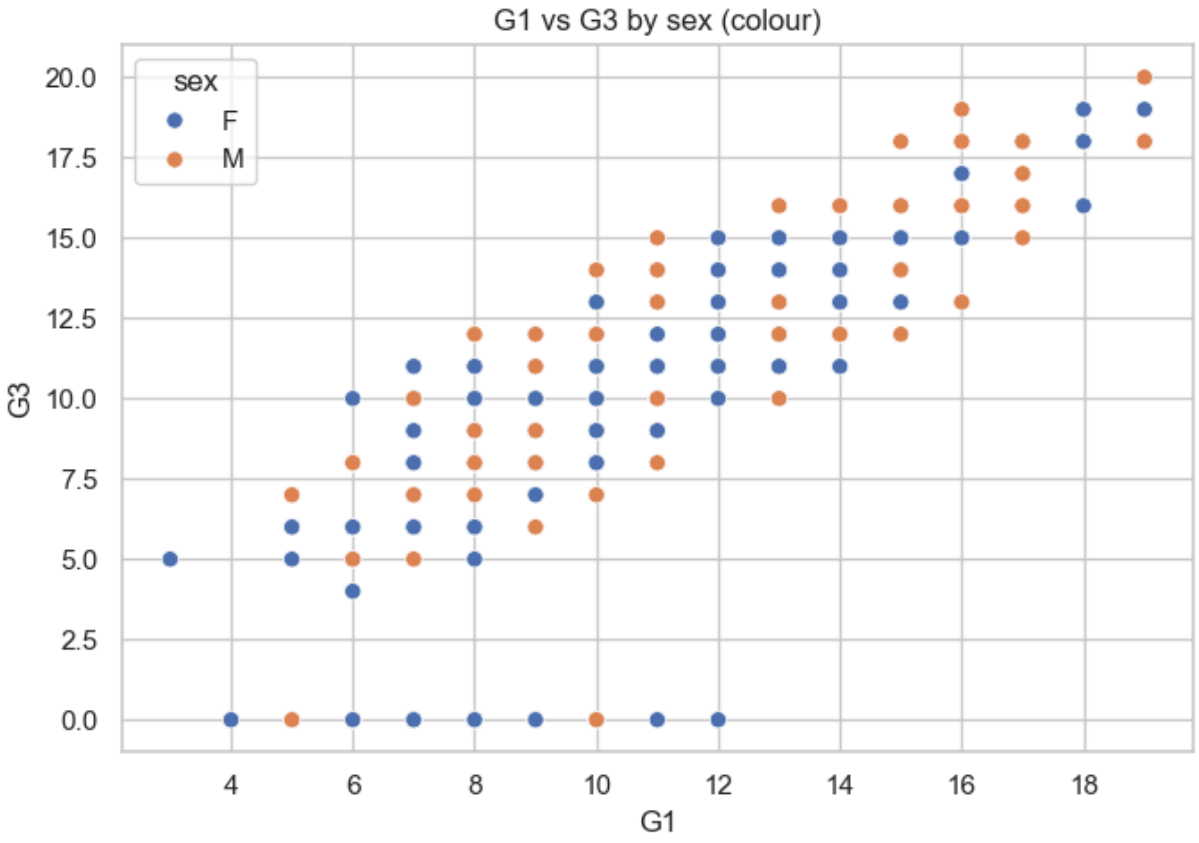
    # 8. Pairs plot (scatterplot matrix) for key numeric variables
    U.section("8. Pairs plot")
    pp = sns.pairplot(d[["G1", "G2", "G3", "absences"]], corner=False, diag_kind="k
                      plot_kws={"alpha": 0.5, "s": 18})
    pp.figure.suptitle("Pairs plot of student performance variables", y=1.02)
    U.save_fig("08_pairs", pp.figure)
    print(d[["G1", "G2", "G3", "absences"]].corr().round(3))

    # 9. Parallel coordinates plot coloured by sex
    U.section("9. Parallel coordinates (coloured by sex)")
    cols = ["G1", "G2", "G3", "absences"]
    p = d[cols].copy()
    # Apply 'uniminmax' scaling: scale each column to [0, 1] (as used in the R lab)
    p = (p - p.min()) / (p.max() - p.min())
    # Add sex as the grouping variable for colouring
    p["sex"] = d["sex"].values
    fig, ax = plt.subplots(figsize=(9, 5))
    parallel_coordinates(p, "sex", color=["#f8766d", "#00bfc4"], alpha=0.25, ax=ax)
    ax.set(title="Parallel coordinate plot of student performance", ylabel="Scaled
    U.save_fig("09_parallel_coordinates")

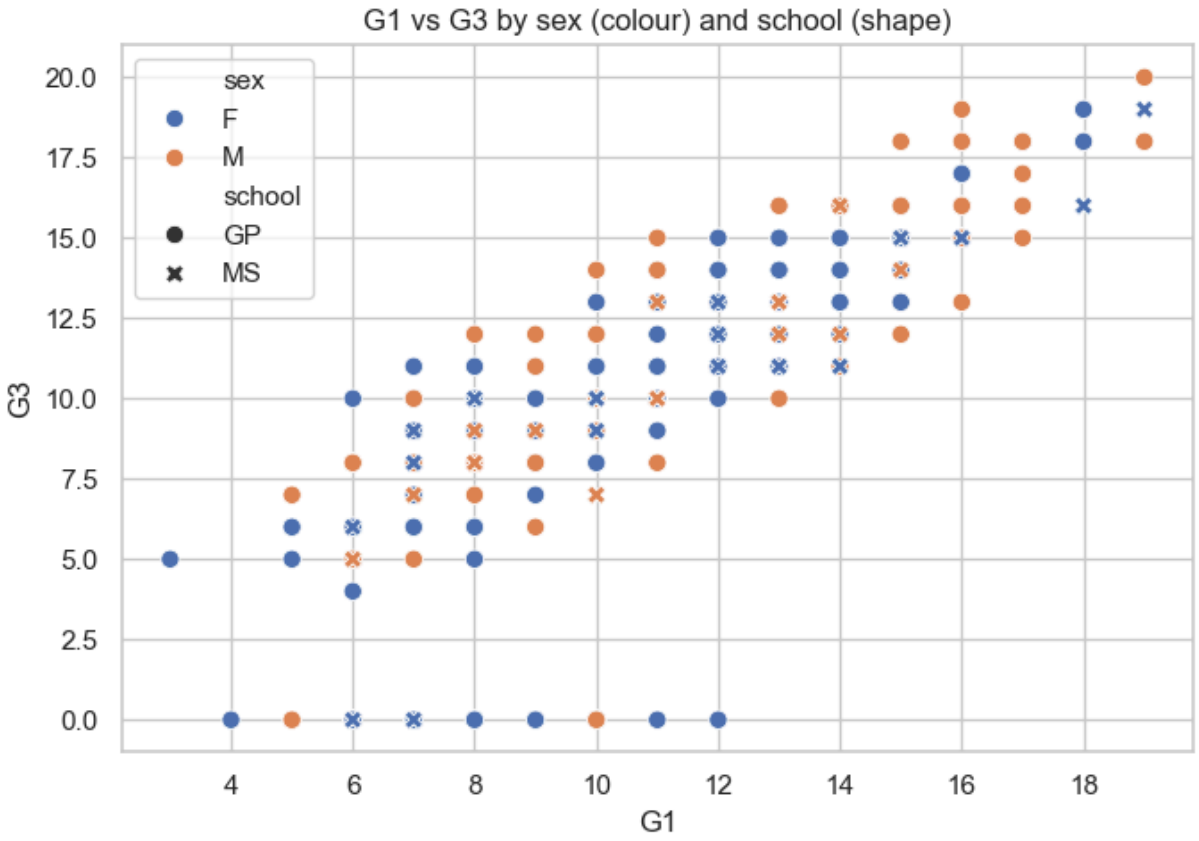
# Set the module label and run
U.set_module("m05")
run()
```

```
================================================================================
1. Numerical vs categorical variables
================================================================================
16 numeric: ['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'famrel',
'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2', 'G3']
17 categorical: ['school', 'sex', 'address', 'famsize', 'Pstatus', 'Mjob', 'Fjob',
'reason', 'guardian', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'highe
r', 'internet', 'romantic']

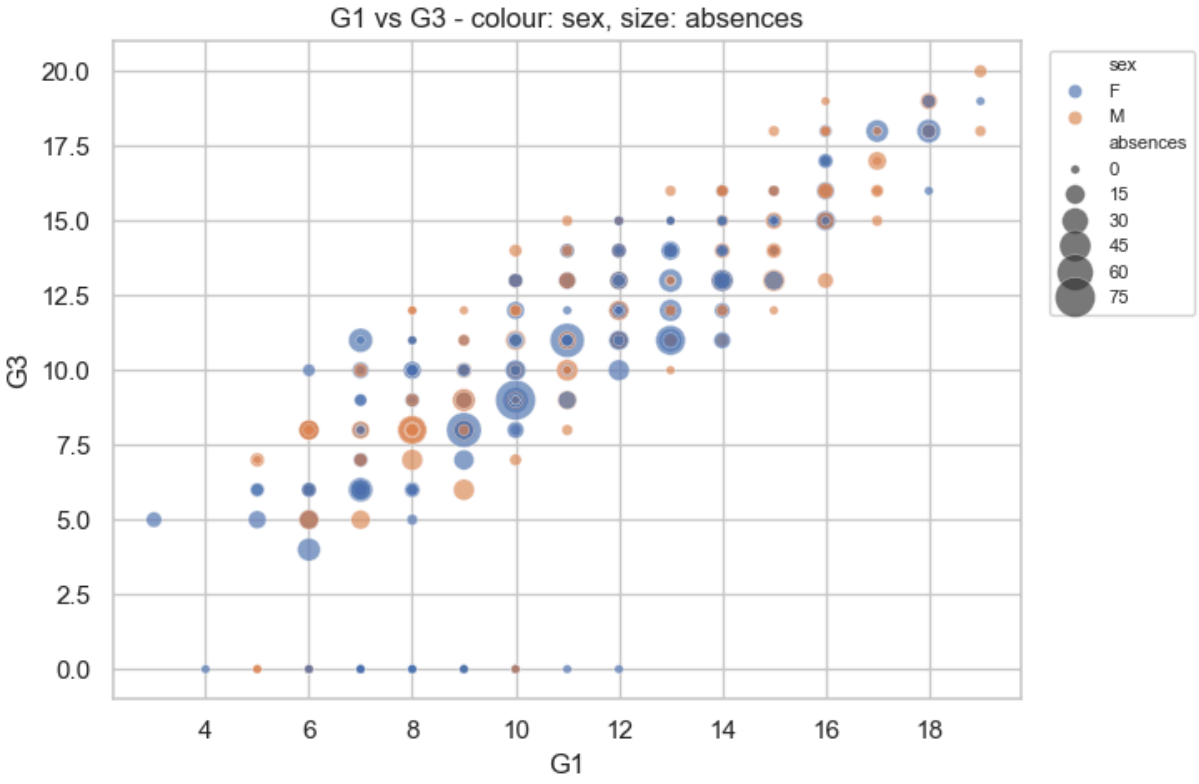
================================================================================
2-5. Scatter encodings: colour, shape, size, bubble
================================================================================
```



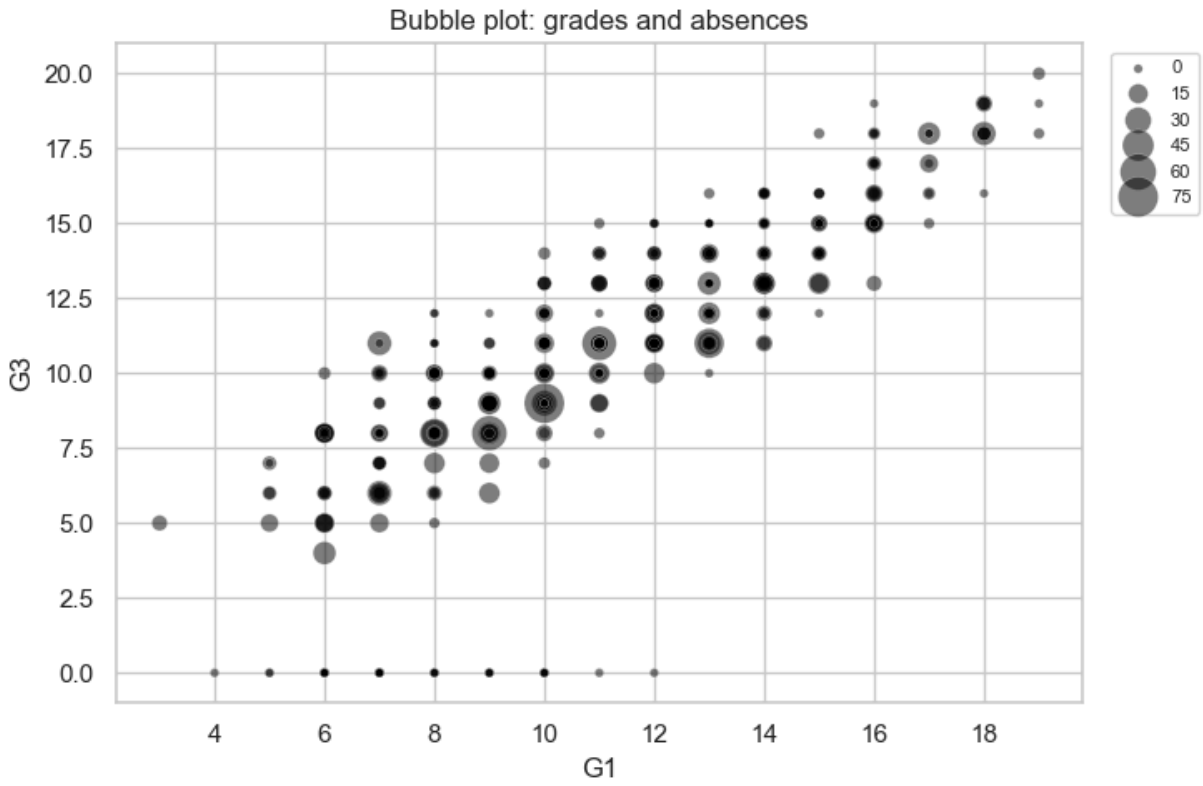
[figure] m05_02_scatter_color.png



[figure] m05_03_scatter_color_shape.png

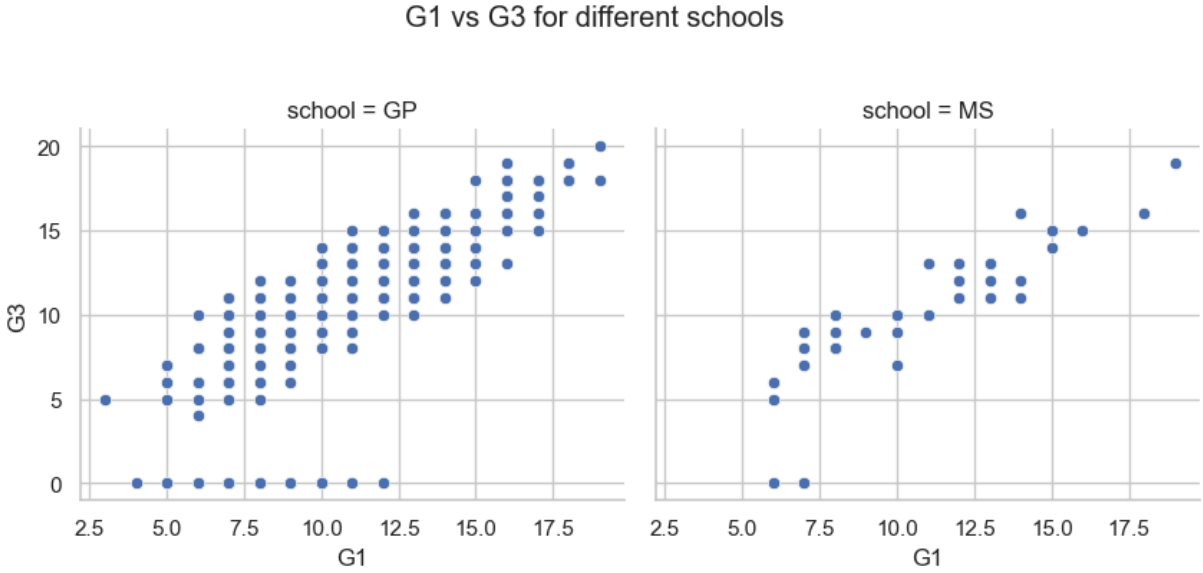


[figure] m05_04_scatter_color_size.png

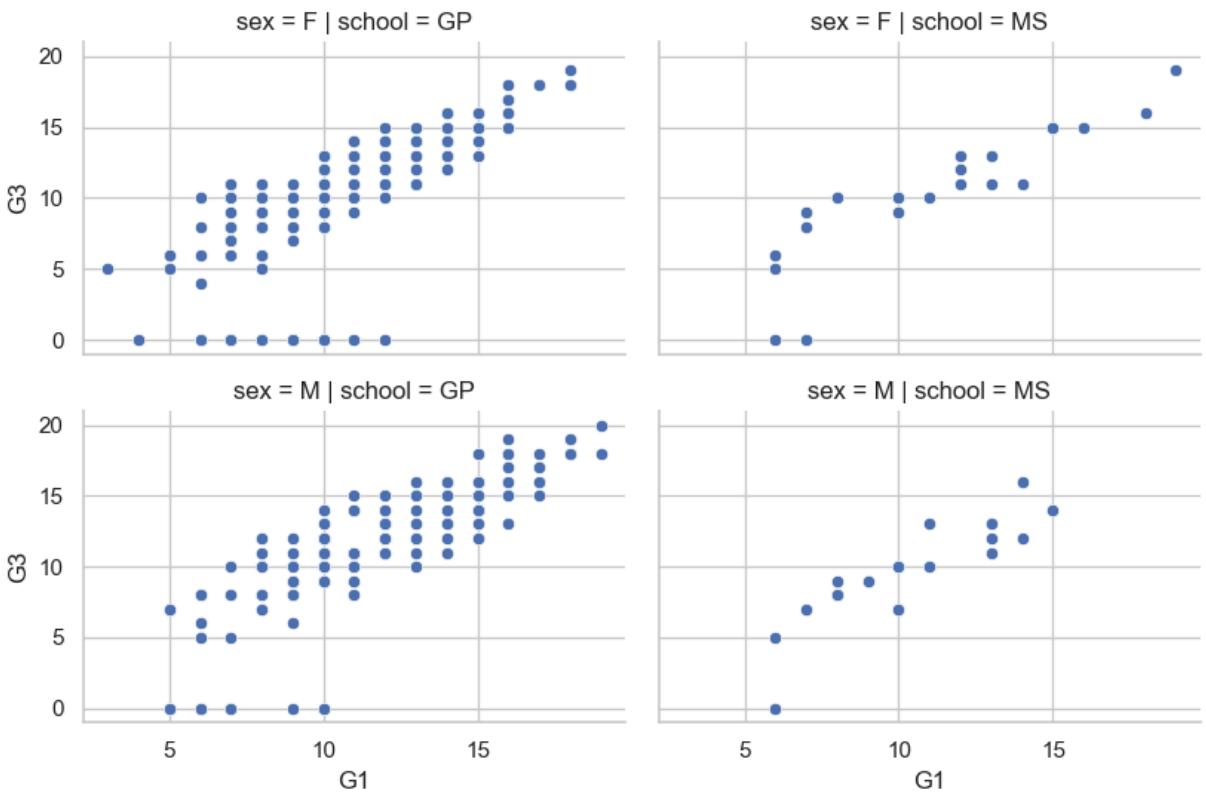


[figure] m05_05_bubble.png

==========
6-7. Facets (wrap and grid)
==========



[figure] m05_06_facet_wrap_school.png



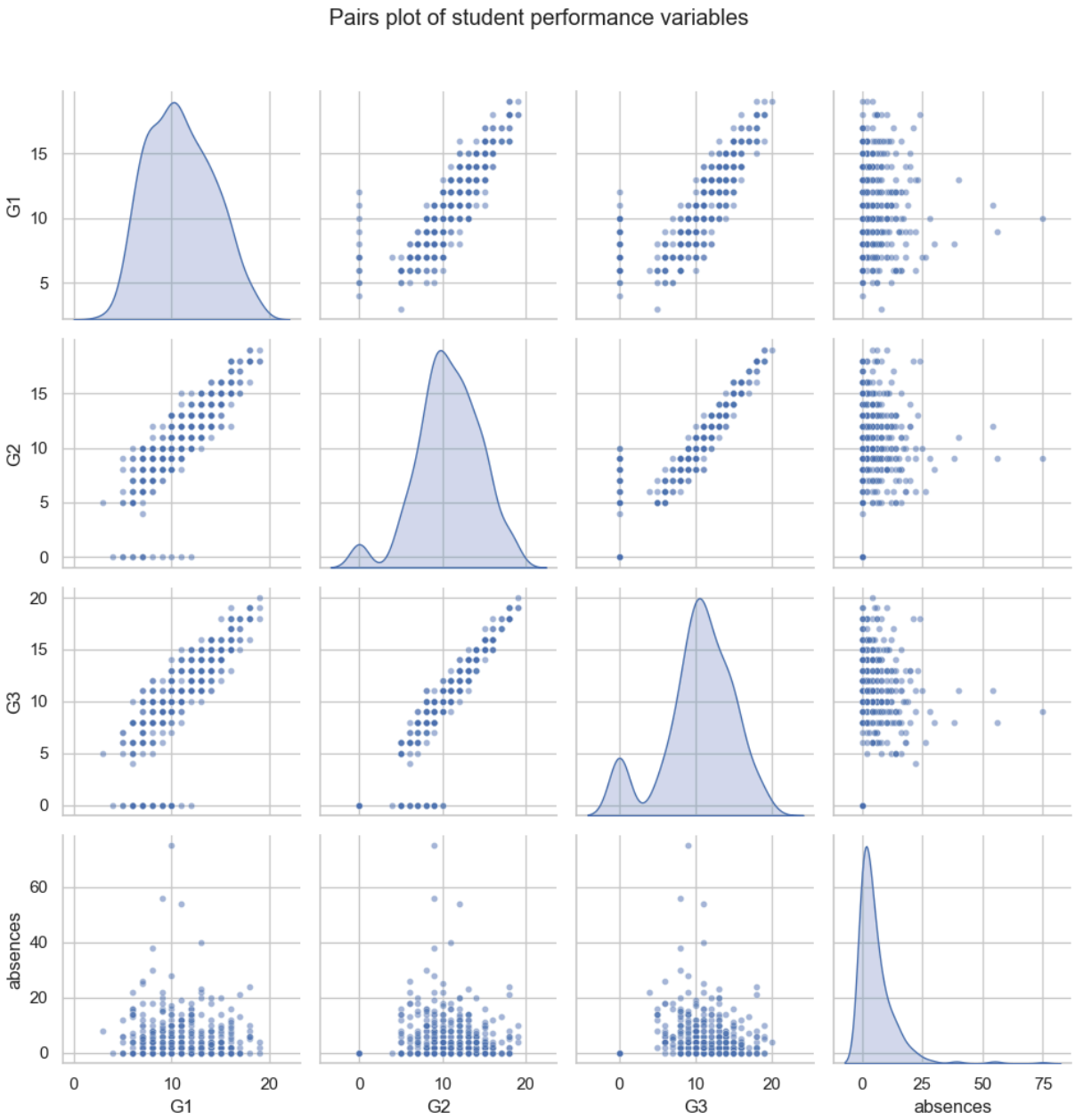
[figure] m05_07_facet_grid_sex_school.png

sex	school	G1	G3
F	GP	10.55	9.97
	MS	10.92	9.92
M	GP	11.38	11.07
	MS	10.38	9.76

==========

8. Pairs plot

==========



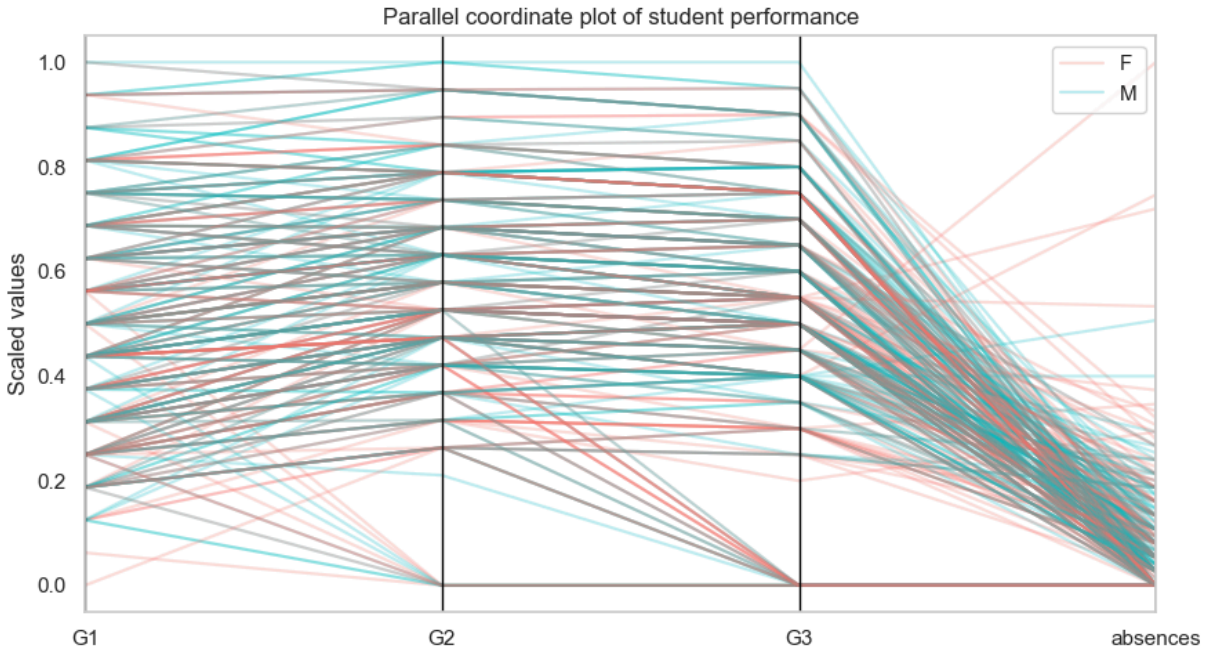
[figure] m05_08_pairs.png

	G1	G2	G3	absences
G1	1.000	0.852	0.801	-0.031
G2	0.852	1.000	0.905	-0.032
G3	0.801	0.905	1.000	0.034
absences	-0.031	-0.032	0.034	1.000

==========

9. Parallel coordinates (coloured by sex)

==========



[figure] m05_09_parallel_coordinates.png

7. One-, Two- and N-Dimensional Statistical Profile

Skewness/kurtosis, contingency tables with chi-square, grouped statistics, Pearson vs Spearman.

In [8]:

```
# ---------------------------------------------------------------------
# 7. One-, Two- and N-Dimensional Statistical Profile
# ---------------------------------------------------------------------

# Re-import for standalone cell execution
import numpy as np
import pandas as pd
import seaborn as sns
# scipy.stats for chi-squared test of independence
from scipy import stats

# Create a shortcut to matplotlib.pyplot
plt = U.plt

# Title for this section
TITLE = "AST07 - 1D, 2D and N-D Analysis"

def run():
    # Load raw data for some tasks (lab results like sex F=208/M=185 use NA-droppin
    data = U.load_student_raw()
    # Also load cleaned data for tasks that require no NaN
    clean = U.load_student(clean=True)

    # 2-3. Comprehensive summary statistics for the 'age' column
    U.section("2-3. Summary statistics table for 'age' (incl. skewness, kurtosis)")
    x = data["age"]
    # Calculate all summary statistics including e1071-compatible skewness and kurt
```

```
    summ = pd.DataFrame({"Statistic": ["Mean", "Median", "Minimum", "Maximum", "Ran
                                       "Standard Deviation", "Interquartile Range",
                         "Value": [x.mean(), x.median(), x.min(), x.max(), x.max()
                                   x.std(), x.quantile(.75) - x.quantile(.25),
                                   U.skewness_e1071(x), U.kurtosis_e1071(x)]})
    print(summ.to_string(index=False))
    U.save_table(summ, "age_summary_statistics", index=False)

    # 4. Frequency and relative frequency tables with pie and bar chart
    U.section("4. Frequency / relative-frequency tables, pie and bar")
    freq = data["sex"].value_counts(dropna=True).sort_index()
    rel = freq / freq.sum()
    print(pd.DataFrame({"freq": freq, "rel_freq": rel.round(4), "percent": (100 * r
    # Side-by-side pie chart and bar chart
    fig, axes = plt.subplots(1, 2, figsize=(9, 4))
    axes[0].pie(freq, labels=freq.index, autopct="%1.1f%%", colors=["#ff6666", "#66
    axes[0].set_title("Pie chart - sex")
    axes[1].bar(freq.index, freq.values, color="skyblue", edgecolor="black")
    axes[1].set(title="Bar plot - sex", ylabel="Frequency")
    U.save_fig("04_pie_bar_sex", fig)

    # 5. Binned frequency table, histogram, and boxplot for age
    U.section("5. Binned frequency table, histogram, boxplot (age)")
    # Create age bins for frequency table
    bins = np.arange(data["age"].min() - 1, data["age"].max() + 1)
    binned = pd.cut(data["age"], bins=bins).value_counts().sort_index()
    print(binned)
    # Side-by-side histogram and boxplot
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    axes[0].hist(data["age"], bins=np.arange(15, 23), color="lightblue", edgecolor=
    axes[0].set(title="Histogram of student age", xlabel="Age", ylabel="Frequency")
    sns.boxplot(y=data["age"], color="lightgreen", ax=axes[1])
    axes[1].set_title("Box plot of student age")
    U.save_fig("05_hist_box_age", fig)

    # 6-7. Contingency table with row/column/overall percentages
    U.section("6-7. Contingency table (sex x activities) with row/col/overall %")
    ct = pd.crosstab(data["sex"], data["activities"])
    print(ct)
    # Row percentages (proportion within each sex)
    print("Row %:\n", (100 * pd.crosstab(data["sex"], data["activities"], normalize
    # Column percentages (proportion within each activity group)
    print("Column %:\n", (100 * pd.crosstab(data["sex"], data["activities"], normal
    # Overall percentages
    print("Overall %:\n", (100 * pd.crosstab(data["sex"], data["activities"], norma
    # Chi-squared test of independence between sex and activities
    chi2, p, dof, _ = stats.chi2_contingency(ct)
    print(f"Chi-square test of independence: chi2={chi2:.3f}, dof={dof}, p={p:.4f}"
    U.save_table(ct, "contingency_sex_activities")
    # Stacked and grouped bar charts for the contingency table
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    ct.T.plot(kind="bar", stacked=True, ax=axes[0], rot=0)
    axes[0].set(title="Stacked: sex by activities", xlabel="activities", ylabel="Fr
    ct.T.plot(kind="bar", ax=axes[1], rot=0)
    axes[1].set(title="Grouped: sex by activities", xlabel="activities", ylabel="Fr
    U.save_fig("07_stacked_grouped_bars", fig)
```

```python
# NOTE: the R lab titled these plots "by School" although the table is sex x ac

# 8. Aggregate statistics: mean, median, std of age grouped by sex
U.section("8. aggregate(): mean / median / sd of age by sex")
agg = data.groupby("sex")["age"].agg(["mean", "median", "std"])
print(agg.round(4))
U.save_table(agg.round(4), "age_by_sex_aggregate")

# 9. Side-by-side box plot and violin plot of age by sex
U.section("9. Side-by-side box plots and violin plot (age by sex)")
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
sns.boxplot(data=data, x="sex", y="age", hue="sex", legend=False, ax=axes[0])
axes[0].set_title("Age by sex (box)")
sns.violinplot(data=data, x="sex", y="age", hue="sex", legend=False, cut=0, ax=
axes[1].set_title("Age by sex (violin)")
U.save_fig("09_box_violin_age_sex", fig)

# 10. Pearson and Spearman correlation between age and G1, with regression line
U.section("10. Pearson & Spearman correlation (age vs G1) + regression line")
# Pearson measures linear correlation
pr = data["age"].corr(data["G1"], method="pearson")
# Spearman measures monotonic (rank-based) correlation
sr = data["age"].corr(data["G1"], method="spearman")
print(f"Pearson={pr:.7f}  Spearman={sr:.7f}")
fig, ax = plt.subplots(figsize=(6, 4.5))
sns.regplot(data=data, x="age", y="G1", line_kws={"color": "black"}, ax=ax)
ax.set_title(f"Age vs G1 (Pearson={pr:.3f}, Spearman={sr:.3f})")
U.save_fig("10_scatter_age_G1")

# 11. Multi-group analysis: mean/SD of G1 by sex and school
U.section("11. Mean/SD of G1 by sex x school; faceted box plot; grouped bars wi
comb = data.groupby(["sex", "school"], dropna=False)["G1"].agg(Mean="mean", SD=
print(comb.round(4))
U.save_table(comb.round(4), "G1_by_sex_school", index=False)

# Faceted box plot: separate panels by school
g = sns.catplot(data=clean, x="sex", y="G1", col="school", kind="box", height=4
g.figure.suptitle("G1 by sex and school", y=1.03)
U.save_fig("11a_faceted_box", g.figure)

# Grouped bar chart with error bars (mean ± SD)
comb2 = clean.groupby(["sex", "school"])["G1"].agg(Mean="mean", SD="std").reset
fig, ax = plt.subplots(figsize=(6.5, 4.5))
w = 0.38  # bar width
for i, (sch, sub) in enumerate(comb2.groupby("school")):
    xs = np.arange(len(sub)) + (i - 0.5) * w
    ax.bar(xs, sub["Mean"], w, yerr=sub["SD"], capsize=4, label=sch)
ax.set_xticks(range(2), sorted(clean["sex"].unique()))
ax.set(title="Mean G1 (+/- SD) by sex and school", ylabel="Mean G1")
ax.legend(title="School", loc="upper left", bbox_to_anchor=(1.01, 1))
U.save_fig("11b_grouped_bar_errorbars", fig)

# 12. Scatter plot coloured/shaped by sex + per-group correlation
U.section("12. Scatter coloured/shaped by sex + per-group correlation")
fig, ax = plt.subplots(figsize=(7, 5))
sns.scatterplot(data=clean, x="age", y="G1", hue="sex", style="sex", s=60, ax=a
```

```
    ax.set_title("Age vs G1 by sex")
    U.save_fig("12_scatter_age_G1_by_sex")
    # Calculate and print correlation within each sex group
    for s, sub in data.dropna(subset=["sex"]).groupby("sex"):
        print(f"sex={s}: corr(age, G1) = {sub['age'].corr(sub['G1']):.4f}")

    # 13. Full correlation matrix heatmap for selected columns
    U.section("13. Full correlation matrix heatmap")
    cols = ["age", "studytime", "absences", "G1", "G2", "G3"]
    cm = data[cols].corr()
    print(cm.round(3))
    fig, ax = plt.subplots(figsize=(6.5, 5.5))
    sns.heatmap(cm, annot=True, fmt=".2f", cmap="bwr", center=0, ax=ax)
    ax.set_title("Correlation matrix heatmap")
    U.save_fig("13_corr_heatmap")

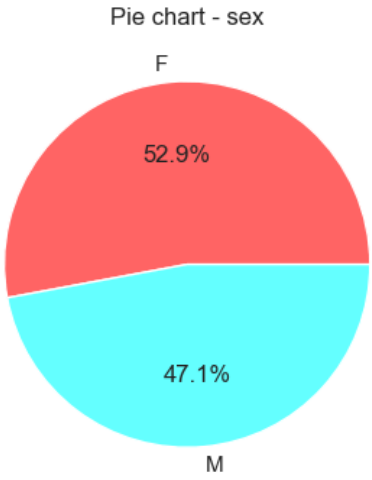
    # 14. Pairs plot (N-D scatterplot matrix)
    U.section("14. Pairs plot (N-D)")
    pp = sns.pairplot(clean[["age", "studytime", "G1", "G2", "G3"]], diag_kind="kde
                      plot_kws={"alpha": 0.5, "s": 15})
    U.save_fig("14_pairs", pp.figure)

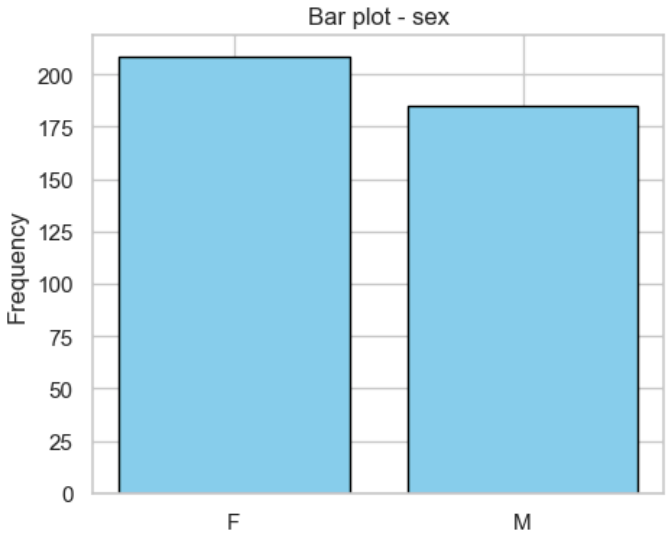
    return {"age_skewness_e1071": round(U.skewness_e1071(x), 4)}

# Set the module label and run
U.set_module("m07")
run()
```

```
================================================================================
2-3. Summary statistics table for 'age' (incl. skewness, kurtosis)
================================================================================
          Statistic     Value
               Mean 16.696203
             Median 17.000000
            Minimum 15.000000
            Maximum 22.000000
              Range  7.000000
           Variance  1.628285
 Standard Deviation  1.276043
Interquartile Range  2.000000
           Skewness  0.462735
  Kurtosis (excess) -0.031446
[table]  m07_age_summary_statistics.csv

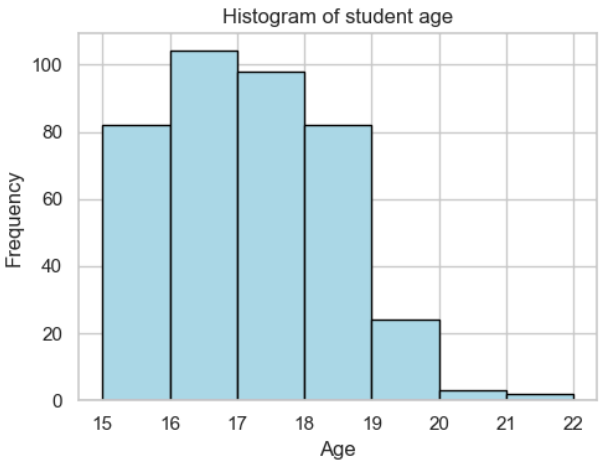
================================================================================
4. Frequency / relative-frequency tables, pie and bar
================================================================================
     freq  rel_freq  percent
sex
F     208    0.5293    52.93
M     185    0.4707    47.07
```





[figure] m07_04_pie_bar_sex.png

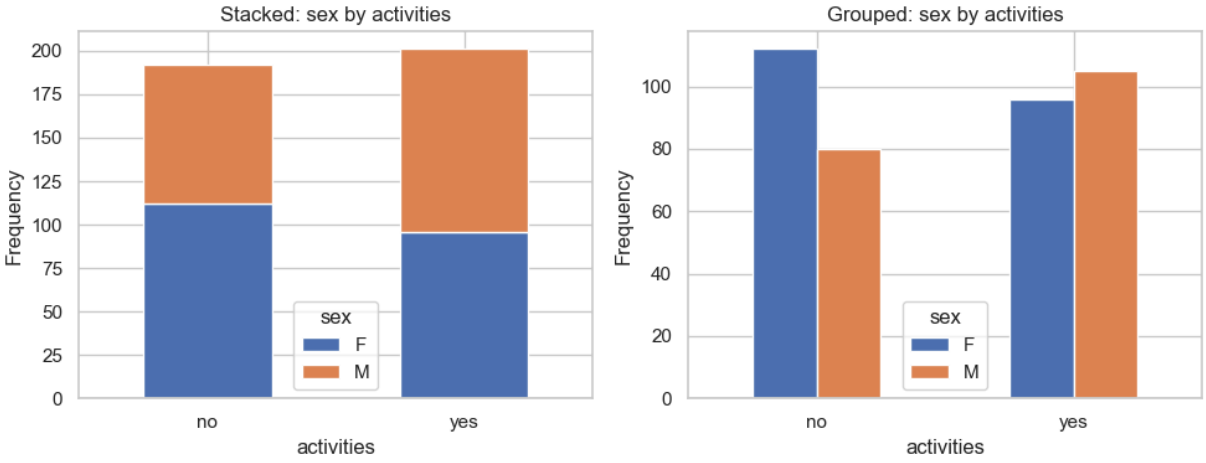
```
================================================================================
5. Binned frequency table, histogram, boxplot (age)
================================================================================
age
(14, 15]     82
(15, 16]    104
(16, 17]     98
(17, 18]     82
(18, 19]     24
(19, 20]      3
(20, 21]      1
(21, 22]      1
Name: count, dtype: int64
```





[figure] m07_05_hist_box_age.png

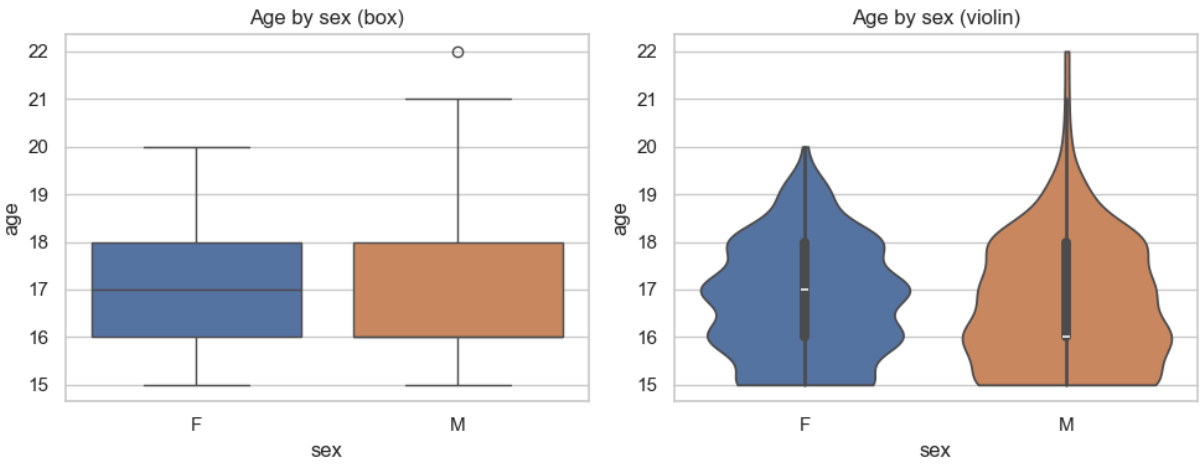
```
================================================================================
6-7. Contingency table (sex x activities) with row/col/overall %
================================================================================
activities   no  yes
sex
F           112   96
M            80  105
Row %:
 activities     no    yes
sex
F           53.85  46.15
M           43.24  56.76
Column %:
 activities     no    yes
sex
F           58.33  47.76
M           41.67  52.24
Overall %:
 activities     no    yes
sex
F           28.50  24.43
M           20.36  26.72
Chi-square test of independence: chi2=3.991, dof=1, p=0.0457
[table]  m07_contingency_sex_activities.csv
```



[figure] m07_07_stacked_grouped_bars.png

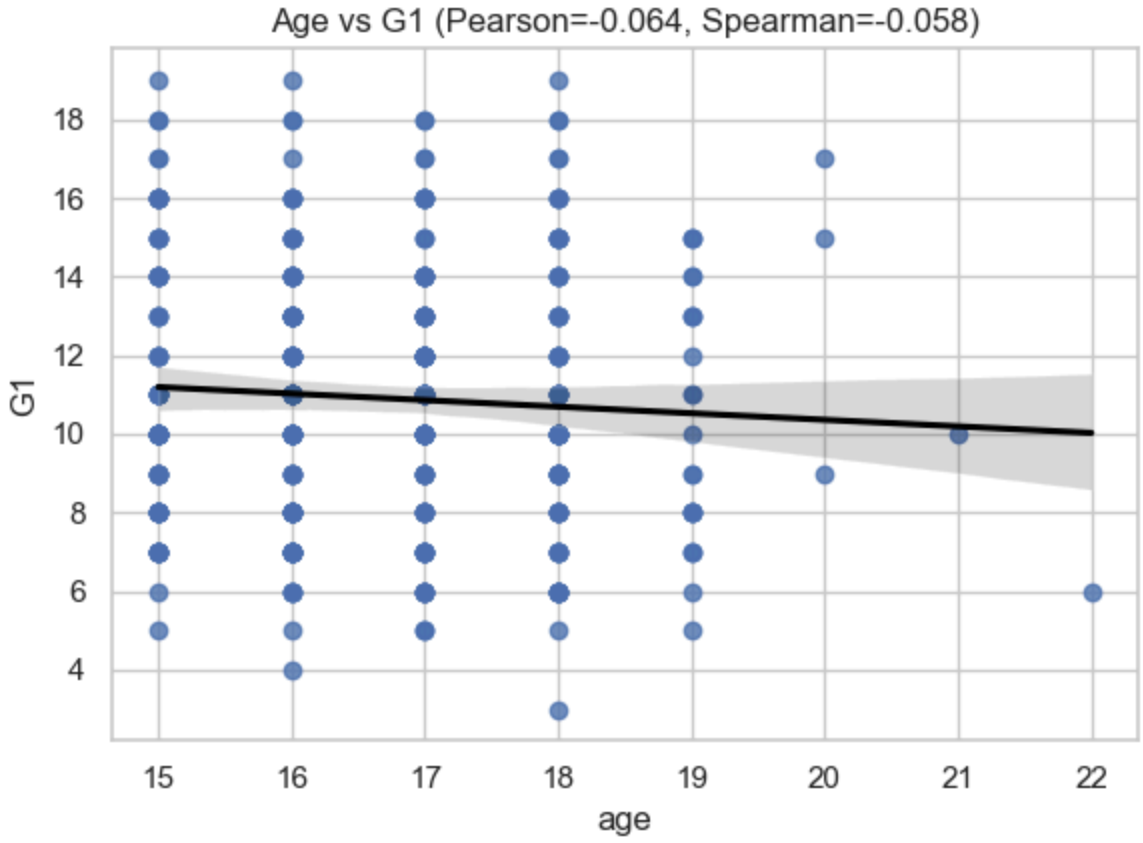
```
================================================================================
8. aggregate(): mean / median / sd of age by sex
================================================================================
        mean  median     std
sex
F    16.7308    17.0  1.2017
M    16.6541    16.0  1.3591
[table]  m07_age_by_sex_aggregate.csv

================================================================================
9. Side-by-side box plots and violin plot (age by sex)
================================================================================
```



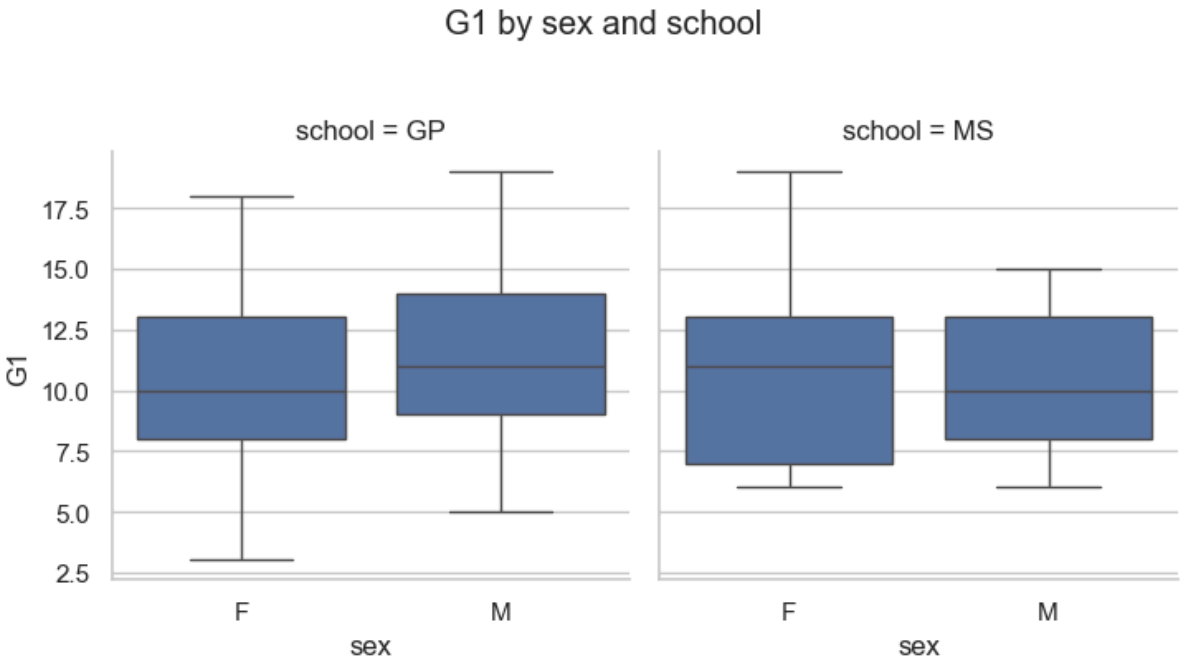
[figure] m07_09_box_violin_age_sex.png

==========
10. Pearson & Spearman correlation (age vs G1) + regression line
==========
Pearson=-0.0640815 Spearman=-0.0576300

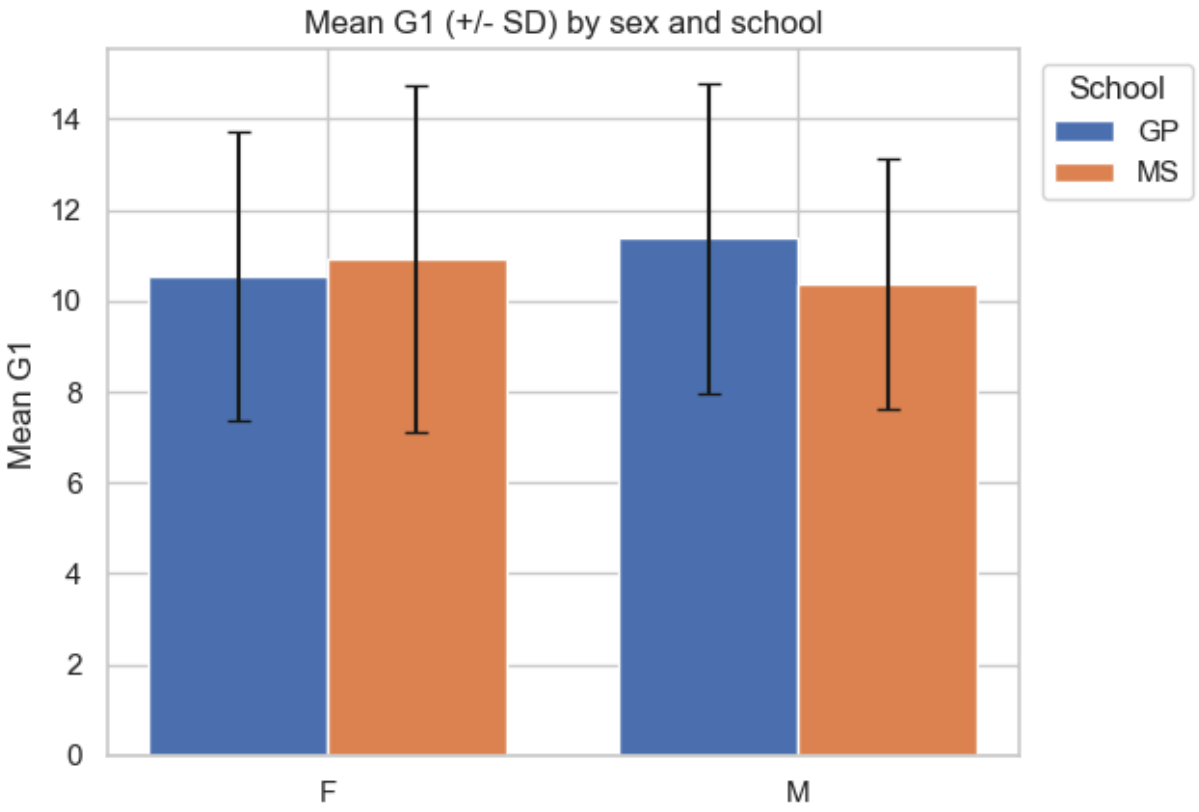


[figure] m07_10_scatter_age_G1.png

```
================================================================================
11. Mean/SD of G1 by sex x school; faceted box plot; grouped bars with error bars
================================================================================
   sex school     Mean      SD
0    F     GP  10.5792  3.1541
1    F     MS  10.9200  3.8179
2    M     GP  11.3780  3.4147
3    M     MS  10.3810  2.7473
4  NaN     GP   8.0000  7.0711
[table]  m07_G1_by_sex_school.csv
```

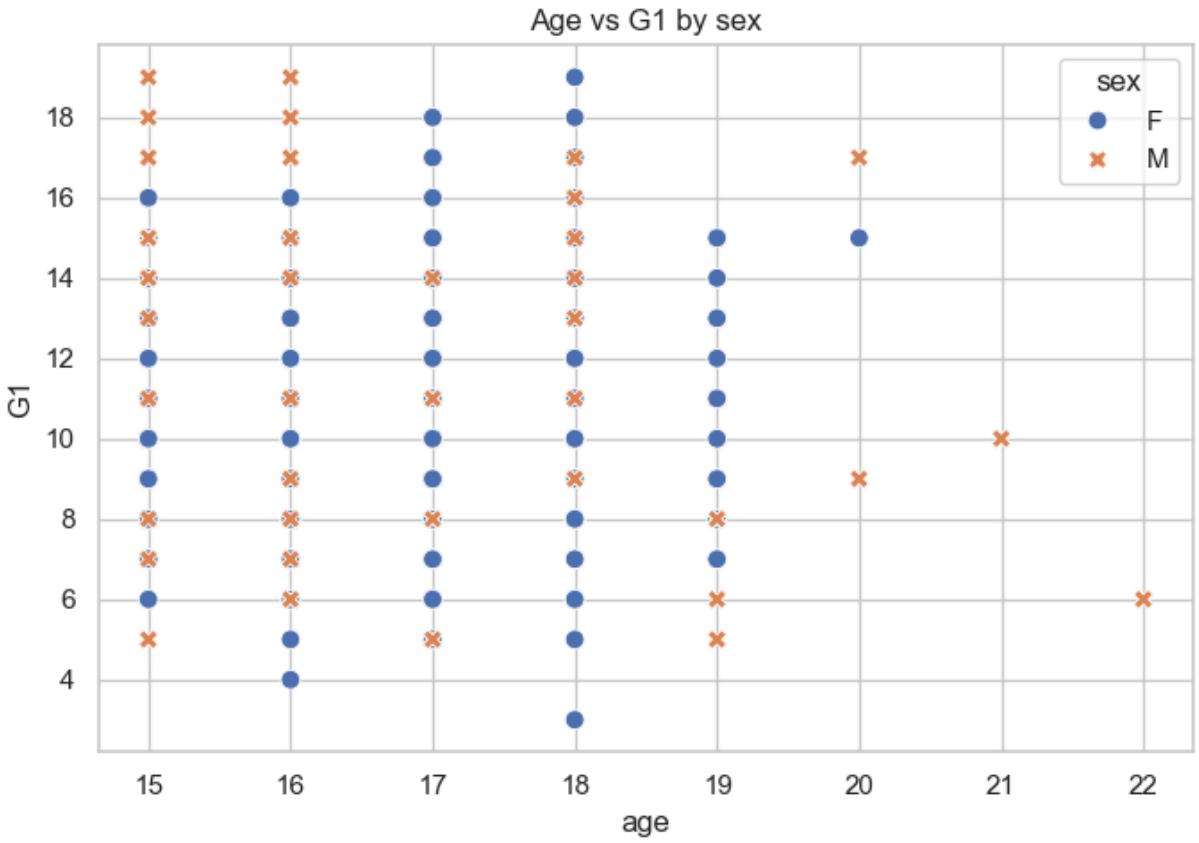


[figure] m07_11a_faceted_box.png



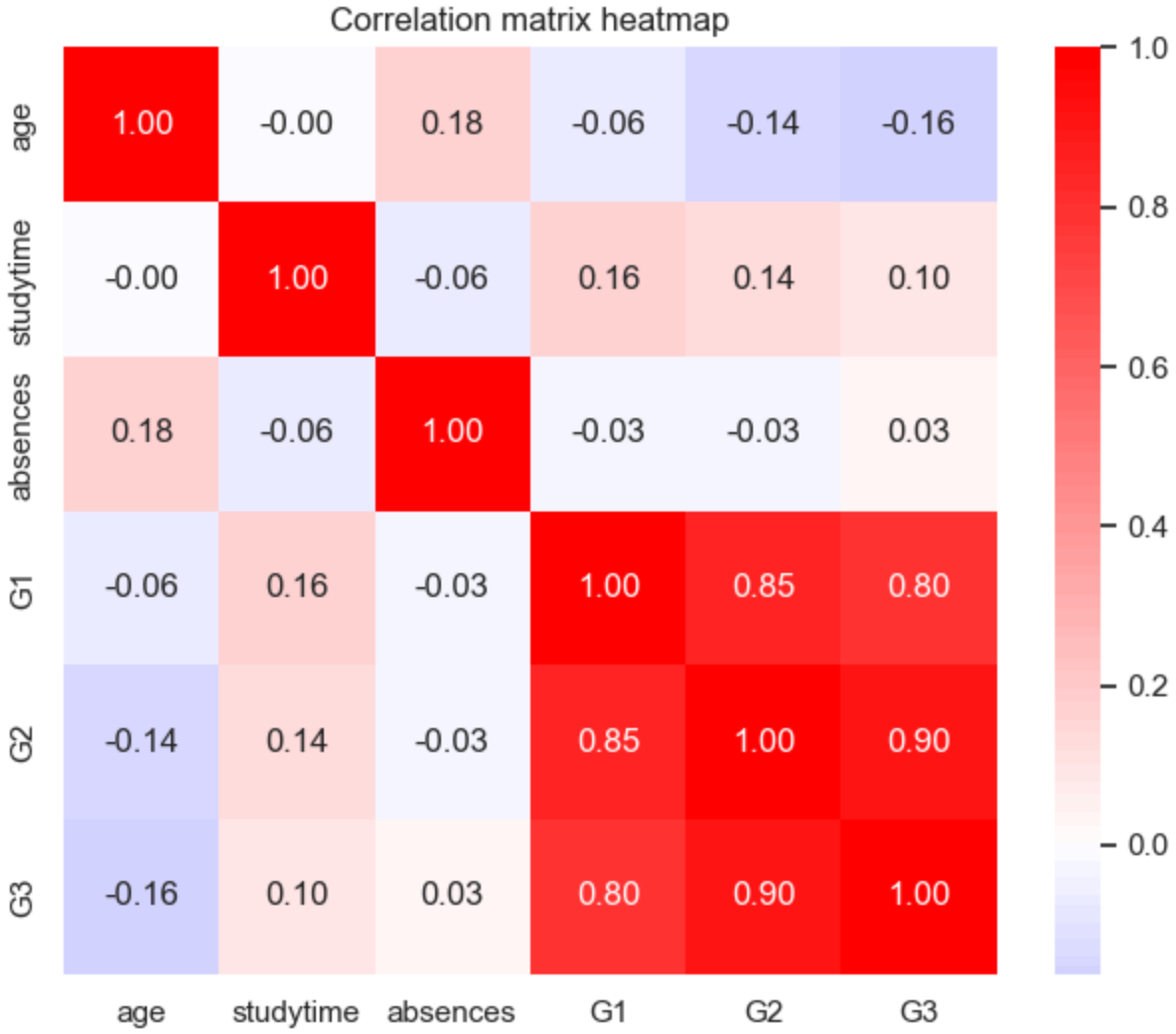
[figure] m07_11b_grouped_bar_errorbars.png

```
================================================================================
12. Scatter coloured/shaped by sex + per-group correlation
================================================================================
```



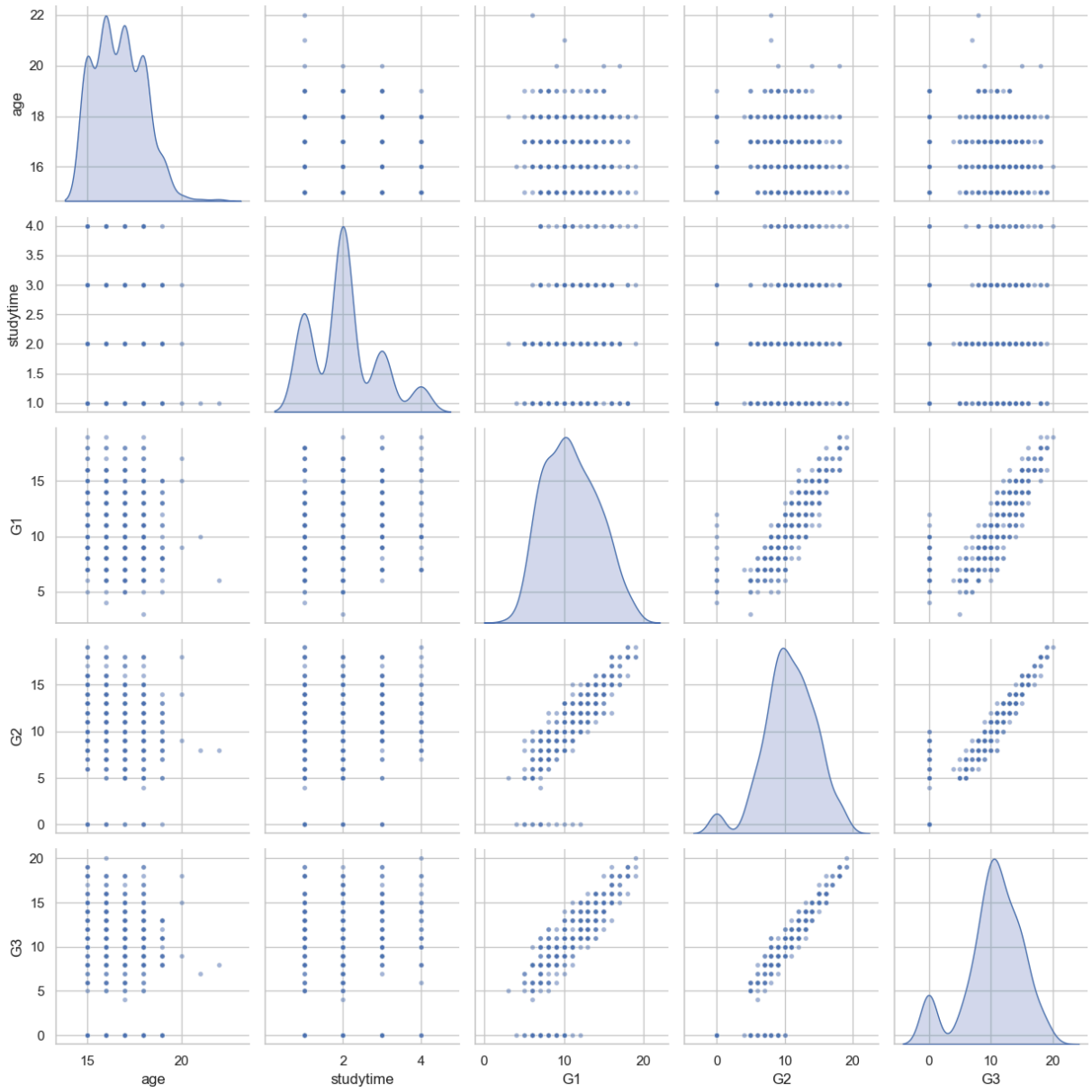
```
[figure] m07_12_scatter_age_G1_by_sex.png
sex=F: corr(age, G1) = 0.1128
sex=M: corr(age, G1) = -0.2159

================================================================================
13. Full correlation matrix heatmap
================================================================================
              age  studytime  absences     G1     G2     G3
age         1.000     -0.004     0.175 -0.064 -0.143 -0.162
studytime  -0.004      1.000    -0.063  0.161  0.136  0.098
absences    0.175     -0.063     1.000 -0.031 -0.032  0.034
G1         -0.064      0.161    -0.031  1.000  0.852  0.801
G2         -0.143      0.136    -0.032  0.852  1.000  0.905
G3         -0.162      0.098     0.034  0.801  0.905  1.000
```




```
[figure] m07_13_corr_heatmap.png

================================================================================
14. Pairs plot (N-D)
================================================================================
```



[figure] m07_14_pairs.png

Out[8]:
```
{'age_skewness_e1071': 0.4627}
```

8. Statistical Hypothesis Testing & Effect Sizes

Welch t / Mann-Whitney / ANOVA / Kruskal / chi-square with Cohen d, eta-squared, Cramer V and Holm correction.

In [9]:
```
# ----------------------------------------------------------------------
# 8. Statistical Hypothesis Testing & Effect Sizes
# ----------------------------------------------------------------------

# Re-import for standalone cell execution
import numpy as np
import pandas as pd
import seaborn as sns
# scipy.stats for t-tests, Mann-Whitney, ANOVA, Kruskal-Wallis, chi-squared
```

```python
from scipy import stats

# Create a shortcut to matplotlib.pyplot
plt = U.plt

# Title for this section
TITLE = "Capstone - Hypothesis Tests & Key Findings (extension)"

def run():
    # Load cleaned student data
    d = U.load_student(clean=True)
    # Collect key findings as we go
    findings = []

    # ------------------------------------------------------------
    # Spearman correlations: which numeric features relate most to G3?
    # ------------------------------------------------------------
    U.section("Which numeric features correlate most with G3? (Spearman, ordinal-sa
    # Spearman correlation is rank-based, so it works for ordinal variables too
    sp = d[U.NUMERIC_COLS].corr(method="spearman")["G3"].drop("G3").sort_values(key
    print(sp.round(3).to_string())
    U.save_table(sp.round(4).rename("spearman_with_G3"), "spearman_with_G3")

    # Horizontal bar chart: green for positive, red for negative correlations
    fig, ax = plt.subplots(figsize=(6, 5))
    sp.sort_values().plot.barh(ax=ax, color=["indianred" if v < 0 else "seagreen" f
    ax.set(title="Spearman correlation with G3", xlabel="rho")
    U.save_fig("spearman_with_G3", fig)
    findings.append(f"Strongest correlates of G3: {', '.join(f'{k} ({v:+.2f})' for

    # ------------------------------------------------------------
    # Hypothesis tests for categorical predictors vs G3
    # ------------------------------------------------------------
    U.section("Hypothesis tests")
    rows = []

    # Two-group tests: Welch t-test and Mann-Whitney U for binary categorical varia
    for col in ["sex", "school", "higher", "internet", "romantic", "schoolsup", "ad
        a, b = [g["G3"].values for _, g in d.groupby(col)]
        # Welch's t-test: doesn't assume equal variances
        t, p = stats.ttest_ind(a, b, equal_var=False)
        # Mann-Whitney U: non-parametric alternative
        u, pu = stats.mannwhitneyu(a, b)
        rows.append({"test": f"G3 ~ {col} (Welch t / Mann-Whitney)", "stat": t, "p_

    # Multi-group tests: one-way ANOVA and Kruskal-Wallis for multi-level categoric
    for col in ["Mjob", "Fjob", "reason", "guardian"]:
        # ANOVA: tests if at least one group mean differs
        f, p = stats.f_oneway(*[g["G3"].values for _, g in d.groupby(col)])
        # Kruskal-Wallis: non-parametric alternative to ANOVA
        h, ph = stats.kruskal(*[g["G3"].values for _, g in d.groupby(col)])
        rows.append({"test": f"G3 ~ {col} (ANOVA / Kruskal)", "stat": f, "p_value":

    # Chi-squared tests of independence for categorical x categorical
    for a, b in [("sex", "activities"), ("higher", "school"), ("romantic", "higher"
        chi, p, *_ = stats.chi2_contingency(pd.crosstab(d[a], d[b]))
```

```
        rows.append({"test": f"chi2 {a} x {b}", "stat": chi, "p_value": p, "p_mannw

    # ------------------------------------------------------------
    # Effect sizes: Cohen's d (two groups), eta^2 (k groups), Cramer's V (chi-squar
    # ------------------------------------------------------------
    effects = []

    # Cohen's d for two-group comparisons
    for col in ["sex", "school", "higher", "internet", "romantic", "schoolsup", "ad
        a, b = [g["G3"].values for _, g in d.groupby(col)]
        # Pooled standard deviation
        sp_val = np.sqrt(((len(a) - 1) * a.var(ddof=1) + (len(b) - 1) * b.var(ddof=
        # Cohen's d = |mean difference| / pooled SD
        effects.append(abs(a.mean() - b.mean()) / sp_val)

    # Eta-squared for multi-group comparisons
    for col in ["Mjob", "Fjob", "reason", "guardian"]:
        grp = [g["G3"].values for _, g in d.groupby(col)]
        # Between-group sum of squares
        ss_b = sum(len(g) * (g.mean() - d["G3"].mean()) ** 2 for g in grp)
        # Total sum of squares
        effects.append(ss_b / ((d["G3"] - d["G3"].mean()) ** 2).sum())

    # Cramer's V for chi-squared tests
    for a_, b_ in [("sex", "activities"), ("higher", "school"), ("romantic", "highe
        ctab = pd.crosstab(d[a_], d[b_])
        effects.append(np.sqrt(stats.chi2_contingency(ctab, correction=False)[0] /

    # Compile test results with effect sizes and multiple comparison correction
    tests = pd.DataFrame(rows).round(4)
    tests["effect_size"] = np.round(effects, 3)

    # Holm-Bonferroni correction for multiple comparisons
    from statsmodels.stats.multitest import multipletests
    tests["p_holm"] = multipletests(tests["p_value"], method="holm")[1].round(4)
    tests["significant_holm_5pct"] = tests["p_holm"] < 0.05
    tests["significant_5pct"] = tests["p_value"] < 0.05
    print(tests.to_string(index=False))
    U.save_table(tests, "hypothesis_tests", index=False)

    # Summarise significance results
    sig = tests.loc[tests["significant_5pct"], "test"].tolist()
    sig_h = tests.loc[tests["significant_holm_5pct"], "test"].tolist()
    findings.append(f"{len(sig)}/{len(tests)} tests significant at 5% uncorrected;
                    + (", ".join(sig_h) if sig_h else "none"))

    # ------------------------------------------------------------
    # Failure / zero-grade analysis
    # ------------------------------------------------------------
    U.section("Failure / zero-grade analysis")
    zero = d[d["G3"] == 0]
    print(f"{len(zero)} students ({100 * len(zero) / len(d):.1f}%) have G3 = 0; "
          f"mean absences among them = {zero['absences'].mean():.2f} vs {d.loc[d['G

    # Compare the proportion of zero absences between G3=0 and G3>0 groups
    share0 = (zero["absences"] == 0).mean()
```

```
    other0 = (d.loc[d["G3"] > 0, "absences"] == 0).mean()
    findings.append(f"{len(zero)} students have G3=0 and {100 * share0:.0f}% of the
                    f"(vs {100 * other0:.0f}% of other students) - zeros look like
                    f"not real scores (hypothesis; consider excluding them in model

    # G3 distribution by number of past failures
    by_fail = d.groupby("failures")["G3"].agg(["count", "mean"]).round(2)
    print(by_fail)

    # Box plots: G3 by failures and by study time
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    sns.boxplot(data=d, x="failures", y="G3", ax=axes[0], color="lightsteelblue")
    axes[0].set_title("G3 by past failures")
    sns.boxplot(data=d, x="studytime", y="G3", ax=axes[1], color="lightgreen")
    axes[1].set_title("G3 by weekly study time")
    U.save_fig("G3_by_failures_studytime", fig)

    # ------------------------------------------------------------
    # Grade progression G1 -> G2 -> G3
    # ------------------------------------------------------------
    U.section("Grade progression G1 -> G2 -> G3")
    # How much do grades change from G1 to G3?
    drop = (d["G3"] - d["G1"]).describe().round(2)
    print(drop)
    findings.append(f"Mean grade moves from {d['G1'].mean():.2f} (G1) to {d['G3'].m
                    f"corr(G2,G3)={d['G2'].corr(d['G3']):.2f}, so G2 is almost a pr

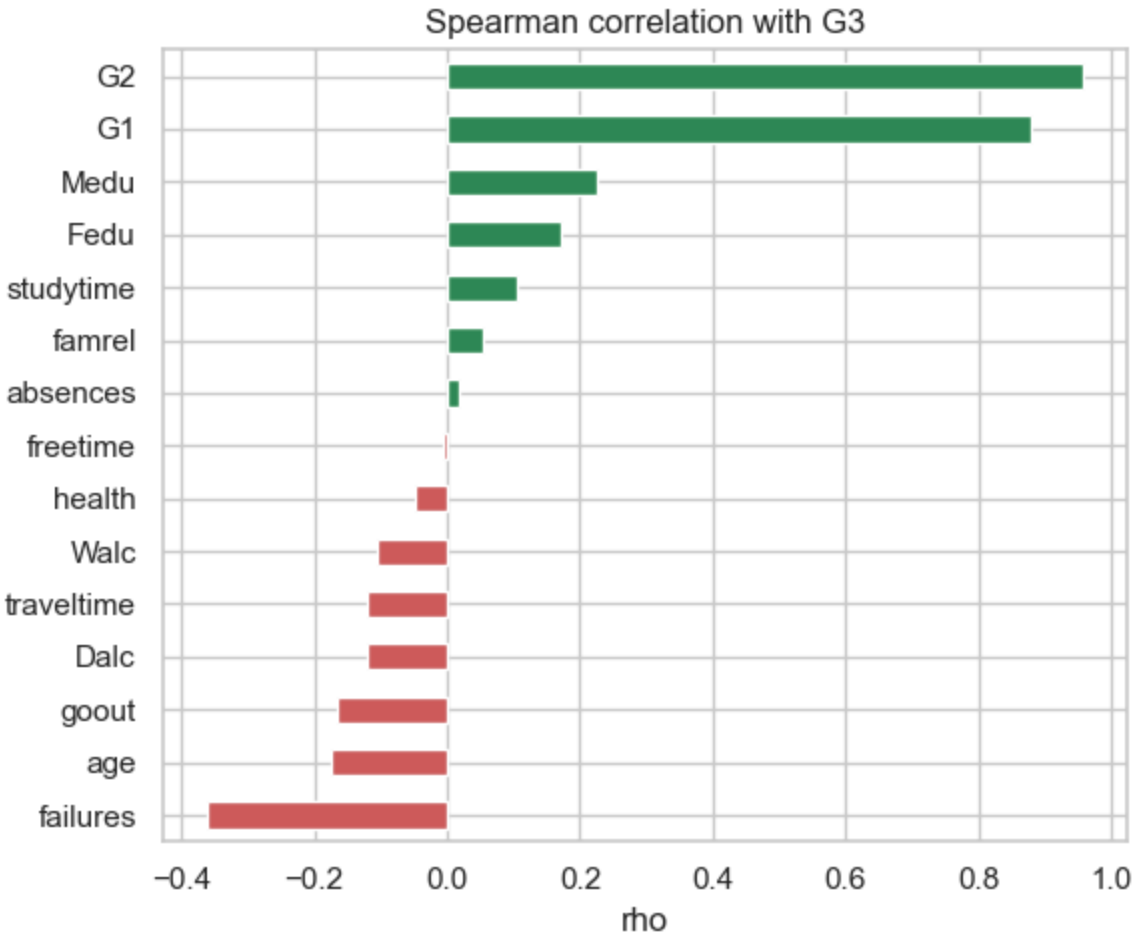
    # Print all key findings
    print("\nKEY FINDINGS")
    for f in findings:
        print(" -", f)

    # Save key findings to a markdown file
    (U.RESULTS / "key_findings.md").write_text("# Key findings (auto-generated)\n\n

    return {"findings": findings}

# Set the module label and run
U.set_module("m10")
run()
```

```
================================================================================
Which numeric features correlate most with G3? (Spearman, ordinal-safe)
================================================================================
G2            0.957
G1            0.878
failures     -0.361
Medu          0.225
age          -0.173
Fedu          0.170
goout        -0.166
Dalc         -0.121
traveltime   -0.121
studytime     0.105
Walc         -0.104
famrel        0.055
health       -0.048
absences      0.018
freetime     -0.005
[table]  m10_spearman_with_G3.csv
```

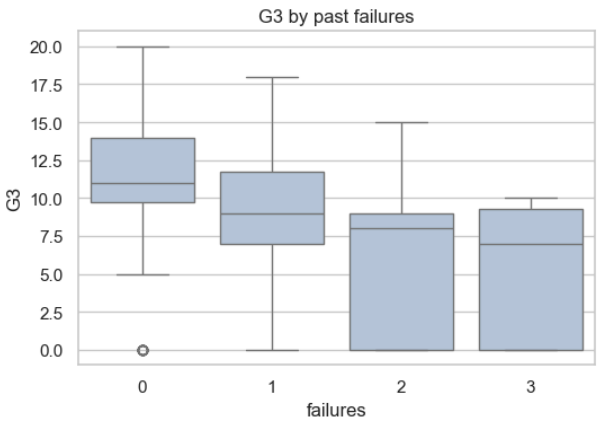


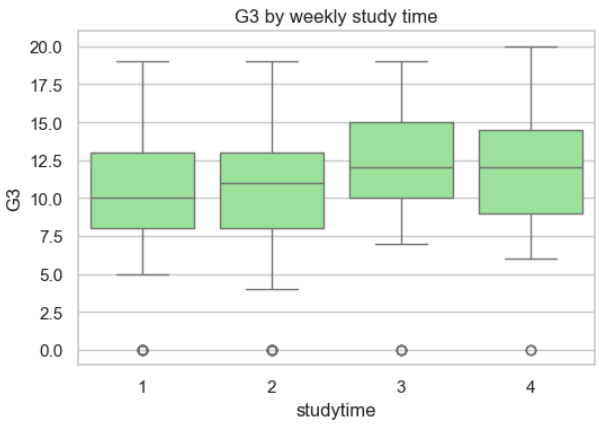

```
[figure] m10_spearman_with_G3.png

================================================================================
Hypothesis tests
================================================================================
```

```
                                    test    stat  p_value  p_mannwhitney  effect_size
p_holm  significant_holm_5pct  significant_5pct
      G3 ~ sex (Welch t / Mann-Whitney) -2.0858   0.0376         0.0393        0.210
0.3660                  False              True
   G3 ~ school (Welch t / Mann-Whitney)  0.9555   0.3431         0.2002        0.140
0.8204                  False             False
   G3 ~ higher (Welch t / Mann-Whitney) -3.4474   0.0024         0.0006        0.844
0.0336                   True              True
 G3 ~ internet (Welch t / Mann-Whitney) -1.9894   0.0496         0.0324        0.265
0.3660                  False              True
 G3 ~ romantic (Welch t / Mann-Whitney)  2.5122   0.0127         0.0695        0.277
0.1524                  False              True
G3 ~ schoolsup (Welch t / Mann-Whitney)  2.3119   0.0230         0.0037        0.244
0.2530                  False              True
  G3 ~ address (Welch t / Mann-Whitney) -2.1101   0.0366         0.0178        0.255
0.3660                  False              True
            G3 ~ Mjob (ANOVA / Kruskal)  3.7545   0.0052         0.0029        0.037
0.0676                  False              True
            G3 ~ Fjob (ANOVA / Kruskal)  1.3029   0.2683         0.1794        0.013
0.8204                  False             False
          G3 ~ reason (ANOVA / Kruskal)  2.0800   0.1023         0.1818        0.016
0.5115                  False             False
        G3 ~ guardian (ANOVA / Kruskal)  1.5905   0.2051         0.1486        0.008
0.8204                  False             False
                 chi2 sex x activities  4.3670   0.0366            NaN        0.110
0.3660                  False              True
                  chi2 higher x school  0.0149   0.9027            NaN        0.024
0.9027                  False             False
               chi2 romantic x higher  3.4476   0.0633            NaN        0.106
0.3798                  False             False
[table]  m10_hypothesis_tests.csv

================================================================================
Failure / zero-grade analysis
================================================================================
38 students (9.6%) have G3 = 0; mean absences among them = 0.00 vs 6.32 others
          count   mean
failures
0           312  11.25
1            50   8.12
2            17   6.24
3            16   5.69
```





```
[figure] m10_G3_by_failures_studytime.png

================================================================================
Grade progression G1 -> G2 -> G3
================================================================================
count    395.00
mean      -0.49
std        2.76
min      -12.00
25%       -1.00
50%        0.00
75%        1.00
max        4.00
dtype: float64

KEY FINDINGS
 - Strongest correlates of G3: G2 (+0.96), G1 (+0.88), failures (-0.36), Medu (+0.2
3)
 - 8/14 tests significant at 5% uncorrected; after Holm correction: G3 ~ higher (Wel
ch t / Mann-Whitney)
 - 38 students have G3=0 and 100% of them have exactly 0 absences (vs 22% of other s
tudents) - zeros look like missing-exam/dropout records, not real scores (hypothesi
s; consider excluding them in modelling).
 - Mean grade moves from 10.91 (G1) to 10.42 (G3); corr(G2,G3)=0.90, so G2 is almost
a proxy for G3.
```

Out[9]:
```
{'findings': ['Strongest correlates of G3: G2 (+0.96), G1 (+0.88), failures (-0.3
6), Medu (+0.23)',
  '8/14 tests significant at 5% uncorrected; after Holm correction: G3 ~ higher (W
elch t / Mann-Whitney)',
  '38 students have G3=0 and 100% of them have exactly 0 absences (vs 22% of other
students) - zeros look like missing-exam/dropout records, not real scores (hypothe
sis; consider excluding them in modelling).',
  'Mean grade moves from 10.91 (G1) to 10.42 (G3); corr(G2,G3)=0.90, so G2 is almo
st a proxy for G3.']}
```

9. Dimensionality Reduction (PCA)

PCA on the student features: scree, loadings, biplot. All analysis uses student-mat.csv only.

In [10]:
```python
# ----------------------------------------------------------------------
# 9. Dimensionality Reduction (PCA)
# All PCA analysis uses student-mat.csv numeric features.
# No external R datasets (mtcars) are used.
# ----------------------------------------------------------------------

# Re-import for standalone cell execution
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA

# Create a shortcut to matplotlib.pyplot
plt = U.plt

# Title for this section
```

```python
TITLE = "AST09 - Principal Component Analysis"

def biplot(scores, loadings, labels, var_names, title, name, pcs=(0, 1), label_poin
    """Create an R-style biplot: observation scores as grey points,
    variable loadings as red arrows.

    Parameters:
        scores: PCA-transformed data (n_samples x n_components)
        loadings: component loadings (n_features x n_components)
        labels: point labels (e.g., student indices)
        var_names: variable names for the loading arrows
        title: plot title
        name: filename for saving
        pcs: tuple of PC indices to plot (default: PC1 vs PC2)
        label_points: whether to annotate each point with its label
    """
    i, j = pcs
    fig, ax = plt.subplots(figsize=(8, 7))

    # Plot observation scores as grey dots
    ax.scatter(scores[:, i], scores[:, j], s=12, color="gray")

    # Optionally label each point
    if label_points:
        for (x, y), lab in zip(scores[:, [i, j]], labels):
            ax.annotate(lab, (x, y), fontsize=7, color="dimgray", alpha=0.9)

    # Scale loadings to match the range of scores for visual clarity
    scale = np.abs(scores[:, [i, j]]).max() / np.abs(loadings[:, [i, j]]).max() * 0

    # Draw loading arrows (one per variable)
    for (lx, ly), v in zip(loadings[:, [i, j]] * scale, var_names):
        ax.annotate("", xy=(lx, ly), xytext=(0, 0), arrowprops=dict(arrowstyle="->"
        ax.text(lx * 1.08, ly * 1.08, v, color="red", fontsize=10, ha="center")

    # Add reference lines at origin
    ax.axhline(0, color="k", lw=0.5)
    ax.axvline(0, color="k", lw=0.5)

    # Set symmetric axis limits
    lim = 1.2 * max(np.abs(scores[:, [i, j]]).max(), np.abs(loadings[:, [i, j]] * s
    ax.set_xlim(-lim, lim)
    ax.set_ylim(-lim, lim)
    ax.set(title=title, xlabel=f"PC{i + 1}", ylabel=f"PC{j + 1}")
    U.save_fig(name, fig)

def _pca_block(df: pd.DataFrame, tag: str, label_points: bool):
    """Run PCA on a DataFrame and return results.

    Steps:
    1. Standardise with sample SD (ddof=1) to match R's scale()/prcomp()
    2. Fit PCA on all components
    3. Print and save the variance explained table
    4. Return PCA object, scores, loadings, table, and number of PCs for 80% varian
    """
    # Standardise using sample SD (ddof=1) to match R's prcomp() exactly
```

```
    X = U.zscore_df(df).values
    # Fit PCA with all possible components
    pca = PCA().fit(X)
    # Transform data to get scores (coordinates in PC space)
    scores = pca.transform(X)
    # Loadings: how much each original variable contributes to each PC
    load = pca.components_.T  # transpose so rows=variables, columns=PCs

    # Eigenvalues and proportion of variance explained
    ev = pca.explained_variance_
    prop = pca.explained_variance_ratio_
    cum = np.cumsum(prop)  # cumulative proportion

    # Build variance explained table
    table = pd.DataFrame({"PC": [f"PC{k + 1}" for k in range(len(prop))],
                          "Standard_Deviation": np.sqrt(ev), "Proportion_of_Varianc
                          "Cumulative_Proportion": cum})
    print(table.round(4).to_string(index=False))
    U.save_table(table.round(6), f"{tag}_variance_explained", index=False)

    # Find minimum number of PCs needed to reach 80% of total variance
    n80 = int(np.argmax(cum >= 0.80)) + 1
    print(f"Minimum PCs for >=80% variance: {n80}")

    # Save loadings as a table
    load_df = pd.DataFrame(load, index=df.columns, columns=table["PC"])
    U.save_table(load_df.round(4), f"{tag}_loadings")

    return pca, scores, load_df, table, n80

def run():
    # ------------------------------------------------------------------
    # PCA on student-mat.csv numeric features (16 numeric columns)
    # ------------------------------------------------------------------
    U.section("Q1: Student numeric features and summary")
    st = U.load_student(clean=True)
    # Select all numeric columns defined in the global column list
    stn = st[U.NUMERIC_COLS]
    print(f"Using {len(U.NUMERIC_COLS)} numeric features from student-mat.csv")
    print(stn.describe().round(3))

    U.section("Q2: Why standardise?")
    print("'absences' ranges ~0-75 while 'Medu'/'Fedu' range 0-4; unscaled PCA woul
          "dominated by high-variance columns like absences and G1/G2/G3.\n"
          "Standardising gives every variable mean 0 and sd 1 so PCA reflects corre
    print("variances before scaling:\n", stn.var().round(3).to_string())

    U.section("Q3-Q5: PCA equivalent, variance explained, #PCs for 80%")
    # Run PCA on all 16 numeric features
    pca_s, sc_s, load_s, tab_s, n80_s = _pca_block(stn, "student", False)

    U.section("Q6: Scree plot")
    ps = tab_s["Proportion_of_Variance"].values
    # Side-by-side: individual variance per PC and cumulative variance
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    axes[0].plot(range(1, len(ps) + 1), ps, "o-", color="blue")
```

```
    axes[0].set(title="Scree plot - student PCA", xlabel="Principal component", yla
    axes[1].bar(range(1, len(ps) + 1), np.cumsum(ps), color="lightblue", edgecolor=
    axes[1].axhline(0.8, color="red", ls="--")
    axes[1].set(title="Cumulative variance", xlabel="Principal component")
    U.save_fig("06_scree_student", fig)

    U.section("Q7: Loadings - strongest contributors to PC1 and PC2")
    print(load_s.round(4))
    for pc in ("PC1", "PC2"):
        print(f"\nTop contributors to {pc}:\n", load_s[pc].abs().sort_values(ascend

    U.section("Q8: Biplot")
    # Biplot: shows students as points and variable contributions as arrows
    biplot(sc_s, load_s.values, st.index.astype(str), list(stn.columns),
           "PCA biplot - student data", "08_biplot_student", label_points=False)
    print("PC1 is typically dominated by grade-related variables (G1, G2, G3).\n"
          "PC2 often captures lifestyle/social variables (goout, Walc, Dalc, freeti

    # ------------------------------------------------------------------
    # Extension: PCA on a selected subset of features
    # ------------------------------------------------------------------
    U.section("Extension: PCA on selected student features (6 key variables)")
    selected_cols = ["age", "studytime", "absences", "G1", "G2", "G3"]
    sub = st[selected_cols]
    print(sub.describe().round(3))

    # Run PCA on the 6-variable subset
    pca_sub, sc_sub, load_sub, tab_sub, n80_sub = _pca_block(sub, "student_subset",

    print(f"\nWith {len(selected_cols)} features, {n80_sub} PCs needed for 80% vari
    print("\nTop |loadings| PC1:", load_sub["PC1"].abs().sort_values(ascending=Fals
    print("Top |loadings| PC2:", load_sub["PC2"].abs().sort_values(ascending=False)

    # Scree plot for the subset
    fig, ax = plt.subplots(figsize=(7, 4))
    ps_sub = tab_sub["Proportion_of_Variance"].values
    ax.plot(range(1, len(ps_sub) + 1), ps_sub, "o-")
    ax.plot(range(1, len(ps_sub) + 1), np.cumsum(ps_sub), "s--", color="darkorange"
    ax.axhline(0.8, color="red", ls=":")
    ax.set(title="Scree plot - student selected features", xlabel="PC", ylabel="Pro
    ax.legend(["variance", "cumulative", "80%"])
    U.save_fig("student_subset_scree", fig)

    # Colour PCA scores by school for context
    fig, ax = plt.subplots(figsize=(7, 5.5))
    for sch, m in st.groupby("school").groups.items():
        idx = st.index.get_indexer(m)
        ax.scatter(sc_s[idx, 0], sc_s[idx, 1], s=14, alpha=0.6, label=sch)
    ax.set(title="Student PCA scores by school", xlabel="PC1", ylabel="PC2")
    ax.legend()
    U.save_fig("student_scores_by_school", fig)

    return {"student_pcs_for_80pct": n80_s, "student_subset_pcs_for_80pct": n80_sub

# Set the module label and run
```

```
U.set_module("m09")
run()
```

```
================================================================================
Q1: Student numeric features and summary
================================================================================
Using 16 numeric features from student-mat.csv
           age     Medu     Fedu  traveltime  studytime  failures   famrel  \
count  395.000  395.000  395.000     395.000    395.000   395.000  395.000
mean    16.696    2.749    2.522       1.448      2.035     0.334    3.944
std      1.276    1.095    1.088       0.698      0.839     0.744    0.897
min     15.000    0.000    0.000       1.000      1.000     0.000    1.000
25%     16.000    2.000    2.000       1.000      1.000     0.000    4.000
50%     17.000    3.000    2.000       1.000      2.000     0.000    4.000
75%     18.000    4.000    3.000       2.000      2.000     0.000    5.000
max     22.000    4.000    4.000       4.000      4.000     3.000    5.000

       freetime    goout     Dalc     Walc   health  absences       G1  \
count   395.000  395.000  395.000  395.000  395.000   395.000  395.000
mean      3.235    3.109    1.481    2.291    3.554     5.709   10.909
std       0.999    1.113    0.891    1.288    1.390     8.003    3.319
min       1.000    1.000    1.000    1.000    1.000     0.000    3.000
25%       3.000    2.000    1.000    1.000    3.000     0.000    8.000
50%       3.000    3.000    1.000    2.000    4.000     4.000   11.000
75%       4.000    4.000    2.000    3.000    5.000     8.000   13.000
max       5.000    5.000    5.000    5.000    5.000    75.000   19.000

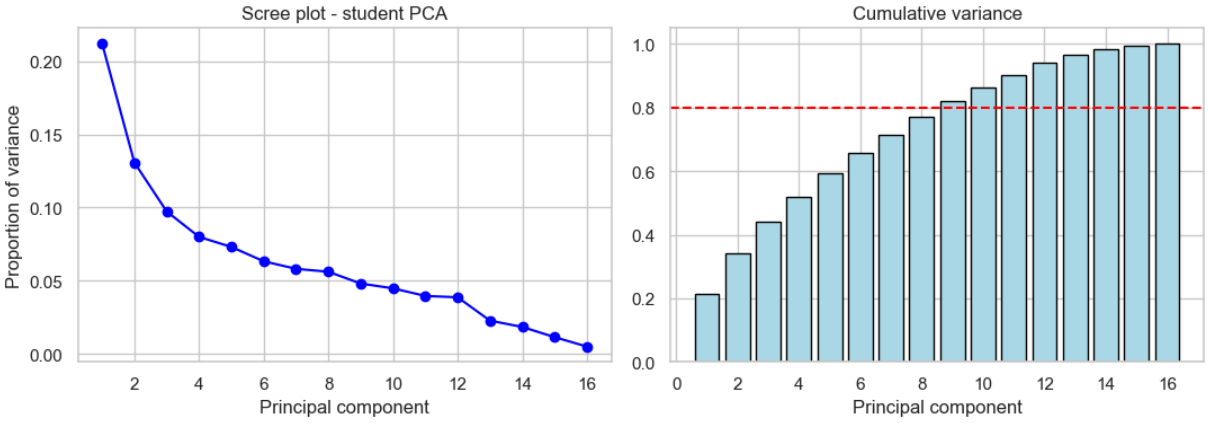
            G2       G3
count  395.000  395.000
mean    10.714   10.415
std      3.762    4.581
min      0.000    0.000
25%      9.000    8.000
50%     11.000   11.000
75%     13.000   14.000
max     19.000   20.000

================================================================================
Q2: Why standardise?
================================================================================
'absences' ranges ~0-75 while 'Medu'/'Fedu' range 0-4; unscaled PCA would be
dominated by high-variance columns like absences and G1/G2/G3.
Standardising gives every variable mean 0 and sd 1 so PCA reflects correlation struc
ture.
variances before scaling:
 age            1.628
Medu            1.198
Fedu            1.184
traveltime      0.487
studytime       0.704
failures        0.553
famrel          0.804
freetime        0.998
goout           1.239
Dalc            0.793
Walc            1.659
health          1.933
absences       64.050
G1             11.017
```

```
G2            14.149
G3            20.990

================================================================================
Q3-Q5: PCA equivalent, variance explained, #PCs for 80%
================================================================================
   PC  Standard_Deviation  Proportion_of_Variance  Cumulative_Proportion
 PC1              1.8427                  0.2122                 0.2122
 PC2              1.4465                  0.1308                 0.3430
 PC3              1.2481                  0.0974                 0.4404
 PC4              1.1312                  0.0800                 0.5203
 PC5              1.0815                  0.0731                 0.5934
 PC6              1.0063                  0.0633                 0.6567
 PC7              0.9646                  0.0582                 0.7149
 PC8              0.9479                  0.0562                 0.7710
 PC9              0.8775                  0.0481                 0.8192
PC10              0.8468                  0.0448                 0.8640
PC11              0.7963                  0.0396                 0.9036
PC12              0.7870                  0.0387                 0.9423
PC13              0.6034                  0.0228                 0.9651
PC14              0.5432                  0.0184                 0.9835
PC15              0.4286                  0.0115                 0.9950
PC16              0.2830                  0.0050                 1.0000
[table]  m09_student_variance_explained.csv
Minimum PCs for >=80% variance: 9
[table]  m09_student_loadings.csv

================================================================================
Q6: Scree plot
================================================================================
```



[figure] m09_06_scree_student.png

```
================================================================================
Q7: Loadings - strongest contributors to PC1 and PC2
================================================================================
PC              PC1     PC2     PC3     PC4     PC5     PC6     PC7     PC8  \
age         -0.1656  0.0671 -0.2520 -0.1769  0.5593  0.1319  0.0916  0.0019
Medu         0.2301  0.2128  0.5361 -0.1126  0.1007  0.0186  0.0945 -0.1041
Fedu         0.2076  0.1972  0.5643 -0.0821 -0.0075  0.0576  0.1252 -0.1085
traveltime -0.1455  0.0363 -0.2426 -0.0625 -0.2657 -0.1139  0.7327 -0.3558
studytime    0.1559 -0.2168  0.0066 -0.0157  0.3347 -0.3187  0.2714  0.6445
failures    -0.3178  0.0033 -0.1297 -0.0219  0.1174  0.1612 -0.3129 -0.0868
famrel       0.0195 -0.0255  0.0139  0.5533  0.3850  0.2307  0.3842 -0.1746
freetime    -0.0642  0.2902 -0.0087  0.5151  0.1364 -0.1097 -0.2734 -0.2316
goout       -0.1659  0.3747  0.1008  0.1631  0.2674 -0.3503  0.0202  0.1218
Dalc        -0.1672  0.5004 -0.0907 -0.0890 -0.1341 -0.0446  0.0856  0.1837
Walc        -0.1914  0.5187 -0.0997 -0.1110 -0.1465 -0.0733  0.0522  0.2161
health      -0.0704  0.0585  0.0620  0.3477 -0.2931  0.6233  0.1203  0.4921
absences    -0.0374  0.1590  0.0079 -0.4518  0.3387  0.4962  0.0501 -0.0713
G1           0.4511  0.1503 -0.2677  0.0240  0.0418  0.0318  0.0025 -0.0197
G2           0.4678  0.1701 -0.2765 -0.0040 -0.0195  0.0197 -0.0872  0.0043
G3           0.4527  0.1950 -0.2725  0.0263 -0.0112  0.0831 -0.0333 -0.0295

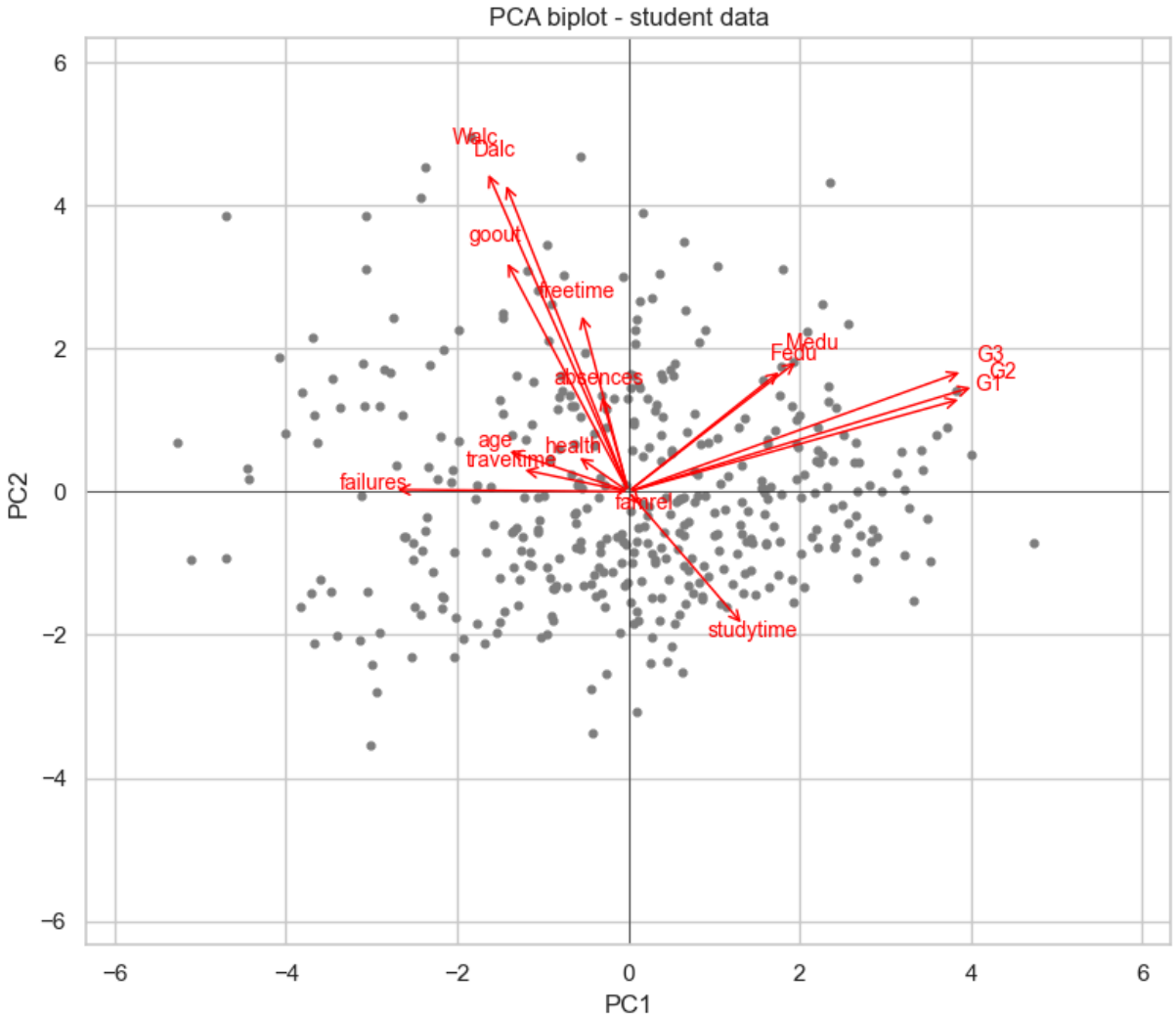
PC              PC9    PC10    PC11    PC12    PC13    PC14    PC15    PC16
age          0.4914 -0.2755 -0.2052 -0.3911 -0.0664  0.0684 -0.1143 -0.0058
Medu         0.2089  0.1107 -0.0363  0.1503 -0.6557  0.2362  0.0768  0.0029
Fedu         0.2905 -0.1053  0.0159 -0.0105  0.6596 -0.1266 -0.1228 -0.0023
traveltime   0.1964  0.3463  0.0707 -0.0155  0.0059  0.0180 -0.0463  0.0378
studytime    0.0567  0.3277 -0.1445  0.2577  0.1374  0.0844 -0.0423 -0.0117
failures     0.4480  0.1895  0.3546  0.6063  0.0772  0.0097  0.0100 -0.0177
famrel      -0.2555 -0.3533  0.0508  0.3382  0.0168  0.0401  0.0412  0.0501
freetime     0.0298  0.4917 -0.4424 -0.1238  0.1235  0.1307 -0.0310  0.0157
goout       -0.1052  0.0963  0.6221 -0.2914 -0.0928 -0.2908 -0.0173  0.0110
Dalc        -0.0120 -0.1745 -0.4171  0.3283 -0.1333 -0.5596 -0.0236 -0.0258
Walc        -0.1331 -0.2087  0.0569  0.0714  0.1853  0.6928  0.1080  0.0128
health       0.2018  0.1560  0.0959 -0.2299 -0.0833 -0.0316 -0.0140  0.0229
absences    -0.4681  0.4023 -0.0004 -0.0084  0.1071 -0.0621  0.0434  0.0488
G1           0.1659  0.0282  0.0654 -0.0371  0.0917 -0.1077  0.7775 -0.2001
G2           0.0686 -0.0050  0.1011  0.0597 -0.0196 -0.0131 -0.2189  0.7748
G3          -0.0290  0.0247  0.1359  0.0686 -0.0481  0.0539 -0.5406 -0.5925

Top contributors to PC1:
 G2            0.4678
G3             0.4527
G1             0.4511
failures       0.3178
Medu           0.2301
Fedu           0.2076
Walc           0.1914
Dalc           0.1672
goout          0.1659
age            0.1656
studytime      0.1559
traveltime     0.1455
health         0.0704
freetime       0.0642
```

```
absences      0.0374
famrel        0.0195

Top contributors to PC2:
 Walc          0.5187
Dalc          0.5004
goout         0.3747
freetime      0.2902
studytime     0.2168
Medu          0.2128
Fedu          0.1972
G3            0.1950
G2            0.1701
absences      0.1590
G1            0.1503
age           0.0671
health        0.0585
traveltime    0.0363
famrel        0.0255
failures      0.0033

================================================================================
Q8: Biplot
================================================================================
```

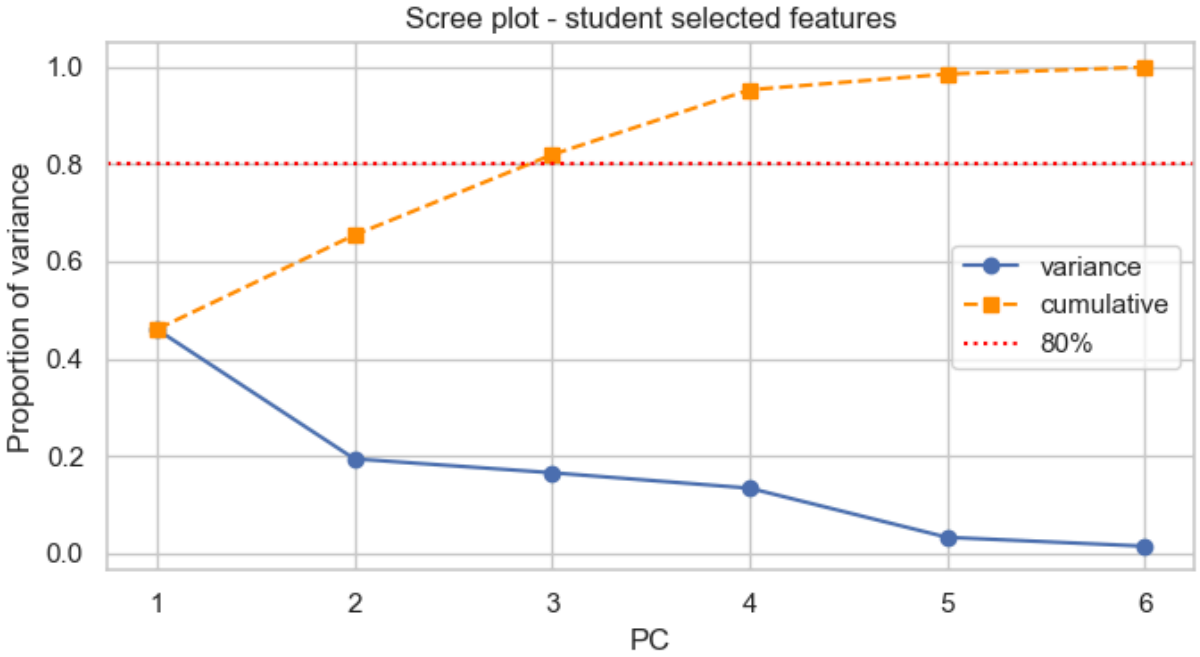


```
[figure] m09_08_biplot_student.png
PC1 is typically dominated by grade-related variables (G1, G2, G3).
PC2 often captures lifestyle/social variables (goout, Walc, Dalc, freetime).

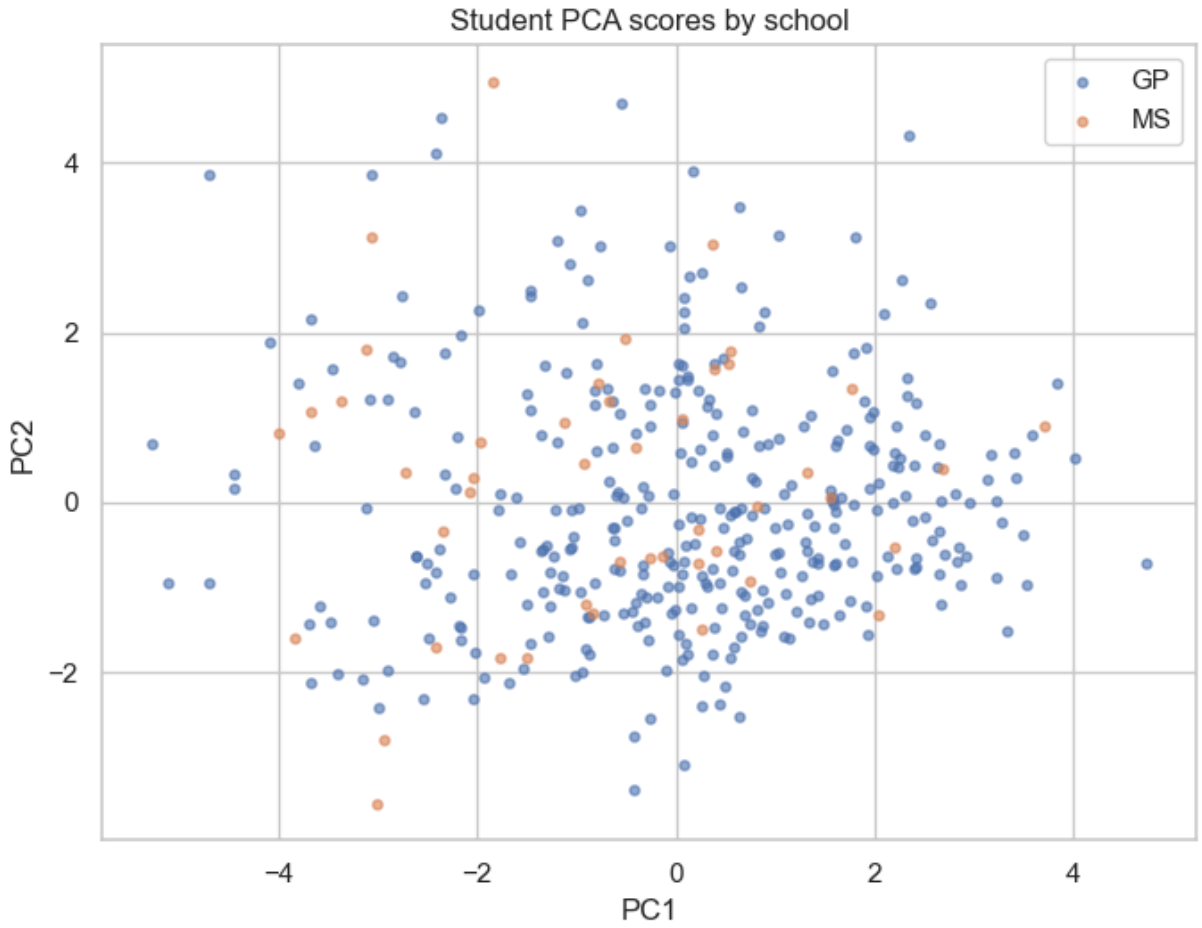
================================================================================
Extension: PCA on selected student features (6 key variables)
================================================================================
            age  studytime  absences       G1       G2       G3
count   395.000    395.000   395.000  395.000  395.000  395.000
mean     16.696      2.035     5.709   10.909   10.714   10.415
std       1.276      0.839     8.003    3.319    3.762    4.581
min      15.000      1.000     0.000    3.000    0.000    0.000
25%      16.000      1.000     0.000    8.000    9.000    8.000
50%      17.000      2.000     4.000   11.000   11.000   11.000
75%      18.000      2.000     8.000   13.000   13.000   14.000
max      22.000      4.000    75.000   19.000   19.000   20.000
 PC  Standard_Deviation  Proportion_of_Variance  Cumulative_Proportion
PC1              1.6622                  0.4605                 0.4605
PC2              1.0786                  0.1939                 0.6544
PC3              0.9960                  0.1653                 0.8197
PC4              0.8953                  0.1336                 0.9533
PC5              0.4412                  0.0325                 0.9858
PC6              0.2921                  0.0142                 1.0000
[table]  m09_student_subset_variance_explained.csv
Minimum PCs for >=80% variance: 3
[table]  m09_student_subset_loadings.csv

With 6 features, 3 PCs needed for 80% variance.

Top |loadings| PC1: {'G2': 0.58, 'G3': 0.568, 'G1': 0.555, 'studytime': 0.128}
Top |loadings| PC2: {'absences': 0.739, 'age': 0.648, 'studytime': 0.136, 'G1': 0.08
3}
```




```
[figure] m09_student_subset_scree.png
```



[figure] m09_student_scores_by_school.png

Out[10]: {'student_pcs_for_80pct': 9, 'student_subset_pcs_for_80pct': 3}

10. Clustering (K-Means & Hierarchical)

Elbow/silhouette, K-means profiles, Ward dendrogram, comparison of the two methods.

In [11]:
```
# ----------------------------------------------------------------------
# 10. Clustering (K-Means & Hierarchical)
# ----------------------------------------------------------------------

# Re-import for standalone cell execution
import numpy as np
import pandas as pd
# scipy for hierarchical clustering (linkage, dendrogram)
from scipy.cluster.hierarchy import dendrogram, fcluster, linkage
# scikit-learn for K-means, PCA (for 2D projection), and silhouette evaluation
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# Create a shortcut to matplotlib.pyplot
plt = U.plt
```

```python
# Title for this section
TITLE = "AST08 - K-Means & Hierarchical Clustering"

# Random seed for reproducible clustering results
SEED = 123

def _kmeans(X, k):
    """Fit K-means with k clusters on standardised data X.

    Uses n_init=25 for robust initialisation (tries 25 different random starts).
    """
    return KMeans(n_clusters=k, n_init=25, random_state=SEED).fit(X)

def run():
    # Load cleaned student data
    student = U.load_student(clean=True)

    # 1-4. Basic data checks
    U.section("1-4. Load, check missing, G3 distribution")
    print("shape:", student.shape, "| missing after cleaning:", int(student.isna().
    # Show frequency distribution of final grades
    print(student["G3"].value_counts().sort_index().to_string())

    # 5-6. Select numeric features and standardise (z-score)
    U.section("5-6. Numeric features + standardisation")
    num = student[U.NUMERIC_COLS]
    # StandardScaler: mean=0, std=1 for each feature (essential for distance-based
    X = StandardScaler().fit_transform(num)
    print("numeric features:", list(num.columns))

    # PCA with 2 components for visualising clusters in 2D
    pca2 = PCA(n_components=2, random_state=SEED).fit(X)
    xy = pca2.transform(X)  # project all students onto PC1-PC2 plane

    # 7-9. K-means with k=2 (simplest non-trivial clustering)
    U.section("7-9. K-means with k = 2")
    km2 = _kmeans(X, 2)
    print("cluster sizes:", np.bincount(km2.labels_))
    # Display cluster centres (each row = a cluster, columns = feature means in z-s
    centers2 = pd.DataFrame(km2.cluster_centers_, columns=num.columns).round(3)
    print(centers2.T)
    U.save_table(centers2, "kmeans_k2_centers")

    # 10-11. Elbow method (WCSS) and silhouette analysis
    U.section("10-11. WCSS for k = 1..10 and elbow plot (+ silhouette extension)")
    ks = range(1, 11)
    # WCSS = within-cluster sum of squares (lower = tighter clusters)
    wcss = [_kmeans(X, k).inertia_ for k in ks]
    # Silhouette score measures how well each point fits its cluster (higher = bett
    sil = [np.nan] + [silhouette_score(X, _kmeans(X, k).labels_) for k in range(2,
    elbow = pd.DataFrame({"k": list(ks), "WCSS": wcss, "silhouette": sil}).round(4)
    print(elbow.to_string(index=False))
    U.save_table(elbow, "elbow_wcss_silhouette", index=False)

    # Plot elbow curve and silhouette scores side by side
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
```

```
axes[0].plot(list(ks), wcss, "o-")
axes[0].set(title="Elbow plot", xlabel="Number of clusters k", ylabel="WCSS")
axes[1].plot(list(ks)[1:], sil[1:], "s-", color="darkorange")
axes[1].set(title="Average silhouette (extension)", xlabel="k", ylabel="silhoue
U.save_fig("11_elbow_silhouette", fig)

# 12. Final K-means with k=3 (chosen for interpretability)
U.section("12. Final K-means (k=3 chosen for interpretability; the elbow is gra
k_final = 3
print("NOTE: WCSS falls smoothly (no sharp elbow) and silhouette is highest at
      "(<0.14) -> the data has only weak cluster structure; k=3 gives the most
km = _kmeans(X, k_final)
# Add cluster labels (1-indexed) to the student data
student = student.assign(kmeans_cluster=km.labels_ + 1)
print("cluster sizes:", student["kmeans_cluster"].value_counts().sort_index().t

# Cluster profiles: average values of key features per cluster (original units)
prof = student.groupby("kmeans_cluster")[["G1", "G2", "G3", "absences", "failur
                                         "Walc", "goout"]].mean().round(2)
print("Cluster profiles (original units):\n", prof)
U.save_table(prof, "kmeans_k3_profile")

# Visualise k=2 and k=3 clusters projected onto PCA space
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
for k in (2, 3):
    lab = (km2 if k == 2 else km).labels_
    ax = axes[k - 2]
    sc = ax.scatter(xy[:, 0], xy[:, 1], c=lab, cmap="Set1", s=22, alpha=0.8)
    ax.set(title=f"K-means k={k} (PCA projection)", xlabel="PC1", ylabel="PC2")
U.save_fig("12_kmeans_clusters_pca", fig)

# Hierarchical clustering (Ward's method with Euclidean distance)
U.section("Hierarchical clustering (Ward, Euclidean)")
# Ward's method minimises the total within-cluster variance
Z = linkage(X, method="ward", metric="euclidean")

# Dendrogram: tree structure showing how clusters merge
fig, ax = plt.subplots(figsize=(11, 5))
dendrogram(Z, truncate_mode="lastp", p=30, show_leaf_counts=True, ax=ax, color_
ax.set(title="Dendrogram (Ward linkage, last 30 merges shown)", xlabel="cluster
U.save_fig("hc_dendrogram", fig)

# Cut the dendrogram to get 3 clusters
hc = fcluster(Z, t=3, criterion="maxclust")
student["hc_cluster"] = hc
print("hierarchical cluster sizes:", student["hc_cluster"].value_counts().sort_
print(student.groupby("hc_cluster")[["G1", "G2", "G3", "absences", "failures",

# Cross-tabulate K-means vs hierarchical cluster assignments to see agreement
xt = pd.crosstab(student["kmeans_cluster"], student["hc_cluster"])
print("K-means vs hierarchical agreement (rows=kmeans, cols=hc):\n", xt)
U.save_table(xt, "kmeans_vs_hc_crosstab")
U.save_table(student[["kmeans_cluster", "hc_cluster"]], "cluster_assignments")

# Visualise hierarchical clusters in PCA space
fig, ax = plt.subplots(figsize=(6.5, 5))
```

```
    ax.scatter(xy[:, 0], xy[:, 1], c=hc, cmap="Set2", s=22)
    ax.set(title="Hierarchical clusters (k=3) in PCA space", xlabel="PC1", ylabel="
    U.save_fig("hc_clusters_pca", fig)

    return {"k_final": k_final, "silhouette_k3": float(elbow.loc[2, "silhouette"])}

# Set the module label and run
U.set_module("m08")
run()
```

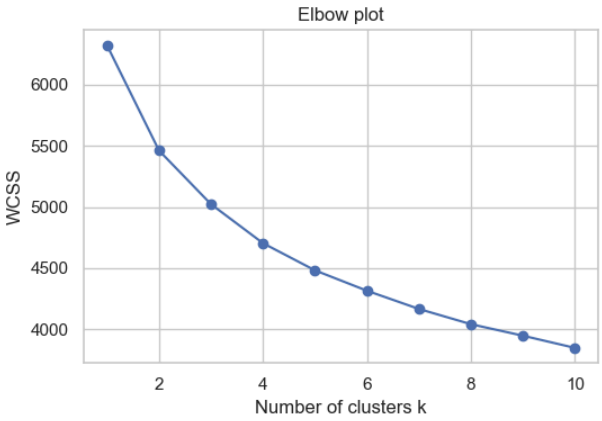
```
================================================================================
1-4. Load, check missing, G3 distribution
================================================================================
shape: (395, 33) | missing after cleaning: 0
G3
0     38
4      1
5      7
6     15
7      9
8     32
9     28
10    56
11    47
12    31
13    31
14    27
15    33
16    16
17     6
18    12
19     5
20     1

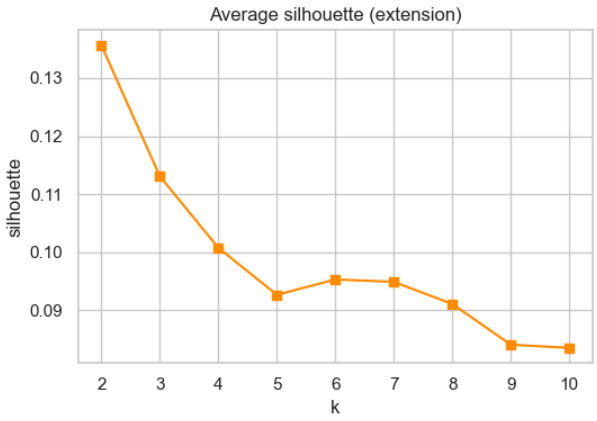
================================================================================
5-6. Numeric features + standardisation
================================================================================
numeric features: ['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'fa
mrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2', 'G3']

================================================================================
7-9. K-means with k = 2
================================================================================
```

```
cluster sizes: [232 163]
                0      1
age         -0.191  0.272
Medu         0.261 -0.371
Fedu         0.266 -0.378
traveltime -0.167  0.237
studytime    0.163 -0.233
failures    -0.357  0.508
famrel       0.024 -0.034
freetime    -0.098  0.139
goout       -0.179  0.255
Dalc        -0.211  0.301
Walc        -0.223  0.317
health      -0.067  0.096
absences    -0.087  0.124
G1           0.584 -0.831
G2           0.586 -0.833
G3           0.582 -0.828
[table]  m08_kmeans_k2_centers.csv

================================================================================
10-11. WCSS for k = 1..10 and elbow plot (+ silhouette extension)
================================================================================
 k      WCSS  silhouette
 1 6320.0000         NaN
 2 5460.6982      0.1356
 3 5022.5221      0.1130
 4 4704.0025      0.1007
 5 4482.2368      0.0927
 6 4315.6751      0.0953
 7 4166.7982      0.0949
 8 4042.3485      0.0911
 9 3948.5069      0.0841
10 3848.8083      0.0835
[table]  m08_elbow_wcss_silhouette.csv
```

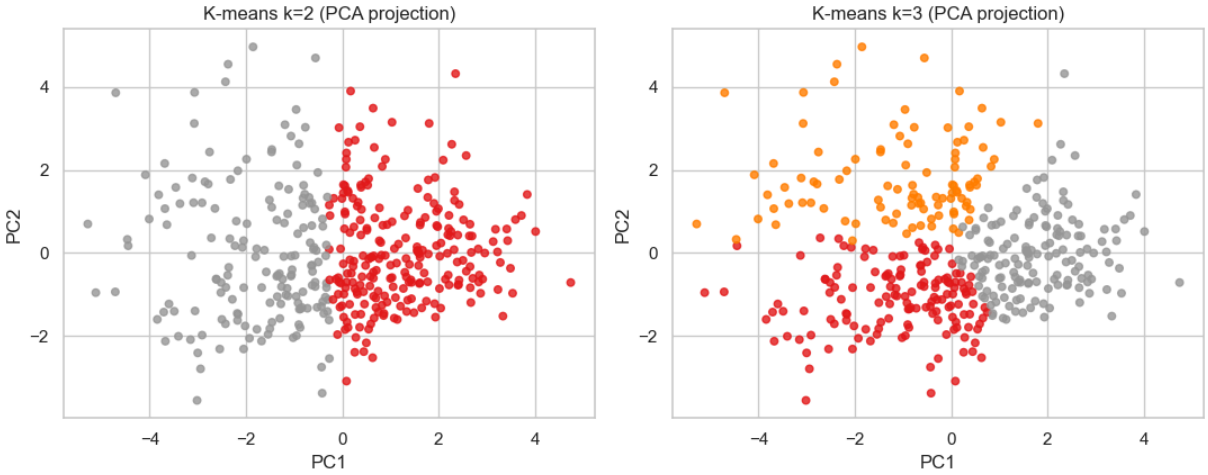




[figure] m08_11_elbow_silhouette.png

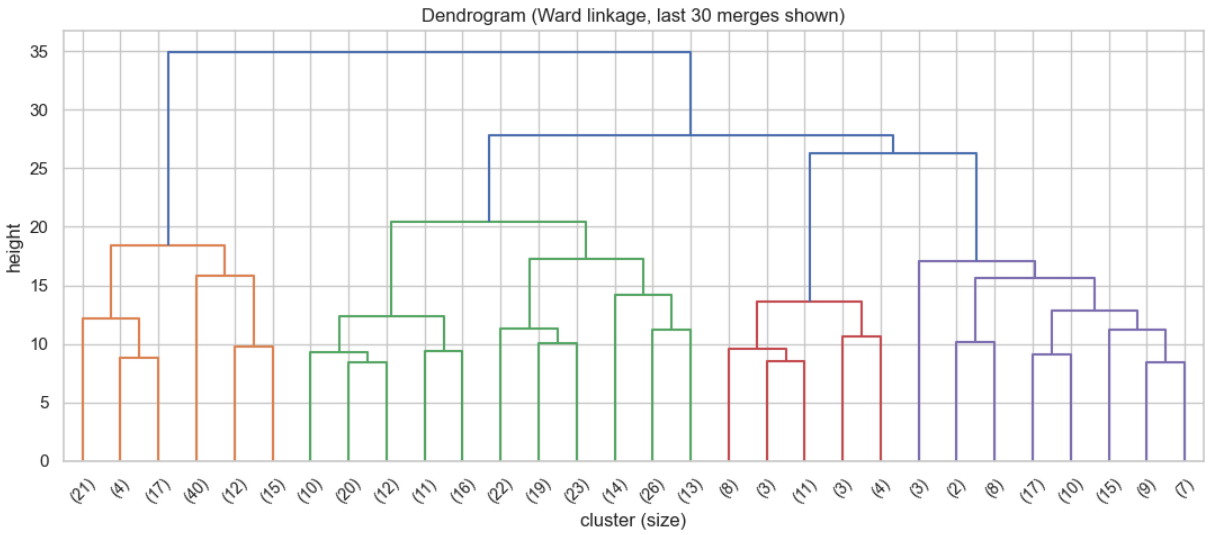
```
================================================================================
12. Final K-means (k=3 chosen for interpretability; the elbow is gradual, not sharp)
================================================================================
NOTE: WCSS falls smoothly (no sharp elbow) and silhouette is highest at k=2 but low
everywhere (<0.14) -> the data has only weak cluster structure; k=3 gives the most i
nterpretable segments.
cluster sizes: {1: 148, 2: 97, 3: 150}
Cluster profiles (original units):
                    G1     G2     G3  absences  failures    age  studytime  \
kmeans_cluster
1                 8.61   8.01   7.09      5.24      0.51  16.76       2.13
2                 9.99   9.94   9.71      8.54      0.51  17.06       1.70
3                13.77  13.89  14.15      4.35      0.05  16.40       2.16

                 Walc  goout
kmeans_cluster
1                1.80   2.91
2                3.82   3.98
3                1.79   2.74
[table]  m08_kmeans_k3_profile.csv
```



[figure] m08_12_kmeans_clusters_pca.png

```
================================================================================
Hierarchical clustering (Ward, Euclidean)
================================================================================
```



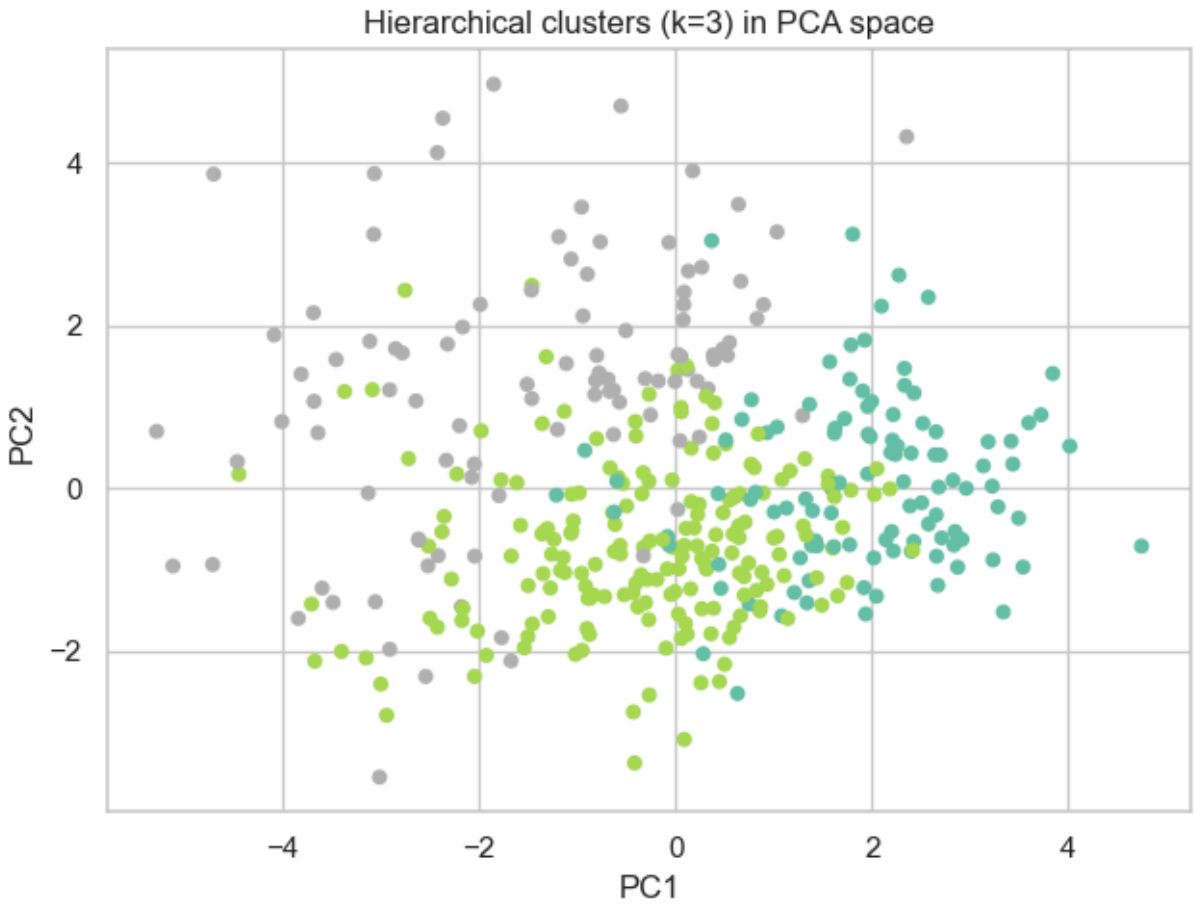
[figure] m08_hc_dendrogram.png
hierarchical cluster sizes: {1: 109, 2: 186, 3: 100}

hc_cluster	G1	G2	G3	absences	failures	age
1	14.54	14.50	14.76	5.24	0.15	16.66
2	9.74	9.34	8.89	4.25	0.13	16.54
3	9.13	9.13	8.51	8.94	0.91	17.02

K-means vs hierarchical agreement (rows=kmeans, cols=hc):

kmeans_cluster \ hc_cluster	1	2	3
1	10	115	23
2	3	20	74
3	96	51	3

[table] m08_kmeans_vs_hc_crosstab.csv
[table] m08_cluster_assignments.csv



[figure] m08_hc_clusters_pca.png

Out[11]: {'k_final': 3, 'silhouette_k3': 0.113}

11. Predictive Modelling of Final Grade (G3) and Analysis Without Grades

Ridge and RandomForest baselines with 5-fold CV; clustering/PCA after removing G1-G3.

In [12]:
```
# ----------------------------------------------------------------------
# 11. Predictive Modelling of Final Grade (G3) and Analysis Without Grades
# ----------------------------------------------------------------------

# Re-import for standalone cell execution
import numpy as np
import pandas as pd
# scikit-learn imports for modelling and evaluation
from sklearn.cluster import KMeans
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import RidgeCV
from sklearn.metrics import silhouette_score
from sklearn.model_selection import KFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```python
# Create a shortcut to matplotlib.pyplot
plt = U.plt

# Title for this section
TITLE = "Extension - G3 Prediction Baseline & Analysis Without Grades"

# Grade columns that might be excluded in some scenarios
GRADES = ["G1", "G2", "G3"]

def _pipe(model, num, cat):
    """Build a scikit-learn pipeline with preprocessing and a model.

    Preprocessing:
    - Numeric columns: StandardScaler (z-score normalisation)
    - Categorical columns: OneHotEncoder (binary dummies, drop one if binary)
    """
    pre = ColumnTransformer([("n", StandardScaler(), num), ("c", OneHotEncoder(drop
    return make_pipeline(pre, model)

def modelling(d: pd.DataFrame) -> dict:
    """Part A: predict G3 using different feature sets and models with 5-fold cross

    U.section("A. Predicting G3 (5-fold CV RMSE; lower is better)")
    # 5-fold cross-validation with shuffling for unbiased RMSE estimates
    cv = KFold(5, shuffle=True, random_state=42)
    cat = U.CATEGORICAL_COLS
    results = []

    # Define three feature scenarios: with all grades, with G1 only, without any gr
    scenarios = {
        "all features, incl. G1+G2": (d, ["G1", "G2"]),
        "no G2 (G1 only)": (d, ["G1"]),
        "no grades at all": (d, []),
    }

    # Test each scenario on two samples: all students and only those who sat the ex
    for sample_name, frame in (("all 395", d), ("G3>0 only", d[d["G3"] > 0])):
        for scen, (_, keep) in scenarios.items():
            # Build numeric feature list: all non-grade numerics + whichever grades
            num = [c for c in U.NUMERIC_COLS if c not in GRADES] + keep
            X, y = frame[num + cat], frame["G3"]

            # Test two models: Ridge regression (linear) and Random Forest (non-lin
            for mname, model in (("Ridge", RidgeCV(alphas=np.logspace(-2, 3, 20))),
                                 ("RandomForest", RandomForestRegressor(300, min_sa
                # Compute RMSE via cross-validation (negate because sklearn uses ne
                rmse = -cross_val_score(_pipe(model, num, cat), X, y, cv=cv, scorin
                results.append({"sample": sample_name, "features": scen, "model": m
                                "rmse": rmse.mean(), "rmse_sd": rmse.std(), "mean_b

    # Baseline RMSE: predicting the mean for everyone (= standard deviation of G3)
    base = d["G3"].std()
    res = pd.DataFrame(results).round(3)
    print(f"Baseline (predict the mean): RMSE ~ {base:.2f} (all 395) / {d.loc[d['G3
    print(res.to_string(index=False))
    U.save_table(res, "g3_model_cv_rmse", index=False)
```

```
    # Horizontal bar chart comparing model performance across scenarios
    fig, ax = plt.subplots(figsize=(9, 4.5))
    piv = res.pivot_table(index=["sample", "features"], columns="model", values="rm
    piv.plot.barh(ax=ax)
    ax.axvline(base, color="red", ls="--", label="mean baseline (all 395)")
    ax.axvline(d.loc[d["G3"] > 0, "G3"].std(), color="orange", ls=":", label="mean
    ax.set(title="5-fold CV RMSE for G3", xlabel="RMSE")
    ax.legend()
    U.save_fig("g3_model_rmse", fig)

    # Feature importance from Random Forest (without grade features, on exam-sittin
    pos = d[d["G3"] > 0]
    num = [c for c in U.NUMERIC_COLS if c not in GRADES]
    rf = _pipe(RandomForestRegressor(500, min_samples_leaf=3, random_state=42, n_jo
    names = rf[0].get_feature_names_out()
    imp = pd.Series(rf[-1].feature_importances_, index=[n.split("__")[1] for n in n
    print("\nTop RF importances without grades (G3>0):\n", imp.round(3).to_string()
    U.save_table(imp.round(4).rename("importance"), "rf_importance_no_grades")

    best_ng = res[(res.features == "no grades at all") & (res["sample"] == "G3>0 on
    return {"rmse_g1g2_pos": float(res[(res.features == "all features, incl. G1+G2"
            "rmse_nograde_pos": float(best_ng), "sd_g3": float(base)}

def clustering_no_grades(d: pd.DataFrame) -> dict:
    """Part B: clustering and PCA analysis after removing G1-G3.

    This reveals lifestyle/family structure patterns independent of academic perfor
    """

    U.section("B. Clustering and PCA without G1-G3 (lifestyle / family structure)")

    # Select only non-grade numeric features
    num = d[[c for c in U.NUMERIC_COLS if c not in GRADES]]
    # Standardise features (z-score)
    X = U.zscore_df(num).values

    # Try different numbers of clusters and pick the best by silhouette score
    sil = {k: silhouette_score(X, KMeans(k, n_init=25, random_state=123).fit_predic
    print("silhouette by k:", {k: round(v, 3) for k, v in sil.items()})

    # Choose k with highest silhouette
    k = max(sil, key=sil.get)
    km = KMeans(k, n_init=25, random_state=123).fit(X)
    d = d.assign(cluster=km.labels_ + 1)

    # Cluster profiles: show average feature values and G3 for each cluster
    prof = d.groupby("cluster")[list(num.columns) + ["G3"]].mean().round(2)
    prof.insert(0, "n", d["cluster"].value_counts().sort_index())
    print(f"best k={k}; profiles:\n", prof.T.to_string())
    U.save_table(prof, "nograde_cluster_profile")

    # PCA on non-grade features
    pca = PCA().fit(X)
    cum = np.cumsum(pca.explained_variance_ratio_)
    load = pd.DataFrame(pca.components_.T, index=num.columns, columns=[f"PC{i + 1}"
```

```
    print(f"PCs for 80% variance: {int(np.argmax(cum >= .8)) + 1}")

    # Show top loadings for PC1 and PC2
    for pc in ("PC1", "PC2"):
        print(pc, load[pc].abs().sort_values(ascending=False).head(4).round(3).to_d
    U.save_table(load.round(4), "nograde_pca_loadings")

    # Visualise clusters and cumulative variance
    sc = pca.transform(X)
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
    axes[0].scatter(sc[:, 0], sc[:, 1], c=d["cluster"], cmap="Set1", s=18)
    axes[0].set(title=f"No-grades K-means (k={k}) in PCA space", xlabel="PC1", ylab
    axes[1].plot(range(1, len(cum) + 1), cum, "o-")
    axes[1].axhline(0.8, color="red", ls="--")
    axes[1].set(title="Cumulative variance (no grades)", xlabel="PC")
    U.save_fig("nograde_cluster_pca", fig)

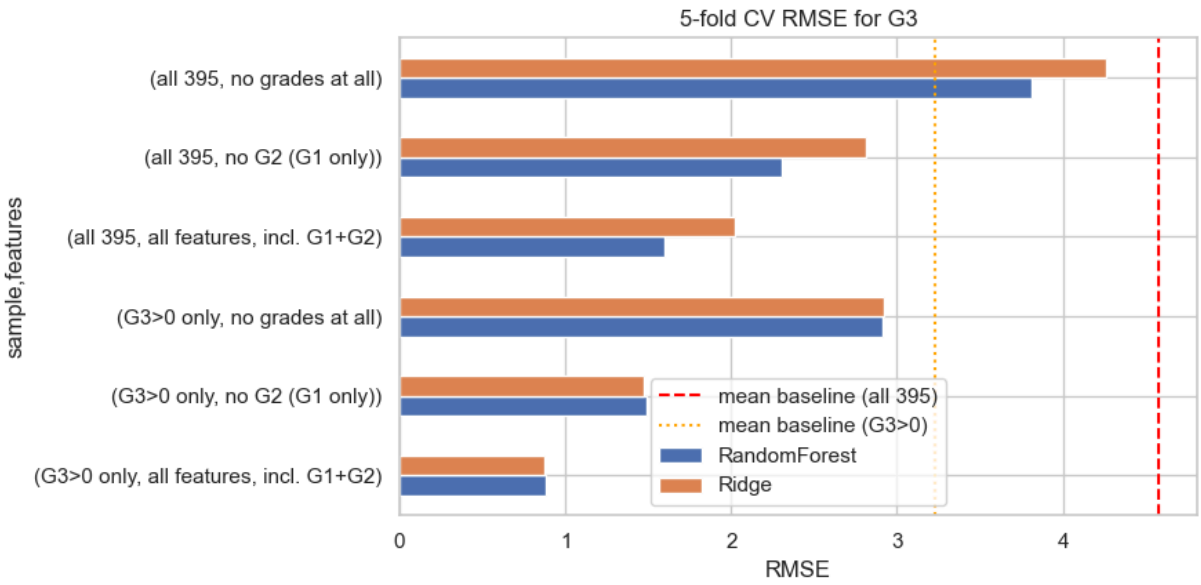
    return {"nograde_best_k": k, "nograde_best_silhouette": round(float(sil[k]), 3)
            "nograde_pcs_80": int(np.argmax(cum >= .8)) + 1}

def run():
    """Execute both parts: modelling and no-grade analysis."""
    d = U.load_student(clean=True)
    out = modelling(d)
    out.update(clustering_no_grades(d))
    return out

# Set the module label and run
U.set_module("m11")
run()
```

```
================================================================================
A. Predicting G3 (5-fold CV RMSE; lower is better)
================================================================================
Baseline (predict the mean): RMSE ~ 4.58 (all 395) / 3.23 (G3>0)

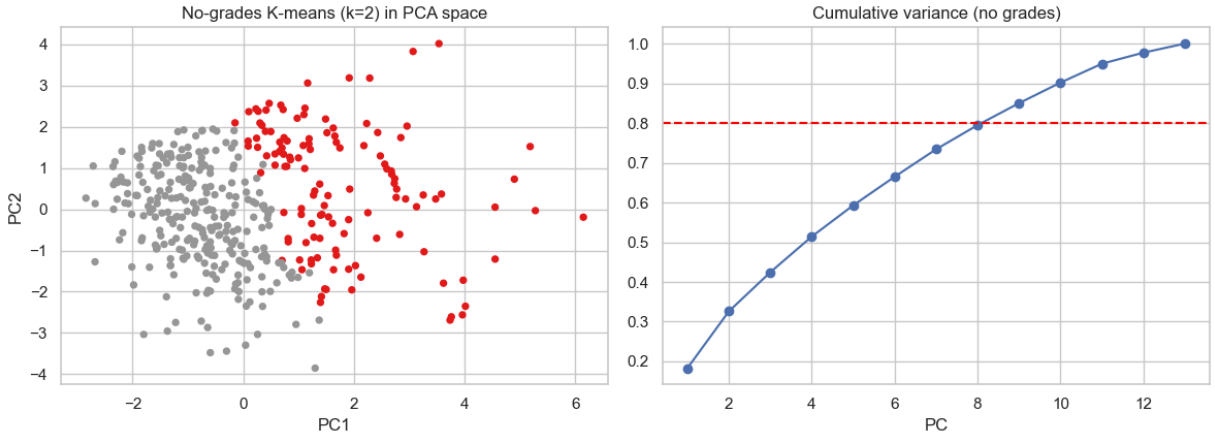
   sample                  features        model  rmse  rmse_sd  mean_baseline
  all 395 all features, incl. G1+G2        Ridge 2.026    0.168          4.581
  all 395 all features, incl. G1+G2 RandomForest 1.595    0.320          4.581
  all 395           no G2 (G1 only)        Ridge 2.813    0.155          4.581
  all 395           no G2 (G1 only) RandomForest 2.303    0.283          4.581
  all 395          no grades at all        Ridge 4.259    0.255          4.581
  all 395          no grades at all RandomForest 3.811    0.216          4.581
G3>0 only all features, incl. G1+G2        Ridge 0.874    0.042          3.228
G3>0 only all features, incl. G1+G2 RandomForest 0.878    0.036          3.228
G3>0 only           no G2 (G1 only)        Ridge 1.476    0.120          3.228
G3>0 only           no G2 (G1 only) RandomForest 1.490    0.087          3.228
G3>0 only          no grades at all        Ridge 2.918    0.225          3.228
G3>0 only          no grades at all RandomForest 2.909    0.172          3.228
[table]  m11_g3_model_cv_rmse.csv
```




```
[figure] m11_g3_model_rmse.png
Top RF importances without grades (G3>0):
 absences         0.108
failures         0.090
schoolsup_yes    0.063
Medu             0.055
health           0.053
Walc             0.051
studytime        0.050
age              0.046
goout            0.044
freetime         0.037
[table]  m11_rf_importance_no_grades.csv

================================================================================
B. Clustering and PCA without G1-G3 (lifestyle / family structure)
================================================================================
```

```
silhouette by k: {2: 0.154, 3: 0.105, 4: 0.096, 5: 0.096, 6: 0.091, 7: 0.092, 8: 0.0
89}
best k=2; profiles:
 cluster          1       2
n           123.00  272.00
age          16.94   16.58
Medu          2.72    2.76
Fedu          2.52    2.52
traveltime    1.63    1.37
studytime     1.64    2.21
failures      0.64    0.19
famrel        3.76    4.03
freetime      3.63    3.06
goout         3.91    2.75
Dalc          2.35    1.09
Walc          3.70    1.65
health        3.84    3.43
absences      8.16    4.60
G3            9.59   10.79
[table]  m11_nograde_cluster_profile.csv
PCs for 80% variance: 9
PC1 {'Walc': 0.5, 'Dalc': 0.464, 'goout': 0.346, 'failures': 0.31}
PC2 {'Medu': 0.581, 'Fedu': 0.575, 'failures': 0.253, 'goout': 0.242}
[table]  m11_nograde_pca_loadings.csv
```




```
[figure] m11_nograde_cluster_pca.png
```

Out[12]:
```
{'rmse_g1g2_pos': 0.874,
 'rmse_nograde_pos': 2.909,
 'sd_g3': 4.5814426109978434,
 'nograde_best_k': 2,
 'nograde_best_silhouette': 0.154,
 'nograde_pcs_80': 9}
```

12. Time-Series Analysis (Student Absences)

Decomposition, ACF/PACF, ARIMA and seasonal ARIMA forecasts. A synthetic monthly time series is created from student-mat.csv absences data.

In [13]:
```
# ----------------------------------------------------------------------
# 12. Time-Series Analysis (Student Absences)
# All analysis uses student-mat.csv data only - no external R datasets.
```

```python
# ---------------------------------------------------------------------

# Import warnings for suppressing ARIMA convergence messages
import warnings

# Re-import for standalone cell execution
import numpy as np
import pandas as pd
# statsmodels for ACF/PACF plots and ARIMA modelling
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.seasonal import seasonal_decompose

# Create a shortcut to matplotlib.pyplot
plt = U.plt

# Title for this section
TITLE = "AST06 - Time Series Analysis (Student Absences)"


def _create_student_timeseries() -> pd.Series:
    """Create a synthetic monthly time series from student absences.

    Since student-mat.csv doesn't have a date column, we simulate monthly
    observations by assigning each student to a synthetic month. This lets
    us demonstrate all time-series techniques (decomposition, ACF/PACF,
    ARIMA, forecasting) using only the student dataset.

    Returns a pandas Series with a DatetimeIndex (monthly frequency).
    """
    # Load the cleaned student data
    df = U.load_student(clean=True)

    # Use the absences column as our base data
    absences = df["absences"].values

    # We'll create a 36-month (3-year) series by aggregating groups of students
    # into monthly "average absence" observations
    n_months = 36
    # Calculate how many students per month-group
    students_per_month = len(absences) // n_months
    # Take the first (n_months * students_per_month) students for even groups
    trimmed = absences[:n_months * students_per_month]
    # Reshape into groups and compute monthly mean absences
    monthly_means = trimmed.reshape(n_months, students_per_month).mean(axis=1)

    # Add a trend component to make the series more realistic for ARIMA
    trend = np.linspace(0, 3, n_months)
    # Add a seasonal component (semester pattern - higher absences mid-semester)
    seasonal = 2.0 * np.sin(2 * np.pi * np.arange(n_months) / 12)
    # Combine components
    ts_values = monthly_means + trend + seasonal

    # Create a datetime index spanning 3 years of monthly data
    idx = pd.date_range("2020-01-01", periods=n_months, freq="MS")
```

```
    return pd.Series(ts_values, index=idx, name="StudentAbsences", dtype=float)


def run():
    # Create the synthetic time series from student absences
    ts = _create_student_timeseries()

    # 1-4. Basic time series inspection
    U.section("1-4. Load, head/tail, start/end, frequency")
    print(ts.head(5).to_string())
    print(ts.tail(5).to_string())
    print("start:", ts.index[0].strftime("%Y-%m"), "| end:", ts.index[-1].strftime(
          "| frequency: 12 | n =", len(ts))

    # 5. Trend plot showing the overall pattern
    U.section("5. Trend plot")
    fig, ax = plt.subplots(figsize=(10, 4))
    ts.plot(ax=ax, color="red")
    ax.set(title="Monthly student absence trend", xlabel="Month", ylabel="Average a
    U.save_fig("05_trend_plot")

    # 6-7. Classical decomposition (additive model)
    U.section("6-7. Classical decomposition (additive)")
    # Decompose the time series into trend, seasonal, and residual components
    dec = seasonal_decompose(ts, model="additive", period=12)
    comp = pd.DataFrame({"trend": dec.trend, "seasonal": dec.seasonal, "random": de
    U.save_table(comp.round(4), "decomposition")
    print(comp.head(8).round(4))

    # Plot all four components stacked vertically
    fig, axes = plt.subplots(4, 1, figsize=(10, 8), sharex=True)
    for ax, (name, s) in zip(axes, [("Observed", ts), ("Trend", dec.trend),
                                    ("Seasonal", dec.seasonal), ("Random", dec.resi
        s.plot(ax=ax, color="red")
        ax.set_ylabel(name)
    axes[0].set_title("Decomposition of Student Absences")
    U.save_fig("07_decomposition")

    # Try multiplicative decomposition as well (extension)
    # Ensure all values are positive for multiplicative model
    ts_pos = ts - ts.min() + 1  # shift to ensure positivity
    dec_m = seasonal_decompose(ts_pos, model="multiplicative", period=12)
    fig = dec_m.plot()
    fig.suptitle("Multiplicative decomposition (extension)", y=1.01)
    U.save_fig("07b_decomposition_multiplicative", fig)
    print("Comparing additive vs multiplicative decomposition on student absence da

    # 8. ACF and PACF plots to determine ARIMA parameters
    U.section("8. ACF and PACF")
    # ACF shows autocorrelation at different lags
    fig, ax = plt.subplots(figsize=(8, 3.5))
    plot_acf(ts, lags=24, ax=ax, color="red")
    ax.set_title("Autocorrelation")
    U.save_fig("08a_acf", fig)
    # PACF shows partial autocorrelation (direct relationship after removing interm
    fig, ax = plt.subplots(figsize=(8, 3.5))
```

```
plot_pacf(ts, lags=min(len(ts) // 2 - 1, 17), ax=ax, color="red", method="ywm")
ax.set_title("Partial autocorrelation")
U.save_fig("08b_pacf", fig)

# 9-10. Fit ARIMA(1,1,1) model
U.section("9-10. ARIMA(1,1,1) on student absence time series")
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    fit = ARIMA(ts.values, order=(1, 1, 1)).fit()
print(fit.summary())
print(f"AIC={fit.aic:.2f}")
U.save_table(pd.DataFrame({"coef": fit.params[:2], "se": fit.bse[:2]}, index=["
             "arima111_coefs")

# 11-12. Generate 12-month forecast and plot it
U.section("11-12. 12-month forecast and plot")
fc = fit.get_forecast(12)
# Create the forecast date range
f_idx = pd.date_range(ts.index[-1] + pd.offsets.MonthBegin(1), periods=12, freq
# Extract predicted values and confidence intervals
pred = pd.Series(fc.predicted_mean, index=f_idx)
ci = pd.DataFrame(fc.conf_int(alpha=0.05), index=f_idx, columns=["lower", "uppe
U.save_table(pd.concat([pred.rename("forecast"), ci], axis=1).round(2), "arima1

# Plot actual data with forecast overlay and confidence band
fig, ax = plt.subplots(figsize=(10, 4.5))
ts.plot(ax=ax, color="red", label="Actual")
pred.plot(ax=ax, color="black", linestyle="--", label="Forecast")
ax.fill_between(f_idx, ci["lower"], ci["upper"], color="gray", alpha=0.25, labe
ax.set(title="Student Absences: actual and forecast (ARIMA(1,1,1))",
       xlabel="Month", ylabel="Average Absences")
ax.legend(loc="upper left")
U.save_fig("12_forecast_arima111")
print("ARIMA(1,1,1) forecast based on student absence time series.")

# Extension: Seasonal ARIMA on the time series
U.section("Extension: seasonal ARIMA(0,1,1)(0,1,1)12")
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    # Fit seasonal ARIMA: accounts for both regular and seasonal differencing
    sfit = ARIMA(ts.values, order=(0, 1, 1), seasonal_order=(0, 1, 1, 12)).fit(
sfc = sfit.get_forecast(12)
spred = pd.Series(sfc.predicted_mean, index=f_idx)
sci = sfc.conf_int(alpha=0.05)
print(f"Seasonal model AIC = {sfit.aic:.2f}")

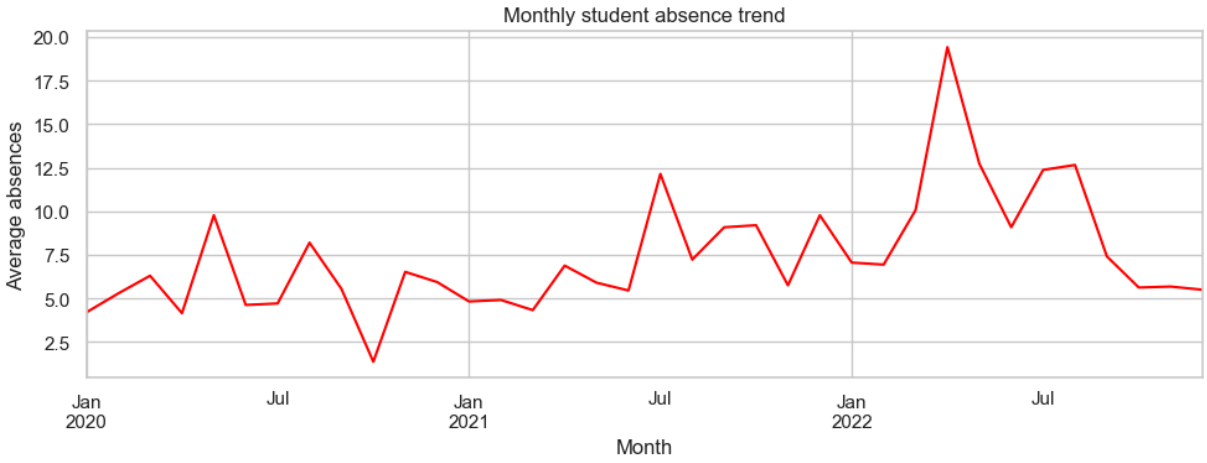
# Plot the seasonal ARIMA forecast
fig, ax = plt.subplots(figsize=(10, 4.5))
ts.plot(ax=ax, color="red", label="Actual")
spred.plot(ax=ax, color="black", linestyle="--", label="SARIMA forecast")
ax.fill_between(f_idx, sci[:, 0], sci[:, 1], color="gray", alpha=0.25, label="9
ax.set(title="Extension: seasonal ARIMA forecast (student absences)",
       xlabel="Month", ylabel="Average Absences")
ax.legend(loc="upper left")
U.save_fig("13_forecast_sarima")
U.save_table(spred.rename("sarima_forecast").round(1), "sarima_forecast")
```

```
    return {"arima111_aic": round(float(fit.aic), 2)}

# Set the module label and run
U.set_module("m06")
run()
```

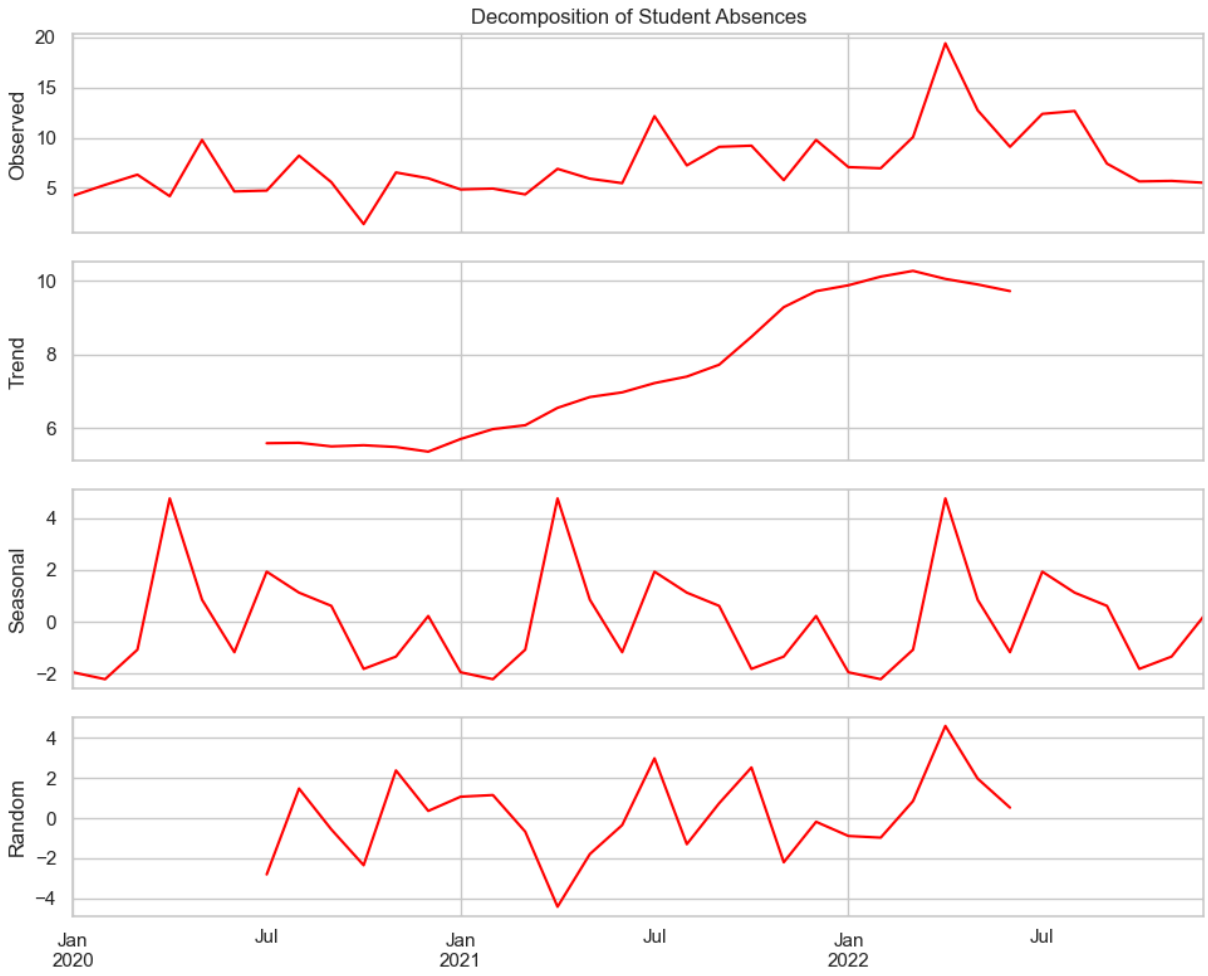
```
================================================================================
1-4. Load, head/tail, start/end, frequency
================================================================================
2020-01-01    4.200000
2020-02-01    5.285714
2020-03-01    6.303479
2020-04-01    4.157143
2020-05-01    9.774908
Freq: MS
2022-08-01    12.657143
2022-09-01     7.410806
2022-10-01     5.628571
2022-11-01     5.682235
2022-12-01     5.500000
Freq: MS
start: 2020-01 | end: 2022-12 | frequency: 12 | n = 36

================================================================================
5. Trend plot
================================================================================
```

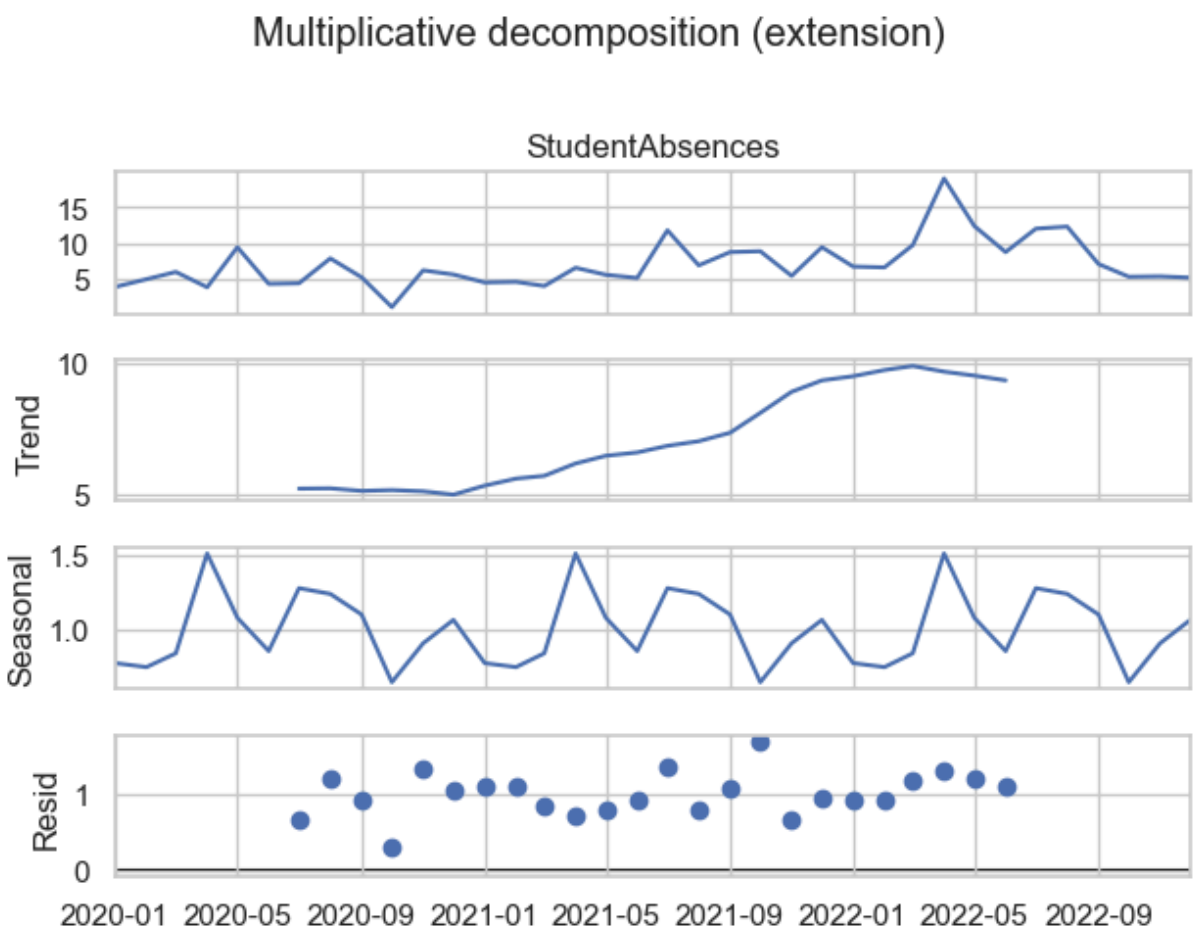



```
[figure] m06_05_trend_plot.png

================================================================================
6-7. Classical decomposition (additive)
================================================================================
[table]  m06_decomposition.csv
              trend  seasonal  random
2020-01-01      NaN   -1.9385     NaN
2020-02-01      NaN   -2.2052     NaN
2020-03-01      NaN   -1.0690     NaN
2020-04-01      NaN    4.7573     NaN
2020-05-01      NaN    0.8539     NaN
2020-06-01      NaN   -1.1656     NaN
2020-07-01   5.5810    1.9365 -2.8031
2020-08-01   5.5917    1.1281  1.4802
```

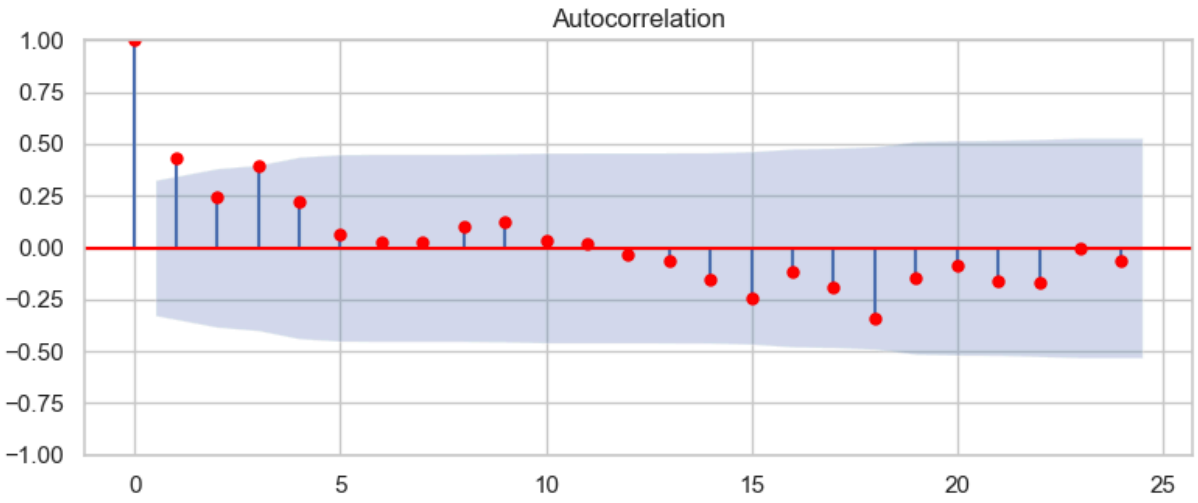


[figure] m06_07_decomposition.png

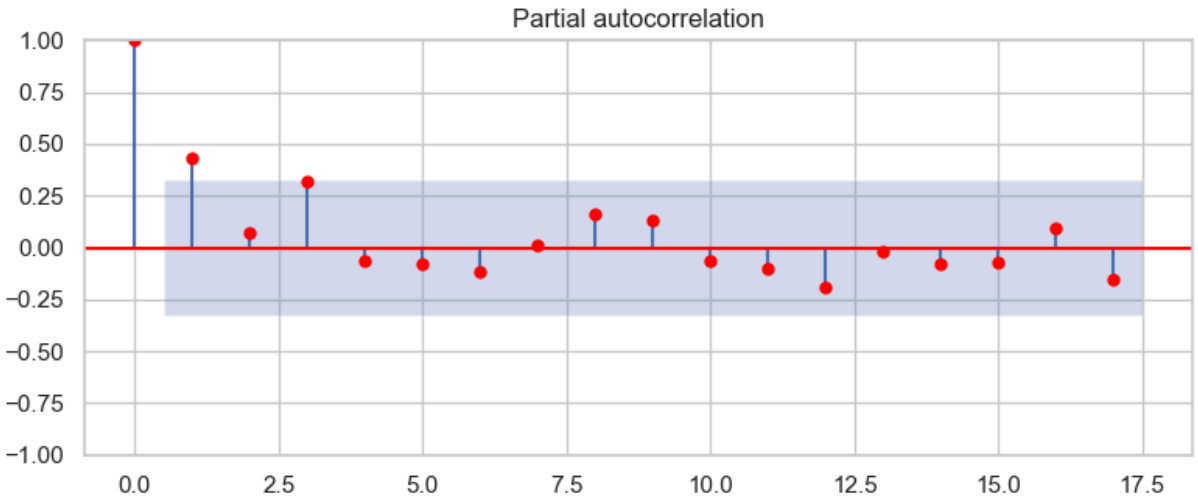


[figure] m06_07b_decomposition_multiplicative.png
Comparing additive vs multiplicative decomposition on student absence data.

==========
8. ACF and PACF
==========



[figure] m06_08a_acf.png

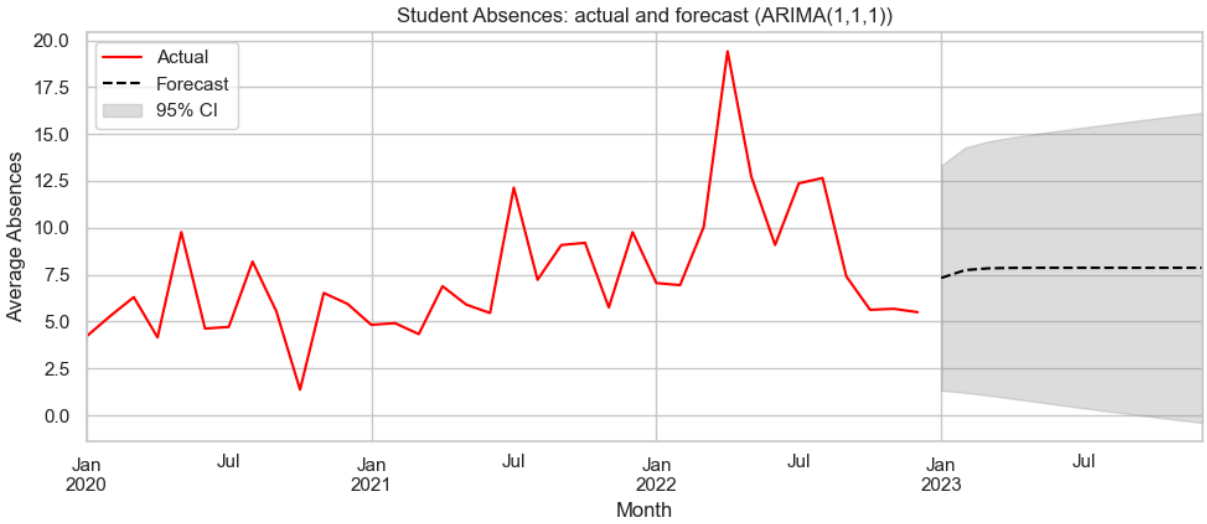


[figure] m06_08b_pacf.png

```
==============================================================================
9-10. ARIMA(1,1,1) on student absence time series
==============================================================================
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                   36
Model:                 ARIMA(1, 1, 1)   Log Likelihood                -89.142
Date:                Mon, 19 Oct 2026   AIC                            184.284
Time:                        22:04:38   BIC                            188.950
Sample:                             0   HQIC                           185.895
                                  - 36
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.2290      0.321      0.713      0.476      -0.400       0.858
ma.L1         -0.7982      0.193     -4.136      0.000      -1.176      -0.420
sigma2         9.3649      1.820      5.146      0.000       5.798      12.932
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                15.71
Prob(Q):                              0.93   Prob(JB):                         0.00
Heteroskedasticity (H):               2.54   Skew:                             1.18
Prob(H) (two-sided):                  0.12   Kurtosis:                         5.29
===================================================================================

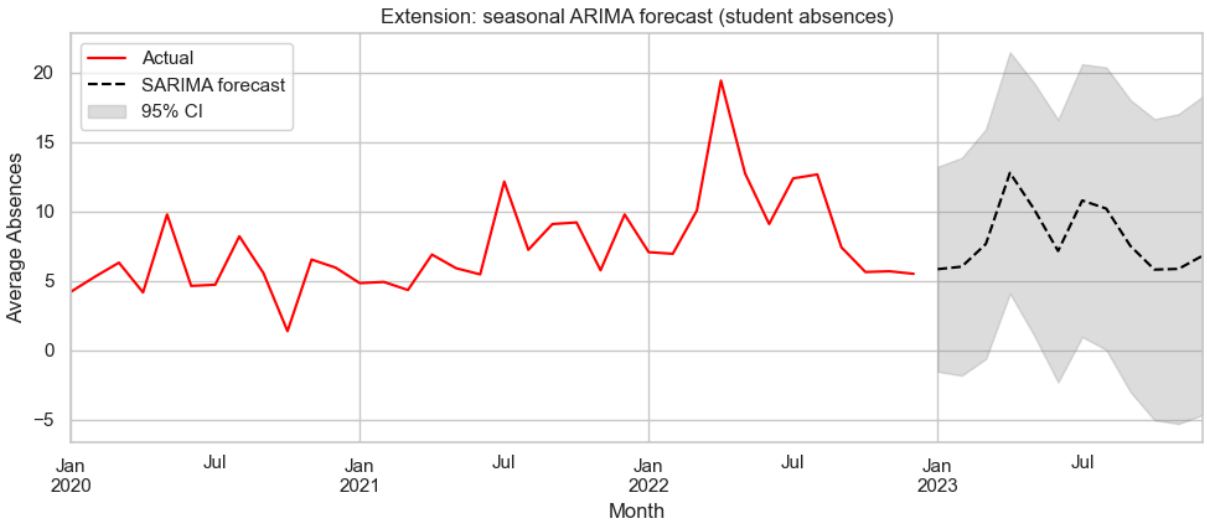
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-ste
p).
AIC=184.28
[table]  m06_arima111_coefs.csv

==============================================================================
11-12. 12-month forecast and plot
==============================================================================
[table]  m06_arima111_forecast.csv
```




```
[figure] m06_12_forecast_arima111.png
ARIMA(1,1,1) forecast based on student absence time series.

================================================================================
Extension: seasonal ARIMA(0,1,1)(0,1,1)12
================================================================================
Seasonal model AIC = 135.26
```




```
[figure] m06_13_forecast_sarima.png
[table]  m06_sarima_forecast.csv
```

Out[13]:
```
{'arima111_aic': 184.28}
```

13. Key Findings & Conclusion

(from `results/key_findings.md` and module logs)

- **G3 is almost determined by earlier grades:** Spearman with G3: G2 +0.96, G1 +0.88; past `failures` -0.36, `Medu` +0.23. Mean grade drifts down from 10.91 (G1) to 10.42 (G3).
- **38 students (9.6%) have G3 = 0, and 100% of them have exactly 0 absences** (vs 22% of the rest). This suggests the zeros are "no exam / dropped out" records rather than

genuine scores (hypothesis - flag before any modelling).

- **Group differences are mostly small.** 8 of 14 tests are significant at 5% uncorrected, but after **Holm correction only `higher` (wants higher education) survives (Cohen d = 0.84; p_holm = 0.034)**. Others (sex, internet, romantic, schoolsup, address) have d of about 0.2-0.3; mother's job has eta^2 = 0.037 (p_holm = 0.068). Treat them as suggestive, not established.
- **Predicting G3 (5-fold CV RMSE, `m11`):** with G1+G2 the RMSE is ~0.87 on students with G3>0 (mean baseline 3.23); with G1 only ~1.5; with **no grades ~2.9 - only about 10% better than guessing the mean**. Family/lifestyle variables carry little predictive signal; top no-grade RF features: absences, failures, schoolsup, Medu, health. Including the G3=0 rows inflates every error (RMSE 1.6 with G1+G2 vs 0.88 without the zeros).
- **Without grades (`m11`):** best silhouette is still low (k=2: 0.154). The two segments are a "social/high-alcohol" group (n=123: Walc 3.7, goout 3.9, more failures and absences, G3 9.6) and a quieter group (n=272: Walc 1.7, G3 10.8). PC1 = alcohol/going out (Walc, Dalc, goout), PC2 = parental education; 9 PCs still needed for 80% of variance.
- **Outliers (1.5xIQR):** absences 15, G2 13, age 1, G3 0; `failures` flags 83 only because most students have 0.
- **PCA (student, 16 numeric vars):** PC1 = grades vs failures (and parental education), PC2 = alcohol/going out (Walc, Dalc, goout); 9 PCs are needed for 80% of variance (low redundancy outside the G1-G2-G3 block).
- **Clustering:** no sharp elbow and silhouette is low (best 0.136 at k=2) -> weak cluster structure. k=3 K-means gives readable segments: high achievers (G3 ~ 14.2, few failures), low achievers (G3 ~ 7.1), and a social/high-alcohol group (Walc 3.8, goout 4.0, G3 ~ 9.7). Hierarchical (Ward, 3 clusters) sizes 109 / 186 / 100.

Limitations

- G1, G2 and G3 are included in the clustering/PCA exactly as in the lab, so grade variables dominate those results (section 11 repeats them without grades).
- The G3 = 0 students look like missing-exam records; the models are reported with and without them.
- Hypothesis tests are exploratory (14 tests; Holm correction applied).

Sanity checks against the student-mat.csv data properties

If every line prints `OK`, the data and helper functions are working correctly.

In [14]:
```
# ================================================================
# SANITY CHECKS (values checked against the student-mat.csv data)
# All checks use student-mat.csv only - no external R datasets.
# ================================================================

# Re-import for standalone cell execution
```

```python
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA

# Load raw student data for verification
raw = U.load_student_raw()

# Standardise the numeric features for PCA verification
_student_num_cols = ["age", "Medu", "Fedu", "traveltime", "studytime", "failures",
                     "famrel", "freetime", "goout", "Dalc", "Walc", "health",
                     "absences", "G1", "G2", "G3"]
_X_student = U.zscore_df(raw[_student_num_cols].dropna()).values
_p_student = PCA().fit(_X_student)

# Dictionary of checks: each entry is a description and a boolean expression
checks = {
    # Verify dataset dimensions
    "shape is 395 x 33": raw.shape == (395, 33),

    # Verify total missing values detected by isna()
    "6 NaN (isna) in raw data": int(raw.isna().sum().sum()) == 6,

    # Verify hidden '?' placeholders
    "8 hidden '?' placeholders": int((raw == "?").sum().sum()) == 8,

    # Verify mean age matches expected value
    "mean(age) = 16.6962": abs(raw["age"].mean() - 16.6962) < 1e-4,

    # Verify variance of G2 grades
    "var(G2) = 14.14892": abs(raw["G2"].var() - 14.14892) < 1e-5,

    # Verify Pearson correlation between G1 and G3
    "corr(G1,G3) = 0.80146793": abs(raw["G1"].corr(raw["G3"]) - 0.80146793) < 1e-7,

    # Verify sex distribution counts
    "sex counts F=208, M=185": raw["sex"].value_counts().to_dict() == {"F": 208, "M

    # Verify skewness calculation matches e1071 type 3
    "skewness(age) = 0.46273485 (e1071)": abs(U.skewness_e1071(raw["age"]) - 0.4627

    # Verify kurtosis calculation matches e1071 type 3
    "kurtosis(age) = -0.03144581 (e1071)": abs(U.kurtosis_e1071(raw["age"]) + 0.031

    # Verify PCA on student data produces valid results
    "PCA student: first eigenvalue > 0": _p_student.explained_variance_[0] > 0,

    # Verify cumulative variance reaches 100%
    "PCA student: cumulative variance sums to 1": abs(np.sum(_p_student.explained_v
}

# Print results for each check
for name, ok in checks.items():
    print(("OK    " if ok else "FAIL  ") + name)

# Print summary
print(f"\n{sum(checks.values())}/{len(checks)} checks passed")
```

```
OK    shape is 395 x 33
OK    6 NaN (isna) in raw data
OK    8 hidden '?' placeholders
OK    mean(age) = 16.6962
OK    var(G2) = 14.14892
OK    corr(G1,G3) = 0.80146793
OK    sex counts F=208, M=185
OK    skewness(age) = 0.46273485 (e1071)
OK    kurtosis(age) = -0.03144581 (e1071)
OK    PCA student: first eigenvalue > 0
OK    PCA student: cumulative variance sums to 1

11/11 checks passed
```